

```

<!DOCTYPE html>

<html lang="en">
<head>
<meta charset="utf-8"/>
<meta content="width=device-width, initial-scale=1.0" name="viewport"/>
<title>CycleStudio</title>
<link href="https://fonts.googleapis.com" rel="preconnect"/>
<link crossorigin="" href="https://fonts.gstatic.com" rel="preconnect"/>
<link
href="https://fonts.googleapis.com/css2?family=Quicksand:wght@400;500;600;700&dis
isplay=swap" rel="stylesheet"/>
<link href="assets/studio.css" rel="stylesheet"/>

<!-- SylCycle cloud: Firebase + shared auth -->
<script
src="https://www.gstatic.com/firebasejs/10.14.1/firebase-app-compat.js"></script>
<script
src="https://www.gstatic.com/firebasejs/10.14.1/firebase-auth-compat.js"></script>
<script
src="https://www.gstatic.com/firebasejs/10.14.1/firebase-firestore-compat.js"></scr
ipt>
<script src="sylcycle-auth.js"></script>

</head>
<body class="studio-booting">
<div class="app-shell">
<div class="topbar">
<div class="brand">
<div class="brand-badge"></div>
<div>
<h1>CycleStudio</h1>
<p>Cycle resource builder with editable preview, curriculum tagging and PDF
export.</p>
</div>
</div>
<div class="topbar-right">
<div class="chip">Built by Syl</div>
</div>
</div>
<div class="layout">
<div class="panel">
<div class="panel-scroll">
<div class="section">
<div class="section-head">
<div>
<h3 class="section-title"></h3>
<p class="section-sub"></p>
</div>

```

```

</div>
<div class="field">
<label for="apiKey">API key</label>
<input id="apiKey" placeholder="Paste your API key" type="password"/>
<div class="tiny" style="margin-top:6px"></div>
</div>
<div class="field">
<label for="modelName">Model ID (optional)</label>
<input id="modelName" placeholder="Leave blank to use the default model"
type="text" value=""/>
</div>
<select aria-hidden="true" class="hidden" id="outputMode">
<option selected="" value="student">Student sheet</option>
<option value="both">Student sheet + answer key</option>
</select>
</div>
<div class="section">
<div class="section-head">
<div>
<h3 class="section-title"></h3>
<p class="section-sub"></p>
</div>
</div>
<div class="field">
<label for="studioCycleSelect">Cycle</label>
<select id="studioCycleSelect"></select>
</div>
<div class="field">
<label for="studioYearLevel">Year level</label>
<select id="studioYearLevel"></select>
</div>
<div class="field">
<label>Curriculum mode</label>
<div class="segmented" style="grid-template-columns:repeat(3,1fr)">
<button class="seg-btn" id="currMode_ai_both"
onclick="setCurriculumMode('ai_both')" type="button">AI chooses domain +
LOs</button>
<button class="seg-btn" id="currMode_manual_domain_ai_lo"
onclick="setCurriculumMode('manual_domain_ai_lo')" type="button">Teacher chooses
domain, AI chooses LOs</button>
<button class="seg-btn" id="currMode_manual_both"
onclick="setCurriculumMode('manual_both')" type="button">Teacher chooses domain +
LOs</button>
</div>
<div class="tiny" style="margin-top:6px"></div>
</div>
<div class="hidden" id="domainArea" style="margin-top:12px">
<div class="field">
<label for="domainSelect"><span id="studioDomainCycleLabel">Cycle 3</span>
domain</label>

```

```

<select id="domainSelect"></select>
</div>
</div>
<div class="hidden" id="manualCurriculumArea" style="margin-top:12px">
<div class="field">
<label>Learning outcomes</label>
<div class="outcomes-box" id="outcomesBox"></div>
<div class="tiny" style="margin-top:6px"></div>
</div>
</div>
<div class="field" style="margin-top:12px">
<label for="worksheetBrief">Worksheet brief</label>
<textarea id="worksheetBrief" placeholder="Describe the worksheet you want: topic,
text type, skill focus, success criteria, number of questions, tone, image use,
anything else useful."></textarea>
<div class="tiny" style="margin-top:6px"></div>
</div>
<div class="field" style="margin-top:12px">
<label for="promptImageUpload">Source inspiration files</label>
<div aria-label="Source inspiration upload panel. Click here and press Control or
Command V to paste an image." class="upload upload-paste-panel"
id="promptImagePanel" tabindex="0">
<input
accept="image/*,.pdf,.pptx,application/pdf,application/vnd.openxmlformats-officedoc
ument.presentationml.presentation" class="hidden-file-input" id="promptImageUpload"
multiple type="file"/>
<div class="custom-file-row">
<button class="small-btn" onclick="qs('promptImageUpload')?.click()"
type="button">Choose images / PDF / PPTX</button>
<span class="tiny" id="promptImageFileStatus">No source files added.</span>
</div>
<div class="tiny" style="margin-top:8px">Upload or paste screenshots, or add a
PDF/PPTX lesson plan or slideshow. CycleStudio will use these as inspiration and
source material when generating the worksheet.</div>
<div class="tiny" style="margin-top:6px">PPTX slide text is extracted in the
browser. If the slideshow is mostly visual, export it as a PDF first for better AI
reading.</div>
<div class="clipboard-tools">
<button class="clipboard-btn" id="pastePromptImageBtn" type="button">Paste image
from clipboard</button>
<div class="tiny">You can also click here and press Ctrl/Cmd + V to paste a
screenshot.</div>
</div>
<div class="thumb-list" id="promptImageThumbs"></div>
</div>
</div>
<div class="grid-2" style="margin-top:12px">
<div class="field">
<label for="difficulty">Difficulty</label>
<select id="difficulty">

```

```

<option value="cm1_easy">CM1 - Easy</option>
<option selected="" value="cm1_moderate">CM1 - Moderate</option>
<option value="cm1_challenging">CM1 - Challenging</option>
<option value="cm1_advanced">CM1 - Advanced</option>
<option value="cm2_easy">CM2 - Easy</option>
<option value="cm2_moderate">CM2 - Moderate</option>
<option value="cm2_challenging">CM2 - Challenging</option>
<option value="cm2_advanced">CM2 - Advanced</option>
</select>
</div>
<div class="field">
<label for="questionCount">Target number of tasks</label>
<select id="questionCount">
<option value="1">1 task</option>
<option value="2">2 tasks</option>
<option value="3">3 tasks</option>
<option value="4">4 tasks</option>
<option value="5">5 tasks</option>
<option selected="" value="6">6 tasks</option>
<option value="7">7 tasks</option>
<option value="8">8 tasks</option>
<option value="ai">AI suggested</option>
</select>
</div>
</div>
<div class="field" style="margin-top:12px">
<label for="answerGenMode">Answer key generation</label>
<select id="answerGenMode">
<option value="student_only">Student sheet only for now</option>
<option value="with_answers">Generate answer key now</option>
</select>
<div class="tiny" style="margin-top:6px">Use student-only generation to save
tokens. You can generate the answer key later from the finished worksheet.</div>
</div>
<div class="section">
<div class="section-head">
<div>
<h3 class="section-title"></h3>
<p class="section-sub">Upload images or add a PDF template.</p>
</div>
</div>
<div class="field">
<label for="imageUpload">Worksheet images</label>
<div class="upload">
<input accept="image/*" id="imageUpload" multiple="" type="file"/>
<div class="tiny" style="margin-top:8px">Multiple images are supported. You can
also paste a screenshot straight from your clipboard.</div>
<div class="clipboard-tools">
<button class="clipboard-btn" id="pasteImageBtn" type="button">Paste from

```

```

clipboard</button>
<div class="tiny">Use Ctrl/Cmd + V in the box below.</div>
</div>
<div aria-label="Paste image from clipboard" class="paste-zone" id="pasteImageZone"
tabindex="0">
<strong>Paste screenshot here</strong>
Click here, then press Ctrl/Cmd + V to paste a screenshot.
</div>
<div class="thumb-list" id="imageThumbs"></div>
</div>
</div>
<div class="field">
<label for="templatePdf">Template PDF</label>
<div class="upload">
<input accept=".pdf" id="templatePdf" type="file"/>
<div class="tiny" id="templatePdfInfo" style="margin-top:8px">No PDF loaded.</div>
</div>
</div>
<div class="field">
<label for="templateText">Template notes or pasted text</label>
<textarea id="templateText" placeholder="Paste worksheet text, structural notes, or
instructions for what should be amended."></textarea>
</div>
</div>
<div class="section">
<div class="section-head">
<div>
<h3 class="section-title"></h3>
<p class="section-sub"></p>
</div>
</div>
<div class="tiny" style="margin-bottom:10px">Choose how you want to begin this
worksheet.</div>
<div class="btn-row generate-starter-row"
style="margin-bottom:10px;grid-template-columns:repeat(3,1fr)">
<button class="btn secondary" type="button" onclick="startBlankWorksheet()">Start
Blank</button>
<button class="btn secondary" type="button" onclick="toggleTemplateStarter()">Start
from Template</button>
<button class="btn secondary" id="adaptPdfStarterBtn" type="button"
onclick="toggleAdaptPdfStarter()">Adapt PDF</button>
</div>
<div id="adaptPdfPanel" class="task-card hidden" style="margin-top:12px">
  <div class="task-head" style="margin-bottom:12px">
    <strong>Adapt PDF</strong>
    <button class="mini" type="button"
onclick="toggleAdaptPdfStarter(false)">Close</button>
  </div>
  <div class="tiny" style="margin-bottom:10px">Upload a PDF and add any notes for
how the AI should adapt it into a worksheet. Then click Generate with AI.</div>

```

```

    <div id="adaptPdfControls"></div>
</div>
<button class="btn" id="generateBtn" onclick="generateWorksheet()">Generate with
AI</button>
<div id="templateStarterPanel" class="task-card hidden" style="margin-top:12px">
    <div class="task-head" style="margin-bottom:12px">
        <strong>Saved templates</strong>
        <button class="mini" type="button"
onclick="toggleTemplateStarter(false)">Close</button>
    </div>
    <div class="tiny" id="templateStarterCount" style="margin-bottom:10px">No saved
templates yet.</div>
    <div id="templateStarterList"></div>
</div>
<div class="status" id="statusBox"></div>
</div>

</div>
</div>
<div class="panel workspace">
<div class="workspace-top">
<div>
<h2></h2>
<p>Edit fields manually, then export to PDF.</p>
</div>
<div class="pill-row">
<button class="small-btn" onclick="addTask()">Add task</button>
<button class="small-btn" onclick="checkWorksheetBeforeExport()">Check
worksheet</button>
<button class="small-btn" onclick="saveCurrent('template')">Save as
template</button>
<button class="small-btn primary" onclick="exportPdf()">Export PDF</button>
<button class="small-btn" onclick="openPrintWindow()">Print / Save as PDF</button>
</div>
</div>
<div class="editor-preview">
<div class="editor" id="editorPane">
<div class="editor-empty">
<strong>No worksheet yet</strong>
        Use Start Blank, Start from Template, Adapt PDF, or Generate with AI.
    </div>
</div>
</div>
<div class="preview-wrap">
<div id="previewRoot">
<div class="preview-page"
style="display:flex;align-items:center;justify-content:center;text-align:center;col
or:#587094">
<div>


<h3 style="margin:16px 0 8px;font-size:22px;color:#12325f">Resource preview</h3>

<p style="margin:0;font-size:14px;max-width:430px;line-height:1.6">Generate a resource and it will appear here.</p>

</div>

</div>

</div>

</div>

</div>

</div>

</div>

</div>

<script

src="https://cdnjs.cloudflare.com/ajax/libs/html2canvas/1.4.1/html2canvas.min.js"><

/script>

<script

src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>

<script

```

src="https://cdnjs.cloudflare.com/ajax/libs/pdf.js/3.11.174/pdf.min.js"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/jszip/3.10.1/jszip.min.js"></script>
<style id="cyclestudio-v4-style">
.success-box{
 background:linear-gradient(180deg,#f8fbff 0%,#f1f7fe 100%);
 border:1px solid #dce8f6;
 border-left:5px solid #2a63bc;
 border-radius:16px;
 padding:14px 16px;
 margin:16px 0;
}
.success-box h3{margin:0 0 8px; font-size:15px; color:#12325f}
.success-box p,.success-box li{font-size:13px; line-height:1.65; color:#2d4b73}
.success-box ul{margin:0; padding-left:18px}
.task-passage{
 margin:0 0 10px;
 padding:12px 14px;
 background:#f7fbff;
 border:1px solid #dce8f6;
 border-radius:12px;
 font-size:13px;
 line-height:1.7;
 color:#29486f;
 white-space:pre-wrap;
}
.answer-question{margin:4px 0 6px; font-size:12px; line-height:1.55; color:#4d688d;
font-weight:700}
.variation-grid{display:grid; grid-template-columns:repeat(2,minmax(0,1fr));
gap:8px; margin-top:8px}
.variation-grid .small-btn{width:100%; justify-content:center}
.field-note{font-size:11px; line-height:1.55; color:#6a84a7; margin-top:6px}
.task-editor-grid{display:grid; grid-template-columns:1.3fr .7fr; gap:12px}
.task-actions{display:flex; flex-wrap:wrap; gap:6px}
.mini.secondary{background:#eef5ff; color:#19407b}
.image-usage-note{font-size:11px; line-height:1.55; color:#6a84a7; margin-top:6px}
.image-gen-grid{display:grid; grid-template-columns:1fr 1fr; gap:12px;
margin-top:12px}
.task-image-toggle{display:flex; align-items:center; gap:10px; font-size:13px;
font-weight:700; color:#23456f; margin-bottom:10px}
.task-image-toggle input{width:16px; height:16px}
.task-image-preview{margin-top:10px; border:1px solid #dce8f6; border-radius:14px;
background:#f8fbff; padding:10px}
.task-image-preview img{display:block; width:100%; max-height:210px;
object-fit:contain; border-radius:10px; background:#fff; border:1px solid #dce8f6;
margin:0 auto}
.task-image-meta{font-size:11px; color:#6a84a7; margin-top:8px; line-height:1.5}
.task-image-status{margin-top:10px; display:inline-flex; align-items:center;
gap:8px; padding:8px 10px; border-radius:12px; background:#fff8eb; border:1px solid
#f2ddb0; color:#8c6200; font-size:12px; font-weight:700}

```



```

.task-image-status::before{content:""; width:10px; height:10px; border-radius:50%;
background:currentColor; opacity:.75; animation:imagePulse 1s ease-in-out infinite}
@keyframes imagePulse{0%,100%{transform:scale(.85);
opacity:.45}50%{transform:scale(1.1); opacity:1}}
.task-image-print{display:block; width:100%; max-width:100%; height:auto;
object-fit:unset; border:1px solid #dce8f6; border-radius:16px; padding:10px;
background:#fff}
.task-image-wrap{margin:0 0 12px; display:flex; justify-content:center}
.task-pdf-tools{margin-top:12px}
.task-pdf-gallery{display:flex;flex-direction:column;gap:10px;margin-top:12px}
.task-pdf-card{border:1px solid
#dce8f6;border-radius:14px;background:#f8fbff;padding:10px}
.task-pdf-card-head{display:flex;justify-content:space-between;align-items:center;gap:10px;margin-bottom:8px}
.task-pdf-card-title{font-size:12px;font-weight:800;color:#12325f}
.task-pdf-thumb{display:block;width:100%;max-height:220px;object-fit:contain;border:1px solid #dce8f6;border-radius:10px;background:#fff}
.task-pdf-meta{font-size:11px;color:#6a84a7;margin-top:8px;line-height:1.45}
.task-pdf-crop-panel{margin-top:10px;padding:10px;border:1px solid
#dce8f6;border-radius:12px;background:#fff}
.task-pdf-crop-grid{display:grid;grid-template-columns:1fr 1fr;gap:10px}
.task-pdf-crop-actions{display:flex;gap:8px;flex-wrap:wrap;margin-top:10px}
.task-pdf-crop-note{font-size:11px;color:#6a84a7;line-height:1.45;margin-top:8px}
.task-pdf-modal{position:fixed;inset:0;background:rgba(18,50,95,.48);display:flex;align-items:center;justify-content:center;padding:20px;z-index:9999}
.task-pdf-modal.hidden{display:none !important}
.task-pdf-modal-card{width:min(1040px,100%);max-height:min(88vh,920px);background:#fff;border:1px solid #dce8f6;border-radius:24px;box-shadow:0 24px 60px
rgba(10,40,95,.22);display:flex;flex-direction:column;overflow:hidden}
.task-pdf-modal-head{padding:18px 20px;border-bottom:1px solid
#e1ebf7;display:flex;justify-content:space-between;align-items:flex-start;gap:16px}
.task-pdf-modal-head h3{margin:0 0
6px;font-size:24px;line-height:1.1;color:#12325f}
.task-pdf-modal-head p{margin:0;font-size:13px;line-height:1.55;color:#5d7698}
.task-pdf-modal-body{padding:18px
20px;overflow:auto;background:linear-gradient(180deg,#ffffff 0%,#fbfdff 100%)}
.task-pdf-grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(180px,1fr));gap:14px}
.task-pdf-choice{border:1px solid
#dce8f6;border-radius:18px;background:#fff;padding:10px;display:flex;flex-direction:column;gap:10px;box-shadow:0 8px 18px rgba(10,40,95,.05)}
.task-pdf-choice
img{display:block;width:100%;height:210px;object-fit:contain;border-radius:12px;border:1px solid #e1ebf7;background:#f8fbff}
.task-pdf-choice-row{display:flex;align-items:center;justify-content:space-between;gap:8px}
.task-pdf-choice-row
label{margin:0;font-size:12px;font-weight:800;color:#12325f;display:flex;align-items:center;gap:8px}
.task-pdf-choice-row input{width:16px;height:16px}

```

```

.task-pdf-choice-note{font-size:11px;color:#6a84a7;line-height:1.4}
.task-pdf-empty{padding:22px;border:1px dashed
#c7d8ef;border-radius:18px;text-align:center;color:#6a84a7;background:#f8fbff}
.task-pdf-modal-actions{padding:16px 20px;border-top:1px solid
#e1ebf7;display:flex;justify-content:space-between;align-items:center;gap:12px;flex
-wrap:wrap}
.task-pdf-modal-actions .left,.task-pdf-modal-actions
.right{display:flex;gap:10px;flex-wrap:wrap;align-items:center}
.task-pdf-loading{padding:28px;text-align:center;color:#5d7698;font-weight:700}
.task-pdf-loading
strong{display:block;color:#12325f;font-size:16px;margin-bottom:6px}
.task-pdf-inline-note{font-size:11px;color:#6a84a7;line-height:1.5;margin-top:8px}

.image-gen-badge{display:inline-flex; align-items:center; gap:8px; padding:8px
12px; border-radius:999px; background:#eff5fc; color:#426286; font-size:12px;
font-weight:700; margin-bottom:12px}
.rich-editor-wrap{display:flex; flex-direction:column; gap:8px}
.rich-toolbar{display:flex; gap:6px; flex-wrap:wrap}
.rich-toolbar-btn{min-width:38px; padding:7px 10px; border-radius:10px;
border:1.5px solid #dce8f6; background:#fff; color:#12325f; font-size:12px;
font-weight:800; cursor:pointer; transition:.14s ease}
.rich-toolbar-btn:hover{background:#eef5ff}
.rich-toolbar-btn u{text-decoration:underline; text-decoration:underline-offset:2px}
.rich-editor{width:100%; border:1.5px solid var(--line); background:#fff;
border-radius:14px; padding:12px 13px; font:inherit; color:var(--ink);
outline:none; transition:.15s ease; line-height:1.5; white-space:pre-wrap;
overflow-wrap:anywhere}
.rich-editor:focus{border-color:var(--blue); box-shadow:0 0 0 4px
rgba(25,89,200,.10)}
.rich-editor:empty:before{content:attr(data-placeholder); color:var(--ink-soft);
pointer-events:none}
.rich-editor b,.rich-editor strong,.rich-render b,.rich-render
strong{font-weight:700}
.rich-render i,.rich-render em,.rich-editor i,.rich-editor em{font-style:italic}
.rich-render u,.rich-editor u{text-decoration:underline; text-decoration:underline-offset:2px}
.rich-render{white-space:pre-wrap; overflow-wrap:anywhere}
.rich-criteria{font-size:13px; line-height:1.65; color:#2d4b73}
.font-size-control{display:inline-flex;align-items:center;gap:8px;padding:8px
12px;border-radius:14px;background:#fff;border:1.5px solid
var(--line);color:var(--ink);font-weight:700;font-size:12px}
.font-size-control select{width:auto;min-width:82px;padding:8px
10px;border-radius:10px}
.worksheet-lo-list{margin:0;padding-left:20px}
.worksheet-lo-list li{margin:0 0
8px;font-size:var(--ws-font-md,13px);line-height:1.55;color:#1f426c;font-weight:500
}
.preview-page .page-kicker{font-size:var(--ws-font-xs,11px)}
.preview-page
.worksheet-title{font-size:var(--ws-title-font,25px);line-height:1.12}
.preview-page .page-meta,.preview-page .student-line,.preview-page

```

```

.teacher-tag,.preview-page .lo-table,.preview-page .instructions p,.preview-page
.section-print p,.preview-page .image-note,.preview-page .intro-text,.preview-page
.task-prompt,.preview-page .mcq-list li,.preview-page .answer-item,.preview-page
.task-passage,.preview-page .rich-criteria,.preview-page
.worksheet-extension-text,.preview-page
.worksheet-teacher-notes{font-size:var(--ws-font-md,13px)}
.preview-page .instructions h3,.preview-page .section-print h3,.preview-page
.success-box h3,.preview-page .answer-key h3,.preview-page .extension-box
h3{font-size:var(--ws-font-lg,14px)}
.preview-page .answer-question,.preview-page .image-note,.preview-page
.teacher-tag{font-size:var(--ws-font-sm,12px)}
.preview-page .muted-note,.preview-page .page-kicker,.preview-page
.answer-stimulus{font-size:var(--ws-font-xs,11px)}
@media (max-width:760px){
 .variation-grid,.task-editor-grid,.image-gen-grid{grid-template-columns:1fr}
 .font-size-control{width:100%;justify-content:space-between}
}

.editor-accordion{padding:0;overflow:hidden}
.editor-accordion + .editor-accordion{margin-top:10px}
.editor-accordion-head{display:flex;align-items:center;justify-content:space-between;gap:10px;padding:12px 14px;cursor:pointer;background:linear-gradient(180deg,#ffffff 0%,#f8fbff 100%)}
.editor-accordion-head:hover{background:linear-gradient(180deg,#fbfdff 0%,#f2f7ff 100%)}
.editor-accordion.open > .editor-accordion-head{border-bottom:1px solid #dce8f6}
.editor-accordion-title-wrap{display:flex;align-items:flex-start;gap:10px;min-width:0;flex:1}
.editor-accordion-toggle{width:28px;height:28px;border-radius:10px;border:1.5px solid #dce8f6;background:#fff;color:#12325f;font-size:14px;font-weight:800;cursor:pointer;flex-shrink:0;display:inline-flex;align-items:center;justify-content:center;line-height:1;transition:.14s ease}
.editor-accordion-toggle:hover{background:#eef5ff}
.editor-accordion-title{font-size:14px;font-weight:800;color:#12325f;line-height:1.25;display:block}
.editor-accordion-sub{font-size:11px;line-height:1.5;color:#6a84a7;margin-top:3px}
.editor-accordion-body{padding:14px}
.editor-accordion-actions{display:flex;gap:6px;flex-wrap:wrap;justify-content:flex-end}
.editor-section-stack{display:flex;flex-direction:column;gap:12px}
.editor-task-tools{display:flex;gap:8px;flex-wrap:wrap;margin-bottom:10px}
.task-accordion .task-actions{justify-content:flex-end}
.task-accordion .task-actions .mini{white-space:nowrap}
.task-accordion .editor-accordion-head{padding:12px}
.task-accordion .editor-accordion-body{padding:12px}
.task-inner-accordion{margin-top:8px;border-radius:14px;box-shadow:none;background:#fff}
.task-inner-accordion .editor-accordion-head{padding:10px 12px;background:linear-gradient(180deg,#ffffff 0%,#fafdff 100%)}

```

```

.task-inner-accordion
.editor-accordion-head:hover{background:linear-gradient(180deg,#fbfdff 0%,#f3f8ff 100%)}
.task-inner-accordion .editor-accordion-title{font-size:13px}
.task-inner-accordion .editor-accordion-body{padding:12px}
.task-inner-accordion
.editor-accordion-toggle{width:24px;height:24px;border-radius:8px;font-size:13px}
.task-inner-accordion .field:last-child{margin-bottom:0}
@media
(max-width:760px){.editor-accordion-head{align-items:flex-start;flex-direction:column}.editor-accordion-actions,.task-accordion
.task-actions{width:100%;justify-content:flex-start}}

```

```

@media screen {
 .preview-wrap{
 --preview-scale:1;
 overflow-x:hidden;
 padding-inline:8px;
 }
 .preview-wrap .preview-page{
 width:min(794px, calc(100% - 8px));
 max-width:794px;
 min-height:1123px;
 margin-left:auto;
 margin-right:auto;
 margin-bottom:18px;
 transform:none;
 }
 .preview-wrap .preview-page:last-child{
 margin-bottom:0;
 }
}

```

</style>

<style id="cyclestudio-photocopy-boost">

```

.preview-page .teacher-box,
.preview-page .instructions,
.preview-page .task-print,
.preview-page .extension-box,
.preview-page .success-box,
.preview-page .task-passage,
.preview-page .section-print{
 border-color:#7387a2 !important;
 border-width:2.1px !important;
 box-shadow:none !important;
}
.preview-page .teacher-box,
.preview-page .success-box,
.preview-page .task-passage,

```

```

.preview-page .section-print{
 background:
 #e4eaf1,
 repeating-linear-gradient(135deg, rgba(72,90,116,.18) 0 3px,
 rgba(255,255,255,0) 3px 8px) !important;
}
.preview-page .instructions{
 background:
 #dde5ee,
 repeating-linear-gradient(135deg, rgba(72,90,116,.20) 0 3px,
 rgba(255,255,255,0) 3px 8px) !important;
 border-left:7px solid #41555e !important;
}
.preview-page .task-print{
 background:
 #ebf0f6,
 repeating-linear-gradient(180deg, rgba(95,113,137,.15) 0 2px,
 rgba(255,255,255,0) 2px 10px) !important;
}
.preview-page .extension-box{
 background:
 #e6ecf3,
 repeating-linear-gradient(135deg, rgba(72,90,116,.17) 0 3px,
 rgba(255,255,255,0) 3px 8px) !important;
 border-style:dashed !important;
}
.preview-page .teacher-tag,
.preview-page .task-number,
.preview-page .tick-cell.active .tick-box{
 background:#d8e0ea !important;
 color:#102847 !important;
 border-color:#647894 !important;
}
.preview-page .teacher-tag{
 border:1.6px solid #6f839f !important;
 font-weight:800 !important;
}
.preview-page .lo-table th{
 background:#dbe3ec !important;
 color:#29486f !important;
}
.preview-page .lo-table th,
.preview-page .lo-table td{
 border-color:#90a2ba !important;
}
.preview-page .answer-line{
 border-bottom:1.8px solid #7f93af !important;
 background:none !important;
}
.preview-page .worksheet-images img,

```

```

.preview-page .task-image-print{
 border-width:1.8px !important;
 border-color:#7b8ea8 !important;
}
.preview-page .line-fill span:last-child{
 border-bottom:2.2px solid #6f839f !important;
}
.preview-page .answer-key{
 border-top:2.5px solid #7387a2 !important;
}
</style>

<style id="cyclestudio-editor-ai-indicator-style">
.editor-preview{position:relative}
.editor-ai-overlay{
 position:absolute;
 inset:12px;
 display:none;
 align-items:flex-start;
 justify-content:center;
 pointer-events:none;
 z-index:60;
}
.editor-preview.editor-ai-busy .editor-ai-overlay{display:flex}
.editor-ai-overlay-box{
 margin-top:10px;
 display:flex;
 align-items:center;
 gap:12px;
 min-width:min(460px, calc(100% - 24px));
 max-width:min(560px, calc(100% - 24px));
 padding:14px 18px;
 border-radius:18px;
 background:rgba(255,255,255,.96);
 border:1.5px solid #bfd2ef;
 box-shadow:0 18px 34px rgba(18,50,95,.16);
 color:#12325f;
 backdrop-filter:blur(6px);
}
.editor-ai-overlay-spinner{
 width:24px;
 height:24px;
 border-radius:50%;
 border:3px solid #d9e6fb;
 border-top-color:#1959c8;
 flex-shrink:0;
 animation:editorAiSpin .8s linear infinite;
}
.editor-ai-overlay-title{
 font-size:13px;

```

```

 font-weight:800;
 line-height:1.25;
 margin:0 0 3px;
 color:#12325f;
}
.editor-ai-overlay-text{
 font-size:12px;
 line-height:1.45;
 color:#5a7293;
 margin:0;
}
@keyframes editorAiSpin{to{transform:rotate(360deg)}}
</style>

<style id="cyclestudio-task-media-safe-style">
/* Safe media layout: image gets its own tight frame; written task keeps the
normal task box. */
.preview-page .task-media-frame{
 border:2.1px solid #7387a2;
 border-radius:18px;
 background:#fff;
 padding:6px;
 margin:0 0 8px;
 overflow:hidden;
 box-shadow:none;
 break-inside:avoid;
 break-inside:avoid-page;
 page-break-inside:avoid;
}
.preview-page .task-media-frame .task-image-wrap{
 margin:0;
 display:flex;
 justify-content:center;
 align-items:flex-start;
}
.preview-page .task-media-frame .task-image-wrap +
.task-image-wrap{margin-top:6px}
.preview-page .task-media-frame .task-image-print{
 display:block;
 border:0 !important;
 padding:0 !important;
 background:#fff;
 border-radius:12px;
 max-width:100%;
 height:auto;
}
.preview-page .task-print-after-media{
 margin-top:0;
}
</style>

```

```

<script src="assets/cycle3-domain-library.js"></script>
<script src="assets/cycle-domain-library.js"></script>
<script>

// =====
// CycleStudio – consolidated single-script build
// =====

// — Base: domain library, state, core utilities —

const CYCLE_LABELS = {
 cycle1: 'Cycle 1',
 cycle2: 'Cycle 2',
 cycle3: 'Cycle 3'
};
const CYCLE_YEAR_LEVELS = {
 cycle1: ['PS', 'MS', 'GS'],
 cycle2: ['CP', 'CE1', 'CE2'],
 cycle3: ['CM1', 'CM2', 'CM1/CM2']
};
const DIFFICULTY_LEVELS = {
 easy: 'Easy',
 moderate: 'Moderate',
 challenging: 'Challenging',
 advanced: 'Advanced'
};
const CYCLE_DOMAIN_LIBRARIES = window.CYCLE_DOMAIN_LIBRARIES ||
{cycle3: window.CYCLE3_DOMAIN_LIBRARY || {}};
let currentCycle = 'cycle3';
let DOMAIN_LIBRARY = CYCLE_DOMAIN_LIBRARIES.cycle3 || {};
let DOMAIN_NAMES = Object.keys(DOMAIN_LIBRARY);

function normaliseCycle(value){
 return CYCLE_LABELS[value] ? value : 'cycle3';
}
function getCycleLabel(cycle=currentCycle){
 return CYCLE_LABELS[normaliseCycle(cycle)];
}
function getCycleYears(cycle=currentCycle){
 return CYCLE_YEAR_LEVELS[normaliseCycle(cycle)] || CYCLE_YEAR_LEVELS.cycle3;
}
function setCurrentCycle(cycle){
 currentCycle = normaliseCycle(cycle);
 DOMAIN_LIBRARY = CYCLE_DOMAIN_LIBRARIES[currentCycle] ||
CYCLE_DOMAIN_LIBRARIES.cycle3 || {};
 DOMAIN_NAMES = Object.keys(DOMAIN_LIBRARY);
}
function getStudioYearLevel(){

```



```

 return qs('studioYearLevel')?.value || getCycleYears()[0] || 'CM1';
}
function difficultyYearKey(year=getStudioYearLevel()){
 return String(year || 'CM1').toLowerCase().replace(/^[a-z0-9]+/g, '');
}
function parseDifficulty(value='') {
 const raw = String(value || '').trim();
 const legacy = {
 easy: 'easy',
 moderate: 'moderate',
 challenging: 'challenging',
 advanced: 'advanced'
 };
 if(legacy[raw]) return { yearKey: difficultyYearKey(), level: legacy[raw] };
 const match = raw.match(/^[a-z0-9]+_(easy|moderate|challenging|advanced)$/);
 return match ? { yearKey: match[1], level: match[2] } : { yearKey:
difficultyYearKey(), level: 'moderate' };
}
function difficultyValueFor(year=getStudioYearLevel(), level='moderate'){
 const safeLevel = DIFFICULTY_LEVELS[level] ? level : 'moderate';
 return `${difficultyYearKey(year)}_${safeLevel}`;
}
function populateDifficultyOptions(preferredValue='') {
 const select = qs('difficulty');
 if(!select) return;
 const year = getStudioYearLevel();
 const parsed = parseDifficulty(preferredValue || select.value);
 const selectedLevel = DIFFICULTY_LEVELS[parsed.level] ? parsed.level :
'moderate';
 select.innerHTML = Object.entries(DIFFICULTY_LEVELS)
 .map(([level, label]) => `<option value="${esc(difficultyValueFor(year,
level))}">${esc(year)} - ${esc(label)}</option>`)
 .join('');
 select.value = difficultyValueFor(year, selectedLevel);
}
function defaultStudioDomain(){
 if(DOMAIN_LIBRARY['English Literacy']) return 'English Literacy';
 if(DOMAIN_LIBRARY['Literacy']) return 'Literacy';
 if(DOMAIN_LIBRARY['Oral and Written Language']) return 'Oral and Written
Language';
 return DOMAIN_NAMES[0] || '';
}
function studioCyclePromptGuidance(){
 const year = getStudioYearLevel();
 if(currentCycle === 'cycle1'){
 return `Age and cycle fit: This is Cycle 1 (${year}, maternelle). Make the
resource play-based, oral, visual and practical. Prefer sorting, matching, drawing,
tracing, simple mark-making, oral rehearsal, manipulatives and teacher-led prompts.
Keep written output very light and developmentally appropriate.`;
 }
}

```

```

 if(currentCycle === 'cycle2'){
 return `Age and cycle fit: This is Cycle 2 (${year}). Keep the resource
suitable for early primary learners. Use clear steps, short readable text, strong
scaffolding, concrete examples and tasks that fit CP/CE1/CE2 independence.`;
 }
 return `Age and cycle fit: This is Cycle 3 (${year}). Keep the resource suitable
for upper-primary learners, with clear instructions, purposeful challenge and
enough scaffolding for classroom use.`;
}
function studioAgeGroupText(){
 switch(getStudioYearLevel()){
 case 'PS': return '3 to 4 year olds';
 case 'MS': return '4 to 5 year olds';
 case 'GS': return '5 to 6 year olds';
 case 'CP': return '6 to 7 year olds';
 case 'CE1': return '7 to 8 year olds';
 case 'CE2': return '8 to 9 year olds';
 case 'CM2': return '10 to 11 year olds';
 case 'CM1/CM2': return '9 to 11 year olds';
 default: return '9 to 10 year olds';
 }
}
function populateStudioCycleSelect(){
 const select = qs('studioCycleSelect');
 if(!select) return;
 select.innerHTML = Object.entries(CYCLE_LABELS).map(([value,label]) => `<option
value="${esc(value)}">${esc(label)}</option>`).join('');
 select.value = currentCycle;
}
function populateStudioYearLevels(preferredYear='') {
 const select = qs('studioYearLevel');
 if(!select) return;
 const years = getCycleYears();
 select.innerHTML = years.map(year => `<option
value="${esc(year)}">${esc(year)}</option>`).join('');
 select.value = years.includes(preferredYear) ? preferredYear : years[0];
}
function applyStudioCycle(cycle, preferredYear='') {
 setCurrentCycle(cycle);
 populateStudioCycleSelect();
 populateStudioYearLevels(preferredYear);
 populateDifficultyOptions(qs('difficulty')?.value || readStoredPrefs().difficulty
|| '');
 populateDomains();
 const domainSelect = qs('domainSelect');
 if(domainSelect && !DOMAIN_LIBRARY[domainSelect.value]){
 domainSelect.value = defaultStudioDomain();
 }
 state.selectedOutcomes = (state.selectedOutcomes || []).filter(outcome =>
getFlatDomainOutcomes(domainSelect?.value || '').includes(outcome));

```

```

 if(qs('studioDomainCycleLabel')) qs('studioDomainCycleLabel').textContent =
getCycleLabel();
 if(typeof renderOutcomeOptions === 'function') renderOutcomeOptions();
}

function getGroupedDomainOutcomes(domain){
 const entry = DOMAIN_LIBRARY[domain];
 return entry && !Array.isArray(entry) && typeof entry === 'object' ? entry :
null;
}

function getFlatDomainOutcomes(domain){
 const entry = DOMAIN_LIBRARY[domain];
 if(Array.isArray(entry)) return entry;
 if(entry && typeof entry === 'object') return Object.values(entry).flat();
 return [];
}

const MATCH_STOPWORDS = new
Set(['and','the','for','with','from','into','their','them','this','that','these','t
hose','using','use','your','about','across','within','between','through','around','
while','where','when','what','which','into','over','under','such','than','then','th
ey','them','have','has','had','its','our','out','off','all','too','can','may','how'
]);

function uniqueList(items){
 return [...new Set((items || []).filter(Boolean))];
}

function normaliseForMatch(str=''){
 return String(str)
 .normalize('NFD')
 .replace(/[\u0300-\u036f]/g, '')
 .toLowerCase()
 .replace(/&/g, ' and ')
 .replace(/[^a-z0-9]+/g, ' ')
 .trim();
}

function getMeaningfulTokens(str=''){
 return uniqueList(
 normaliseForMatch(str)
 .split(/\s+/)
 .filter(token => token && token.length > 2 && !MATCH_STOPWORDS.has(token))
);
}

function canonicaliseDomain(value=''){
 const raw = String(value || '').trim();
 if(!raw) return '';

```

```

 if(DOMAIN_NAMES.includes(raw)) return raw;
 if(currentCycle === 'cycle2' && raw === 'French' &&
DOMAIN_NAMES.includes('Literacy')) return 'Literacy';
 const norm = normaliseForMatch(raw);
 const exact = DOMAIN_NAMES.find(name => normaliseForMatch(name) === norm);
 if(exact) return exact;
 const loose = DOMAIN_NAMES.find(name => {
 const domainNorm = normaliseForMatch(name);
 return domainNorm.includes(norm) || norm.includes(domainNorm);
 });
 return loose || '';
}

function scoreTextAgainstTokens(textNorm, tokens){
 let score = 0;
 tokens.forEach(token => {
 if(textNorm.includes(token)) score += token.length >= 7 ? 2.4 : 1.4;
 });
 return score;
}

function domainKeywordSeed(domain){
 switch(domain){
 case 'Mathematics': return
['number','calculate','calculation','fraction','decimal','problem','measure','measu
rement','geometry','shape','data','graph','table','pattern','sequence','angle','per
imeter','area'];
 case 'French': return
['read','reading','write','writing','grammar','spelling','punctuation','verb','sent
ence','text','vocabulary','oral','speaking'];
 case 'Literacy': return
['read','reading','write','writing','grammar','spelling','punctuation','verb','sent
ence','text','vocabulary','oral','speaking','comprehension','phonics','handwriting'
];
 case 'Oral and Written Language': return
['oral','speaking','listen','listening','story','book','letter','word','sentence','
sound','draw','trace','write','dictation'];
 case 'English Literacy': return
['read','reading','write','writing','grammar','spelling','punctuation','verb','sent
ence','text','vocabulary','oral','speaking','inference','comprehension'];
 case 'Science and Technology': return
['science','scientific','investigation','experiment','material','energy','force','m
otion','living','habitat','plant','animal','earth','environment','weather','technol
ogy'];
 case 'History and Geography': return
['history','historical','geography','geographical','timeline','map','place','king',
'republic','revolution','empire','city','environment','travel'];
 case 'Civics and Moral Education': return
['citizen','citizenship','rights','responsibilities','rules','debate','republic','d
iscrimination','stereotypes','ethical'];
 }
}

```

```

 case 'Visual Arts': return
['art','artist','artwork','colour','color','shape','texture','drawing','painting','
creative'];
 case 'Music': return
['music','musical','rhythm','pitch','tempo','sound','sing','compose'];
 case 'PE': return
['sport','physical','movement','team','game','exercise','fitness','swimming','runni
ng'];
 case 'Modern Languages': return
['language','speaking','listening','a1','a2','cultural','vocabulary'];
 default: return [];
}
}

```

```

function inferDomainFromText(text='') {
 const textNorm = normaliseForMatch(text);
 if(!textNorm) return defaultStudioDomain();
 let bestDomain = defaultStudioDomain();
 let bestScore = -Infinity;
 DOMAIN_NAMES.forEach(domain => {
 let score = 0;
 score += scoreTextAgainstTokens(textNorm, getMeaningfulTokens(domain));
 score += scoreTextAgainstTokens(textNorm, domainKeywordSeed(domain)) * 1.5;
 getFlatDomainOutcomes(domain).forEach(outcome => {
 const outcomeNorm = normaliseForMatch(outcome);
 if(outcomeNorm && textNorm.includes(outcomeNorm)) score += 12;
 score += Math.min(8, scoreTextAgainstTokens(textNorm,
getMeaningfulTokens(outcome)) * 0.45);
 });
 if(score > bestScore){
 bestScore = score;
 bestDomain = domain;
 }
 });
 return bestDomain;
}

```

```

function canonicaliseOutcome(value='', domain='') {
 const raw = String(value || '').trim();
 if(!raw) return '';
 const allowed = getFlatDomainOutcomes(domain);
 if(!allowed.length) return '';
 if(allowed.includes(raw)) return raw;
 const norm = normaliseForMatch(raw);
 const exact = allowed.find(outcome => normaliseForMatch(outcome) === norm);
 if(exact) return exact;
 let best = '';
 let bestScore = 0;
 const valueTokens = getMeaningfulTokens(raw);
 allowed.forEach(outcome => {

```

```

 const outcomeNorm = normaliseForMatch(outcome);
 let score = 0;
 if(outcomeNorm && (outcomeNorm.includes(norm) || norm.includes(outcomeNorm)))
score += 10;
 valueTokens.forEach(token => {
 if(outcomeNorm.includes(token)) score += token.length >= 7 ? 2.2 : 1.2;
 });
 if(score > bestScore){
 bestScore = score;
 best = outcome;
 }
});
return bestScore >= 4 ? best : '';
}

```

```

function chooseAllowedOutcomes(domain, text='', preferredCount=3,
forcedOutcomes=[]){
 const allowed = getFlatDomainOutcomes(domain);
 if(!allowed.length) return [];
 const targetCount = Math.max(2, Math.min(4, preferredCount || 3));
 const picked = uniqueList((forcedOutcomes || []).map(outcome =>
canonicaliseOutcome(outcome, domain)).filter(Boolean));
 const textNorm = normaliseForMatch(text);
 const ranked = allowed.map(outcome => {
 const outcomeNorm = normaliseForMatch(outcome);
 let score = 0;
 if(textNorm && outcomeNorm && textNorm.includes(outcomeNorm)) score += 20;
 score += scoreTextAgainstTokens(textNorm, getMeaningfulTokens(outcome));
 return { outcome, score };
 }).sort((a,b) => b.score - a.score || allowed.indexOf(a.outcome) -
allowed.indexOf(b.outcome));

 for(const item of ranked){
 if(picked.length >= targetCount) break;
 if(!picked.includes(item.outcome) && (item.score > 0 || picked.length < 2))
picked.push(item.outcome);
 }
 for(const outcome of allowed){
 if(picked.length >= targetCount) break;
 if(!picked.includes(outcome)) picked.push(outcome);
 }
 return picked.slice(0, Math.min(6, targetCount));
}

```

```

function getAllowedDomainCatalogueText(domain='') {
 const domains = domain && DOMAIN_LIBRARY[domain] ? [domain] : DOMAIN_NAMES;
 return domains.map(name => {
 const grouped = getGroupedDomainOutcomes(name);
 if(grouped){
 const groupedText = Object.entries(grouped).map(([strand, outcomes]) => `

```

```

 ${strand}:\n${outcomes.map(outcome => ` -
 "${outcome}"`).join('\n')}\n`).join('\n');
 return `${name}:\n${groupedText}`;
 }
 return `${name}:\n${getFlatDomainOutcomes(name).map(outcome => ` -
 "${outcome}"`).join('\n')}\n`
 }).join('\n\n');
}

function buildCurriculumEvidenceText(result={}){
 const taskBits = Array.isArray(result?.tasks)
 ? result.tasks.flatMap(task => [task?.passage || '', task?.prompt || '',
 ...(Array.isArray(task?.options) ? task.options : []), task?.answer || ''])
 : [];
 return [
 qs('worksheetBrief')?.value.trim() || '',
 qs('templateText')?.value.trim() || '',
 result?.domain || '',
 ...(Array.isArray(result?.learning_outcomes) ? result.learning_outcomes : []),
 result?.title || '',
 result?.instructions || '',
 result?.success_criteria || '',
 result?.intro_text || '',
 result?.image_note || '',
 ...taskBits
].filter(Boolean).join('\n');
}

const state = {
 curriculumMode: 'ai_both',
 images: [],
 promptImages: [],
 promptDocuments: [],
 templatePdf: null,
 worksheet: null,
 qualityCheck: null,
 currentWizardStep: 1,
 selectedOutcomes: [],
 loStatus: {},
 displayOptions: {
 instructions: false,
 successCriteria: false,
 extension: false,
 learningObjectives: false
 },
};

const LS_KEYS = {
 prefs: 'wg_v1_prefs',
 drafts: 'wg_v1_drafts',

```

```
 templates: 'wg_v1_templates',
 history: 'cyclestudio_history_v1'
};
```

```
const PREF_DEFAULTS = {
 apiKey: '',
 modelName: '',
 outputMode: 'student',
 difficulty: 'cm1_moderate',
 questionCount: '6',
 worksheetBrief: '',
 templateText: '',
 imageApiKey: '',
 imageModel: 'gpt-image-1.5',
 imageGenSize: '1024x1024',
 imageUsage: 'teacher_provided',
 preferredCycle: 'cycle3',
 yearLevel: 'CM1',
 curriculumMode: 'ai_both',
 useAICurriculum: true,
 showInstructions: true,
 showSuccessCriteria: true,
 showExtension: true,
 showLearningObjectives: false,
 fontSize: '12'
};
```

```
const AI_SUGGESTED_TASK_COUNT = 'ai';
```

```
function qs(id){ return document.getElementById(id); }
function qsa(sel, root=document){ return Array.from(root.querySelectorAll(sel)); }
function isTaskCountAiSuggested(){
 return (qs('questionCount')?.value || '') === AI_SUGGESTED_TASK_COUNT;
}
function readStoredPrefs(){
 try{
 const raw = JSON.parse(localStorage.getItem(LS_KEYS.prefs) || '{}');
 return { ...PREF_DEFAULTS, ...(raw || {}) };
 }catch(err){
 return { ...PREF_DEFAULTS };
 }
}
function writeStoredPrefs(prefs){
 try{
 localStorage.setItem(LS_KEYS.prefs, JSON.stringify(prefs || {}));
 if(window.SylCycle) SylCycle.cloudSave(LS_KEYS.prefs, prefs || {});
 }catch(err){}
}
function buildPrefsObject(){
 return {
 ...readStoredPrefs(),
```



```

 apiKey: qs('apiKey')?.value.trim() || '',
 modelName: qs('modelName')?.value.trim() || '',
 outputMode: qs('outputMode')?.value || 'student',
 difficulty: qs('difficulty')?.value || 'cm1_moderate',
 questionCount: qs('questionCount')?.value || '6',
 answerGenMode: getAnswerGenMode(),
 worksheetBrief: qs('worksheetBrief')?.value || '',
 templateText: qs('templateText')?.value || '',
 imageApiKey: qs('imageApiKey')?.value.trim() || '',
 imageModel: qs('imageModel')?.value.trim() || 'gpt-image-1.5',
 imageGenSize: (typeof getImageGenSize === 'function' ? getImageGenSize() :
(readStoredPrefs().imageGenSize || PREF_DEFAULTS.imageGenSize)),
 imageUsage: (typeof getImageUsage === 'function' ? getImageUsage() :
(state.imageUsage || readStoredPrefs().imageUsage || PREF_DEFAULTS.imageUsage)),
 preferredCycle: currentCycle,
 yearLevel: qs('studioYearLevel')?.value || getCycleYears()[0] || 'CM1',
 curriculumMode: state.curriculumMode || 'ai_both',
 useAICurriculum: (state.curriculumMode || 'ai_both') === 'ai_both',
 showInstructions: state.displayOptions?.instructions !== false,
 showSuccessCriteria: state.displayOptions?.successCriteria !== false,
 showExtension: state.displayOptions?.extension !== false,
 showLearningObjectives: state.displayOptions?.learningObjectives === true,
 fontSize: String(typeof getWorksheetFontSize === 'function' ?
getWorksheetFontSize() : (state.fontSize || readStoredPrefs().fontSize || 12))
 };
}

function getAnswerGenMode(){
 const stored = readStoredPrefs();
 return qs('answerGenMode')?.value || state.answerGenMode || stored.answerGenMode
|| 'student_only';
}

function shouldGenerateAnswers(){
 return getAnswerGenMode() === 'with_answers';
}

function hasAnswerKeyData(worksheet=state.worksheet){
 return Array.isArray(worksheet?.tasks) && worksheet.tasks.some(task =>
String(task?.answer || '').trim());
}

function stripTaskAnswers(worksheet){
 if(!worksheet?.tasks?.length) return worksheet;
 worksheet.tasks.forEach(task => { task.answer = ''; });
 return worksheet;
}

function normaliseDifficulty(val=''){
 const parsed = parseDifficulty(val);
 const currentYearKey = difficultyYearKey();
 const cycleYearKeys = getCycleYears().map(year => difficultyYearKey(year));
 const yearKey = cycleYearKeys.includes(parsed.yearKey) ? parsed.yearKey :

```

```

currentYearKey;
 const level = DIFFICULTY_LEVELS[parsed.level] ? parsed.level : 'moderate';
 return `${yearKey}_${level}`;
}

function esc(str='') {
 return String(str)
 .replaceAll('&', '&')
 .replaceAll('<', '<')
 .replaceAll('>', '>')
 .replaceAll('"', '"')
 .replaceAll("'", ''');
}

const escapeHtml = esc;
function nl2br(str='') { return esc(str).replace(/\n/g, '
'); }
function jsq(str='') { return
String(str).replace(/\\/g, '\\\\').replace(/'/g, '\\\'').replace(/\n/g, '\\n').replace(
/\r/g, ''); }

function showStatus(message, kind='ok') {
 const box = qs('statusBox');
 if(!box) return;
 box.textContent = message;
 box.className = 'status show ' + (kind || 'ok');
}

function clearStatus() {
 const box = qs('statusBox');
 if(!box) return;
 box.className = 'status';
 box.textContent = '';
}

let previewRaf = 0;
function queuePreviewUpdate() {
 if(previewRaf) cancelAnimationFrame(previewRaf);
 previewRaf = requestAnimationFrame(() => {
 previewRaf = 0;
 updatePreview();
 });
}

function loadPrefs() {
 const prefs = readStoredPrefs();
 applyStudioCycle(prefs.preferredCycle || 'cycle3', prefs.yearLevel || '');
 if(qs('apiKey')) qs('apiKey').value = prefs.apiKey || '';
 if(qs('modelName')) qs('modelName').value = prefs.modelName || '';
 if(qs('outputMode')) qs('outputMode').value = prefs.outputMode || 'student';
 if(qs('difficulty')) qs('difficulty').value =
normaliseDifficulty(prefs.difficulty || 'cm1_moderate');
 if(qs('questionCount')) qs('questionCount').value = prefs.questionCount || '6';
}

```

```

 if(qs('answerGenMode')) qs('answerGenMode').value = prefs.answerGenMode ||
'student_only';
 state.answerGenMode = prefs.answerGenMode || 'student_only';
 if(qs('worksheetBrief')) qs('worksheetBrief').value = prefs.worksheetBrief || '';
 if(qs('templateText')) qs('templateText').value = prefs.templateText || '';
 state.curriculumMode = prefs.curriculumMode || (prefs.useAICurriculum !==
undefined ? (prefs.useAICurriculum ? 'ai_both' : 'manual_both') : 'ai_both');
 syncToggles();
}
function savePrefs(){
 writeStoredPrefs(buildPrefsObject());
}

function bindPrefInputs(){

['apiKey','modelName','outputMode','difficulty','questionCount','answerGenMode','wo
rksheetBrief','templateText','studioCycleSelect','studioYearLevel'].forEach(id => {
 const el = qs(id);
 if(!el || el.dataset.prefBound === '1') return;
 el.dataset.prefBound = '1';
 el.addEventListener('input', savePrefs);
 el.addEventListener('change', savePrefs);
});
const cycleSelect = qs('studioCycleSelect');
if(cycleSelect && cycleSelect.dataset.cycleBound !== '1'){
 cycleSelect.dataset.cycleBound = '1';
 cycleSelect.addEventListener('change', () => {
 applyStudioCycle(cycleSelect.value, qs('studioYearLevel')?.value || '');
 savePrefs();
 syncToggles();
 });
}
const yearSelect = qs('studioYearLevel');
if(yearSelect && yearSelect.dataset.yearBound !== '1'){
 yearSelect.dataset.yearBound = '1';
 yearSelect.addEventListener('change', () => {
 applyStudioCycle(currentCycle, yearSelect.value || '');
 savePrefs();
 syncToggles();
 });
}
}

function setCurriculumMode(mode){
 state.curriculumMode = mode;
 syncToggles();
 savePrefs();
}

function syncToggles(){

```

```

['ai_both', 'manual_domain_ai_lo', 'manual_both'].forEach(mode => {
 const btn = qs('currMode_' + mode);
 if(btn) btn.classList.toggle('active', state.curriculumMode === mode);
});
const showDomain = state.curriculumMode !== 'ai_both';
const showManualOutcomes = state.curriculumMode === 'manual_both';
if(qs('domainArea')) qs('domainArea').classList.toggle('hidden', !showDomain);
if(qs('manualCurriculumArea'))
 qs('manualCurriculumArea').classList.toggle('hidden', !showManualOutcomes);
}

function populateDomains(){
 qs('domainSelect').innerHTML = '<option value="">Select a domain</option>' +
 DOMAIN_NAMES.map(name => `<option
value="${esc(name)}">${esc(name)}</option>`).join('');
 renderOutcomeOptions();
}
function renderOutcomeOptions(){
 const domain = qs('domainSelect').value;
 const box = qs('outcomesBox');
 if(!domain){
 box.innerHTML = '<div class="tiny">Choose a domain first.</div>';
 return;
 }
 const selected = new Set(state.selectedOutcomes);
 const grouped = getGroupedDomainOutcomes(domain);
 if(grouped){
 box.innerHTML = Object.entries(grouped).map(([strand, outcomes])=>`
 <div class="outcome-group">
 <div class="outcome-group-title">${esc(strand)}</div>
 <div class="outcome-group-list">
 ${outcomes.map(outcome=>`
 <label class="outcome-option">
 <input type="checkbox" data-outcome="${esc(outcome)}"
${selected.has(outcome) ? 'checked' : ''} onchange="toggleOutcome(this)">
 ${esc(outcome)}
 </label>
 `).join('')}
 </div>
 </div>
 `).join('');
 return;
 }
 box.innerHTML = getFlatDomainOutcomes(domain).map((outcome)=>`
 <label class="outcome-option">
 <input type="checkbox" data-outcome="${esc(outcome)}" ${selected.has(outcome)
? 'checked' : ''} onchange="toggleOutcome(this)">
 ${esc(outcome)}
 </label>
 `).join('');
}

```

```

}
function toggleOutcome(el){
 const value = el.getAttribute('data-outcome');
 if(el.checked){
 if(!state.selectedOutcomes.includes(value)) state.selectedOutcomes.push(value);
 }else{
 state.selectedOutcomes = state.selectedOutcomes.filter(item => item !== value);
 }
}
}

```

```

function resetManualOutcomesIfDomainChanged(){
 state.selectedOutcomes = [];
 renderOutcomeOptions();
}

```

```

function bindUploads(){
 const imageUpload = qs('imageUpload');
 const promptImageUpload = qs('promptImageUpload');
 const templatePdf = qs('templatePdf');
 const domainSelect = qs('domainSelect');
 const pasteImageZone = qs('pasteImageZone');
 const pasteImageBtn = qs('pasteImageBtn');
 const promptImagePanel = qs('promptImagePanel');
 const pastePromptImageBtn = qs('pastePromptImageBtn');

 if(imageUpload && imageUpload.dataset.boundUploads !== '1'){
 imageUpload.dataset.boundUploads = '1';
 imageUpload.addEventListener('change', async (e)=>{
 const files = [...e.target.files || []];
 for(const file of files){
 await addImageToState(file);
 }
 renderImageThumbs();
 savePrefs();
 e.target.value = '';
 });
 }
}

```

```

if(promptImageUpload && promptImageUpload.dataset.boundUploads !== '1'){
 promptImageUpload.dataset.boundUploads = '1';
 promptImageUpload.addEventListener('change', async (e)=>{
 const files = [...e.target.files || []];
 let added = 0;
 let skipped = 0;
 for(const file of files){
 try{
 if(await addPromptSourceToState(file)) added += 1;
 else skipped += 1;
 }catch(err){

```

```

 console.warn('Could not add source file', file?.name, err);
 skipped += 1;
 }
}
renderPromptImageThumbs();
savePrefs();
if(added) showStatus(`${added} source file${added === 1 ? '' : 's'} added.`,
skipped ? 'warn' : 'ok');
if(skipped && !added) showStatus('No usable source files were added. Use
images, PDFs or PPTX files.', 'warn');
e.target.value = '';
});
}

if(pasteImageZone && pasteImageZone.dataset.boundUploads !== '1'){
 pasteImageZone.dataset.boundUploads = '1';
 const setActive = (isActive)=> pasteImageZone.classList.toggle('active',
!!isActive);
 pasteImageZone.addEventListener('focus', ()=>setActive(true));
 pasteImageZone.addEventListener('blur', ()=>setActive(false));
 pasteImageZone.addEventListener('click', ()=>pasteImageZone.focus());
 pasteImageZone.addEventListener('paste', async (e)=>{
 const count = await addImagesFromClipboardItems(e.clipboardData?.items ||
[]);
 if(count > 0){
 e.preventDefault();
 renderImageThumbs();
 savePrefs();
 showStatus(`${count} pasted image${count === 1 ? '' : 's'} added.`, 'ok');
 }else{
 showStatus('No image found in the clipboard. Copy the screenshot first,
then paste it here.', 'warn');
 }
 });
}

```

```

if(promptImagePanel && promptImagePanel.dataset.boundUploads !== '1'){
 promptImagePanel.dataset.boundUploads = '1';
 const setPromptActive = (isActive)=>
promptImagePanel.classList.toggle('active', !!isActive);
 promptImagePanel.addEventListener('focus', ()=>setPromptActive(true));
 promptImagePanel.addEventListener('blur', ()=>setPromptActive(false));
 promptImagePanel.addEventListener('click', ()=>promptImagePanel.focus());
 promptImagePanel.addEventListener('paste', async (e)=>{
 const count = await addPromptImagesFromClipboardItems(e.clipboardData?.items
|| []);
 if(count > 0){
 e.preventDefault();
 renderPromptImageThumbs();

```

```

 savePrefs();
 showStatus(`${count} prompt image${count === 1 ? '' : 's'} added.`, 'ok');
 }else{
 showStatus('No image found in the clipboard. Copy the screenshot first,
then paste it here.', 'warn');
 }
 });
}

```

```

if(pasteImageBtn && pasteImageBtn.dataset.boundUploads !== '1'){
 pasteImageBtn.dataset.boundUploads = '1';
 pasteImageBtn.addEventListener('click', async ()=>{
 if(navigator.clipboard && navigator.clipboard.read){
 try{
 const items = await navigator.clipboard.read();
 const count = await addImagesFromClipboardApi(items);
 if(count > 0){
 renderImageThumbs();
 savePrefs();
 showStatus(`${count} clipboard image${count === 1 ? '' : 's'} added.`,
'ok');
 return;
 }
 }catch(err){
 console.warn('Clipboard read failed', err);
 }
 }
 pasteImageZone?.focus();
 showStatus('Click the paste box and press Ctrl/Cmd + V to paste your
screenshot.', 'warn');
 });
}

```

```

if(pastePromptImageBtn && pastePromptImageBtn.dataset.boundUploads !== '1'){
 pastePromptImageBtn.dataset.boundUploads = '1';
 pastePromptImageBtn.addEventListener('click', async ()=>{
 if(navigator.clipboard && navigator.clipboard.read){
 try{
 const items = await navigator.clipboard.read();
 const count = await addPromptImagesFromClipboardApi(items);
 if(count > 0){
 renderPromptImageThumbs();
 savePrefs();
 showStatus(`${count} clipboard prompt image${count === 1 ? '' : 's'}
added.`, 'ok');
 return;
 }
 }catch(err){
 console.warn('Clipboard read failed', err);
 }
 }
 });
}

```

```

 }
 }
 promptImagePanel?.focus();
 showStatus('Click the upload panel and press Ctrl/Cmd + V to paste your
example image.', 'warn');
});
}

if(templatePdf && templatePdf.dataset.boundUploads !== '1'){
 templatePdf.dataset.boundUploads = '1';
 templatePdf.addEventListener('change', async (e)=>{
 const file = e.target.files?.[0];
 if(!file) return;
 const dataUrl = await fileToDataUrl(file);
 state.templatePdf = {
 name: file.name,
 mimeType: file.type || 'application/pdf',
 dataUrl,
 base64: dataUrl.split(',')[1]
 };
 qs('templatePdfInfo').textContent = 'Loaded: ' + file.name;
 });
}

if(domainSelect && domainSelect.dataset.boundUploads !== '1'){
 domainSelect.dataset.boundUploads = '1';
 domainSelect.addEventListener('change', resetManualOutcomesIfDomainChanged);
}
}

async function addImageToState(file, fallbackName='Pasted image'){
 if(!file || !(file.type || '').startsWith('image/')) return false;
 const dataUrl = await fileToDataUrl(file);
 const safeName = file.name || `${fallbackName} ${state.images.length +
1}.${(file.type || 'image/png').split('/')[1] || 'png'}`;
 state.images.push({
 name: safeName,
 mimeType: file.type || 'image/png',
 dataUrl,
 base64: dataUrl.split(',')[1]
 });
 return true;
}

async function addPromptImageToState(file, fallbackName='Example image'){
 if(!file || !(file.type || '').startsWith('image/')) return false;
 const dataUrl = await fileToDataUrl(file);
 const safeName = file.name || `${fallbackName} ${state.promptImages.length +
1}.${(file.type || 'image/png').split('/')[1] || 'png'}`;

```



```

state.promptImages.push({
 name: safeName,
 mimeType: file.type || 'image/png',
 dataUrl,
 base64: dataUrl.split(',')[1]
});
return true;
}

function fileExt(file){
 const name = String(file?.name || '').toLowerCase();
 return name.includes('.') ? name.split('.').pop() : '';
}

function isPdfFile(file){
 return (file?.type || '') === 'application/pdf' || fileExt(file) === 'pdf';
}

function isPptxFile(file){
 return (file?.type || '') ===
'application/vnd.openxmlformats-officedocument.presentationml.presentation' ||
fileExt(file) === 'pptx';
}

function decodeXmlEntities(str='') {
 const map = { amp: '&', lt: '<', gt: '>', quot: '"', apos: "'" };
 return String(str)
 .replace(/&#(\d+)/g, (_, n) => String.fromCharCode(parseInt(n, 10)))
 .replace(/&#x([0-9a-f]+)/gi, (_, n) => String.fromCharCode(parseInt(n, 16)))
 .replace(/&([a-z]+)/gi, (_, entity) => map[entity] || `&${entity};`);
}

function stripXmlTags(str='') {
 return decodeXmlEntities(String(str || '').replace(/<[^>]+>/g, ' '))
 .replace(/\s+/g, ' ').trim();
}

async function extractPptxText(file) {
 if(!window.JSZip) throw new Error('PPTX support did not load. Try again, or export the slideshow as PDF.');
```

```

 const zip = await window.JSZip.loadAsync(await file.arrayBuffer());
 const slideFiles = Object.keys(zip.files)
 .filter(name => /^ppt\/slides\/slide\d+\.xml$/i.test(name))
 .sort((a,b) => {
 const an = parseInt((a.match(/slide(\d+)\.xml/i) || [0,0])[1], 10);
 const bn = parseInt((b.match(/slide(\d+)\.xml/i) || [0,0])[1], 10);
 return an - bn;
 });
 const noteFiles = Object.keys(zip.files)
 .filter(name => /^ppt\/notesSlides\/notesSlide\d+\.xml$/i.test(name))

```

```

 .sort((a,b) => {
 const an = parseInt((a.match(/notesSlide(\d+)\.xml/i) || [0,0])[1], 10);
 const bn = parseInt((b.match(/notesSlide(\d+)\.xml/i) || [0,0])[1], 10);
 return an - bn;
 });

const readTextRuns = async (name) => {
 const xml = await zip.files[name].async('string');
 const runs = [];
 const regex = /<a:t[^\>]*>([\s\S]*?)<\a:t>/g;
 let match;
 while((match = regex.exec(xml))){
 const text = decodeXmlEntities(match[1]).replace(/\s+/g, ' ').trim();
 if(text) runs.push(text);
 }
 if(runs.length) return runs.join(' ');
 return stripXmlTags(xml);
};

const chunks = [];
for(const name of slideFiles){
 const slideNo = parseInt((name.match(/slide(\d+)\.xml/i) || [0, chunks.length + 1])[1], 10);
 const slideText = await readTextRuns(name);
 const relatedNotesName = noteFiles.find(note =>
note.toLowerCase().endsWith(`notesslide${slideNo}.xml`));
 const notesText = relatedNotesName ? await readTextRuns(relatedNotesName) : '';
 const combined = [slideText, notesText ? `Speaker notes: ${notesText}` :
''].filter(Boolean).join('\n');
 if(combined.trim()) chunks.push(`Slide ${slideNo}:\n${combined.trim()}`);
}
return chunks.join('\n\n').trim();
}

async function addPromptDocumentToState(file){
 const kind = isPdfFile(file) ? 'pdf' : isPptxFile(file) ? 'pptx' : '';
 if(!kind) return false;
 const name = file.name || `Source ${state.promptDocuments.length + 1}.${kind}`;
 const doc = {
 id: Date.now() + '_' + Math.random().toString(36).slice(2,7),
 name,
 kind,
 mimeType: kind === 'pdf' ? 'application/pdf' :
'application/vnd.openxmlformats-officedocument.presentationml.presentation',
 size: file.size || 0,
 addedAt: new Date().toISOString(),
 extractedText: ''
 };

 if(kind === 'pdf'){

```

```

 const dataUrl = await fileToDataUrl(file);
 doc.dataUrl = dataUrl;
 doc.base64 = dataUrl.split(',')[1];
 doc.extractedText = '';
 } else if (kind === 'pptx') {
 doc.extractedText = await extractPptxText(file);
 if (!doc.extractedText) {
 throw new Error('No readable text was found in that PPTX. Export it as PDF if
the slides are mostly visual.');
```

```

 }
 }

 state.promptDocuments.push(doc);
 return true;
}
```

```

async function addPromptSourceToState(file) {
 if (!file) return false;
 if ((file.type || '').startsWith('image/')) return addPromptImageToState(file);
 if (isPdfFile(file) || isPptxFile(file)) return addPromptDocumentToState(file);
 return false;
}
```

```

function removePromptDocument(idx) {
 state.promptDocuments.splice(idx, 1);
 renderPromptImageThumbs();
 savePrefs();
}
```

```

function sourceFileMeta(doc) {
 if (!doc) return '';
 if (doc.kind === 'pdf') return 'PDF attached for AI reading';
 const chars = String(doc.extractedText || '').length;
 return `PPTX text extracted${chars ? ` • ${chars.toLocaleString()} chars` : ''}`;
}
```

```

function truncateForPrompt(str='', max=12000) {
 const value = String(str || '').trim();
 if (value.length <= max) return value;
 return value.slice(0, max) + `\n\n[Truncated after ${max.toLocaleString()}
characters.]`;
}
```

```

function buildPromptDocumentContextText() {
 const docs = state.promptDocuments || [];
 if (!docs.length) return '';
 return docs.map((doc, idx) => {
 const label = doc.kind === 'pdf' ? 'PDF source' : 'PPTX source';
 const attachedNote = doc.kind === 'pdf'
 ? 'This PDF is attached to the message. Read it for topic, structure,
```

vocabulary, examples, sequencing and task ideas.'

: 'The PPTX file itself is not sent to the AI; its extracted slide text is provided below.';

```
const extracted = doc.extractedText
 ? `\\nExtracted text:\\n"${truncateForPrompt(doc.extractedText, 14000)}"\\n`
 : '';
return `${label} ${idx + 1}: ${doc.name}\\n${attachedNote}${extracted}`;
}).join('\\n\\n');
}
```

```
function buildPromptDocumentInstruction(){
 const docs = state.promptDocuments || [];
 if(!docs.length) return 'No PDF or PPTX source documents are attached.';
 const names = docs.map(doc => `${doc.name} (${String(doc.kind ||
'file')).toUpperCase()})`).join(', ');
 return `Source documents: ${docs.length} PDF/PPTX file${docs.length === 1 ? '' :
's'} supplied: ${names}. Use them as the basis and inspiration for the worksheet.
If they are lesson plans, turn the learning sequence into student-facing worksheet
tasks. If they are slides, preserve the core teaching progression and examples, but
create original worksheet wording. Do not copy long passages verbatim unless the
teacher clearly supplied their own text to reuse.`;
}
```

```
async function addImagesFromClipboardItems(items){
 let added = 0;
 for(const item of [...items || []]){
 if(item.kind === 'file' && (item.type || '').startsWith('image/')){
 const file = item.getAsFile?();
 if(file && await addImageToState(file, 'Pasted screenshot')) added += 1;
 }
 }
 return added;
}
```

```
async function addImagesFromClipboardApi(items){
 let added = 0;
 for(const item of [...items || []]){
 const imageType = (item.types || []).find(type => type.startsWith('image/'));
 if(!imageType) continue;
 const blob = await item.getType(imageType);
 const file = new File([blob], `Clipboard image ${state.images.length + added +
1}.${imageType.split('/')[1] || 'png'}`, { type:imageType });
 if(await addImageToState(file, 'Clipboard image')) added += 1;
 }
 return added;
}
```

```
async function addPromptImagesFromClipboardItems(items){
 let added = 0;
```

```

 for(const item of [...items || []]){
 if(item.kind === 'file' && (item.type || '').startsWith('image/')){
 const file = item.getAsFile?();
 if(file && await addPromptImageToState(file, 'Pasted example')) added += 1;
 }
 }
 return added;
 }
}

async function addPromptImagesFromClipboardApi(items){
 let added = 0;
 for(const item of [...items || []]){
 const imageType = (item.types || []).find(type => type.startsWith('image/'));
 if(!imageType) continue;
 const blob = await item.getType(imageType);
 const file = new File([blob], `Prompt image ${state.promptImages.length + added + 1}.${imageType.split('/')[1] || 'png'}`, { type:imageType });
 if(await addPromptImageToState(file, 'Prompt image')) added += 1;
 }
 return added;
}

function renderImageThumbs(){
 const wrap = qs('imageThumbs');
 if(!wrap) return;
 if(!state.images.length){
 wrap.innerHTML = '';
 return;
 }
 wrap.innerHTML = state.images.map((img, idx)=>`
 <div class="thumb">
 <button class="remove" onclick="removeImage(${idx})" title="Remove
image">x</button>

 <div class="tiny"
style="margin-top:6px;white-space:nowrap;overflow:hidden;text-overflow:ellipsis">${
esc(img.name)}</div>
 </div>
 `).join('');
}

function renderPromptImageThumbs(){
 const wrap = qs('promptImageThumbs');
 const status = qs('promptImageFileStatus');
 const imageCount = (state.promptImages || []).length;
 const docCount = (state.promptDocuments || []).length;
 const total = imageCount + docCount;
 if(status) status.textContent = total ? `${total} source file${total === 1 ? '' : 's'} added.` : 'No source files added.';
 if(!wrap) return;
 if(!total){

```

```

 wrap.innerHTML = '';
 return;
 }
 const imageCards = (state.promptImages || []).map((img, idx)=>`
 <div class="thumb">
 <button class="remove" onclick="removePromptImage(${idx})" title="Remove
example image">x</button>

 <div class="tiny"
style="margin-top:6px;white-space:nowrap;overflow:hidden;text-overflow:ellipsis">${
esc(img.name)}</div>
 </div>
 `).join('');
 const documentCards = (state.promptDocuments || []).map((doc, idx)=>`
 <div class="thumb source-file-card">
 <button class="remove" onclick="removePromptDocument(${idx})" title="Remove
source file">x</button>
 <div class="source-file-icon">${esc(String(doc.kind ||
'file')).toUpperCase())}</div>
 <div class="source-file-name" title="${esc(doc.name)}">${esc(doc.name)}</div>
 <div class="source-file-meta">${esc(sourceFileMeta(doc))}</div>
 </div>
 `).join('');
 wrap.innerHTML = imageCards + documentCards;
}
function removeImage(idx){
 const removed = state.images[idx];
 state.images.splice(idx,1);
 if(state.worksheet?.worksheet_images?.length && removed?.dataUrl){
 state.worksheet.worksheet_images = state.worksheet.worksheet_images.filter(img
=> img.dataUrl !== removed.dataUrl);
 renderEditor();
 }
 renderImageThumbs();
 updatePreview();
 savePrefs();
}
function removePromptImage(idx){
 state.promptImages.splice(idx,1);
 renderPromptImageThumbs();
 savePrefs();
}

function fileToDataURL(file){
 return new Promise((resolve,reject)=>{
 const reader = new FileReader();
 reader.onload = ()=>resolve(reader.result);
 reader.onerror = reject;
 reader.readAsDataURL(file);
 });
}

```

```

}

function difficultyInstruction(val){
 const parsed = parseDifficulty(normaliseDifficulty(val));
 const year = getStudioYearLevel();
 switch(parsed.level){
 case 'easy':
 return `Create a very accessible worksheet for ${year} students. Keep
instructions clear, language simple, scaffolding strong and cognitive load light.`;
 case 'challenging':
 return `Create a challenging worksheet for confident ${year} students.
Require stronger thinking, precision and independence while staying
age-appropriate.`;
 case 'advanced':
 return `Create an advanced worksheet for high-attaining ${year} students.
Push complexity, reasoning, depth and precision while staying age-appropriate.`;
 default:
 return `Create a solid moderate-difficulty worksheet for mainstream ${year}
students.`;
 }
}

/* — Unified prompt builder —————
Every AI call in CycleStudio goes through this function.
Modes: generate, modify, regenerate_task, variation,
 regenerate_section, answer_key, detect_los
*/

function buildAIPrompt(opts = {}){
 const mode = opts.mode || 'generate';
 const difficulty = difficultyInstruction(qs('difficulty')?.value ||
'cm1_moderate');
 const cycleLabel = getCycleLabel();
 const yearLevel = getStudioYearLevel();
 const cycleGuidance = studioCyclePromptGuidance();
 const fallbackDomain = defaultStudioDomain();
 const brief = qs('worksheetBrief')?.value.trim() || '';
 const briefAnchor = brief
 ? `Keep the worksheet topic anchored to this teacher brief:\n"${brief}"`
 : 'No separate brief supplied; use the worksheet itself as the anchor.';
 const imageCount = (state.images || []).length;
 const promptImageCount = (state.promptImages || []).length;
 const promptDocumentCount = (state.promptDocuments || []).length;
 const imageUsage = imageUsageInstruction(getImageUsage(), imageCount);
 const promptReferenceInstruction = [
 promptImageCount
 ? `Reference images: ${promptImageCount} example activity image(s) are
attached. Use them as inspiration for the activity style, task type, structure,
layout, and level of detail. Create original activities inspired by them. Do not
copy worksheet text verbatim.`
 : 'No example activity images are attached.',
]

```

```

 buildPromptDocumentInstruction()
].join('\n');
 const promptDocumentContext = promptDocumentCount ?
buildPromptDocumentContextText() : '';
 const taskCountAiSuggested = isTaskCountAiSuggested();
 const taskCount = taskCountAiSuggested ? (state.worksheet?.tasks?.length || 6) :
(parseInt(qs('questionCount')?.value, 10) || state.worksheet?.tasks?.length || 6);
 const taskCountInstruction = taskCountAiSuggested
 ? 'The teacher selected AI suggested for task count. Choose a sensible number
of tasks for the worksheet brief, normally 3-8. Return that concrete number of task
objects; do not write "AI suggested" anywhere in the worksheet.'
 : `Target approximately ${taskCount} tasks.`;

 const preamble = `You are working on a printable worksheet for ${cycleLabel}
(${yearLevel}) students at an International French School. Write everything in
English only. ${cycleGuidance}`;
 const worksheetJson = state.worksheet ? JSON.stringify(buildWorksheetAiContext(),
null, 2) : '{}';
 const imageContextNote = 'Image data inside the worksheet JSON has been
summarised to keep this request lightweight. If relevant task images were attached
with the request, use them as the visual reference.';

/* — Generate from scratch — */
if(mode === 'generate'){
 const curriculumMode = state.curriculumMode || 'ai_both';
 const domain = qs('domainSelect')?.value.trim() || '';
 const fixedDomain = domain || fallbackDomain;
 const outcomes = [...state.selectedOutcomes];
 const templateText = qs('templateText')?.value.trim() || '';
 const includeAnswers = shouldGenerateAnswers();

 const allowedDomainBank = getAllowedDomainCatalogueText();
 const allowedOutcomeBank = getAllowedDomainCatalogueText(fixedDomain);

 const domainInstruction = curriculumMode === 'ai_both'
 ? `Choose the single best-fitting ${cycleLabel} domain from this exact list
only: ${DOMAIN_NAMES.join(', ')}. You must return one exact domain string from the
list with no variation.`
 : `Use this domain exactly: ${fixedDomain}. Do not change it or rename it.`;

 const outcomeInstruction = curriculumMode === 'ai_both'
 ? `Choose 2 to 4 learning outcomes by copying them VERBATIM from the allowed
curriculum bank for the chosen domain. Do not invent, paraphrase, merge or rewrite
outcomes. Here is the exact allowed bank:\n${allowedDomainBank}`
 : curriculumMode === 'manual_domain_ai_lo'
 ? `Choose 2 to 4 learning outcomes by copying them VERBATIM from this exact
allowed bank for the fixed domain "${fixedDomain}" only. Do not invent, paraphrase,
merge or rewrite outcomes:\n${allowedOutcomeBank}`
 : outcomes.length
 ? `Use these learning outcomes exactly as provided and do not invent

```



```

extras:\n${outcomes.map(o=>' - ' + o).join('\n')}`
 : `Choose 2 to 4 learning outcomes by copying them VERBATIM from this
exact allowed bank for the fixed domain "${fixedDomain}" only. Do not invent,
paraphrase, merge or rewrite outcomes:\n${allowedOutcomeBank}`;

 const briefInstruction = brief
 ? `The teacher brief below is the fixed topic and focus of the worksheet. You
may expand it into a fuller classroom-ready concept, but you must stay on this
topic and must not replace it with a different subject:\n"${brief}"`
 : `No teacher brief was provided. Invent a coherent worksheet concept that
fits the selected difficulty band. Keep it practical, printable and
classroom-ready.`;

 const templateInstruction = templateText
 ? brief
 ? `A template or amendment note has been provided. Use it to shape
structure, tone or task style, but do not let it override the teacher
brief:\n""""${templateText}""""`
 : `A template or amendment note has been provided. Use it as structural
inspiration or source material to adapt and improve, but do not copy large chunks
verbatim:\n""""${templateText}""""`
 : `No text template note has been provided.`;

 const genImageInstruction = imageUsageInstruction(getImageUsage(), imageCount);

 const exportInstruction = includeAnswers
 ? 'Generate accurate teacher answers in the worksheet JSON so the finished
resource can later be exported either as a student sheet or as a student sheet with
answer key.'
 : 'Do not generate the teacher answer key now. Leave task answers blank or
omit them entirely so only the student sheet is generated.';

 return `You are designing a printable worksheet for an International French
School teacher working with ${cycleLabel} (${yearLevel}) students.
Write EVERYTHING in English only.
Keep the final product clean, practical, school-appropriate and easy to print on A4
portrait pages.
${cycleGuidance}
${difficulty}
${taskCountInstruction}
${domainInstruction}
${outcomeInstruction}
${briefInstruction}
${templateInstruction}
${genImageInstruction}
${promptReferenceInstruction}
${promptDocumentContext}
${exportInstruction}

```

Leave "instructions", "success\_criteria" and "extension" as empty strings by

default unless the teacher explicitly asks for those sections in the brief.

```
${schemaHint(taskCount, includeAnswers, taskCountAiSuggested)}
```

Extra rule: when a teacher brief is provided, treat it as non-negotiable topic control. Do not swap it for a different subject.`;

```
}
```

```
/* — Modify existing worksheet — */
```

```
if(mode === 'modify'){
 return `${preamble}
```

```
${difficulty}
```

The teacher request is:

```
"${opts.request || ''}"
```

Current worksheet JSON:

```
${worksheetJson}
```

```
${imageContextNote}
```

IMPORTANT:

- Preserve existing task numbers. If you add content related to an existing task, modify that task in-place rather than creating a new one.
- Tasks with `image_enabled:true` have images attached. Do NOT remove or replace those tasks unless the teacher explicitly asks you to. Keep their task number, `image_enabled` flag, and image settings intact.
- When adding new tasks, append them after existing ones.

Return ONLY valid JSON using the same schema and preserve sensible structure.`;

```
}
```

```
/* — Regenerate a single task — */
```

```
if(mode === 'regenerate_task'){
 const idx = opts.taskIdx ?? 0;
 return `${preamble} ${difficulty}
```

```
${briefAnchor}
```

Worksheet context JSON:

```
${worksheetJson}
```

Regenerate ONLY task number `${idx + 1}`. Keep the same overall topic, domain alignment and learning-outcome alignment. Make it fresh and classroom-ready. You may improve the task type if helpful, but do not derail the worksheet.

```
${imageUsage}
```

Return ONLY valid JSON with this exact schema:

```
{
 "passage": "<optional separate passage, sentence set, word bank or stimulus text>",
 "prompt": "<task text>",
 "options": ["<option A>", "<option B>"],
 "answer_lines": 0,
 "answer": "<teacher answer or marking point>"
}
```

Rules:

- Keep the task suitable for the worksheet level.
- `${imageContextNote}`
- Use "passage" only if the task genuinely needs separate stimulus text.
- Use "options" only if they help.
- Keep "answer\_lines" at 0 unless the teacher explicitly asked for answer lines.
- Keep it printable and coherent.`;

```
/* — Generate a variation — */
if(mode === 'variation'){
 return `${preamble} ${difficulty}
```

Current worksheet JSON:

```
${worksheetJson}
```

```
${briefAnchor}
```

```
Variation instruction: ${variationInstruction(opts.variationMode || 'easier',
opts.extraTopic || '')}
```

```
${imageUsage}
```

```
${schemaHint(taskCount, undefined, taskCountAiSuggested)}
```

```
Keep the result practical, printable and coherent.`;
}
```

```
/* — Regenerate a single section (instructions / success_criteria / extension)
— */
if(mode === 'regenerate_section'){
 const meta = typeof getWorksheetSectionMeta === 'function' ?
getWorksheetSectionMeta(opts.sectionKey) : null;
 return `${preamble} ${difficulty}
```

```
${briefAnchor}
```

Current worksheet JSON:

```
${worksheetJson}
```

```
${meta?.regenPrompt || 'Regenerate the requested section.'}
```

`${imageUsage}`

Return ONLY valid JSON with this exact schema:

```
{
 "text": "<the regenerated content only>"
}
```

Rules:

- Keep the same worksheet topic, domain alignment and age level.
- `${imageContextNote}`
- Do not rewrite the whole worksheet.
- Do not include labels like "Instructions:" or "Success criteria:" in the text.
- Keep it printable and concise.`;

```
/* — Generate answer key only — */
if(mode === 'answer_key'){
 return `You are generating ONLY the teacher answer key for an existing
printable ${cycleLabel} (${yearLevel}) worksheet. Write everything in English only.
${cycleGuidance} ${difficulty}
```

Current worksheet JSON:

`${worksheetJson}`

`${briefAnchor}`

Return ONLY valid JSON with this exact schema:

```
{
 "tasks": [
 {
 "number": 1,
 "answer": "<teacher answer or marking point>"
 }
]
}
```

Rules:

- Return one item for every task in the worksheet.
- Keep the task numbers exactly aligned with the current worksheet.
- Do not rewrite prompts, passages, options, domain, title or learning outcomes.
- For open-ended tasks, provide concise marking guidance, acceptable answers or key points rather than a fake single perfect sentence.
- Keep answers practical for a teacher using an answer key.`;

```
/* — Auto-detect learning outcomes — */
if(mode === 'detect_los'){
 const domain = canonicaliseDomain(state.worksheet?.domain || '') ||
fallbackDomain;
```

```

const allOutcomes = getFlatDomainOutcomes(domain);
const context = {
 domain,
 title: state.worksheet?.title || '',
 tasks: (state.worksheet?.tasks || []).map(t => ({
 prompt: t.prompt || '',
 passage: t.passage || '',
 options: t.options || []
 })),
 instructions: state.worksheet?.instructions || '',
 extension: state.worksheet?.extension || ''
};
return `You are analysing an existing ${cycleLabel} (${yearLevel}) worksheet to
identify which learning outcomes it actually addresses.
${cycleGuidance}

```

Worksheet JSON:

```

${JSON.stringify(context, null, 2)}

```

Here is the complete list of valid learning outcomes for the "\${domain}" domain:

```

${allOutcomes.map((o, i) => `${i + 1}. ${o}`).join('\n')}

```

Select the outcomes from the list above that this worksheet genuinely addresses based on its tasks. Choose 1-5 outcomes. Only pick outcomes that clearly match the worksheet content.

Return ONLY valid JSON:

```

{ "outcome_indexes": [1, 3, 5] }

```

Use the 1-based index numbers from the list above.`;

```

}

```

```

return '';
}

```

```

async function callAI({prompt, modify=false, extraImages=[], extraDocuments=[]}){
 const apiKey = qs('apiKey').value.trim();
 const model = qs('modelName').value.trim() || 'claude-opus-4-5';
 if(!apiKey){ if(typeof openSettingsModal === 'function') openSettingsModal();
 throw new Error('Add your API key in Settings first.')} }

```

```

const content = [];
const pdfDocuments = [];
if(state.templatePdf?.base64){
 pdfDocuments.push({ name: state.templatePdf.name || 'Template PDF', base64:
state.templatePdf.base64 });
}
(Array.isArray(extraDocuments) ? extraDocuments : []).forEach(doc => {
 if(doc?.kind === 'pdf' && doc?.base64){
 pdfDocuments.push({ name: doc.name || 'Source PDF', base64: doc.base64 });
 }
}

```

```

 }
 });
 pdfDocuments.forEach(doc => {
 content.push({ type:'text', text:`Attached PDF source: ${doc.name}` });
 content.push({
 type: 'document',
 source: {
 type: 'base64',
 media_type: 'application/pdf',
 data: doc.base64
 }
 });
 });
 if(!modify){
 state.images.forEach(img => {
 content.push({
 type:'image',
 source:{ type:'base64', media_type: img.mimeType, data: img.base64 }
 });
 });
 }
 (Array.isArray(extraImages) ? extraImages : []).forEach(img => {
 if(!img?.base64) return;
 content.push({
 type:'image',
 source:{ type:'base64', media_type: img.mimeType || 'image/png', data:
img.base64 }
 });
 });
 content.push({ type:'text', text: prompt });

 const headers = {
 'Content-Type':'application/json',
 'x-api-key': apiKey,
 'anthropic-version':'2023-06-01',
 'anthropic-dangerous-direct-browser-access':'true'
 };
 if(pdfDocuments.length) headers['anthropic-beta'] = 'pdfs-2024-09-25';

 const res = await fetch('https://api.anthropic.com/v1/messages', {
 method:'POST',
 headers,
 body: JSON.stringify({
 model,
 max_tokens: 4000,
 messages: [{ role:'user', content }]
 })
 });

 let data;

```

```

try{ data = await res.json(); }
catch(err){ throw new Error('Could not read the AI response.');
```

```

 if(!res.ok || data.error){
 throw new Error(data.error?.message || ('API error ' + res.status));
 }

 const raw = (data.content || []).map(block => block.text || '').join('').trim();
 const cleaned = raw.replace(/```json|```/g, '').trim();
 try{
 return JSON.parse(cleaned);
 }catch(err){
 console.error(cleaned);
 throw new Error('The AI returned something that was not valid JSON.');
```

```

 }
 }

 async function generateWorksheet(){
 clearStatus();
 savePrefs();
 qs('generateBtn').disabled = true;
 qs('generateBtn').textContent = 'Generating...';
 try{
 if(state.curriculumMode !== 'ai_both' && !qs('domainSelect').value){
 throw new Error('Choose a domain or switch the curriculum mode back to full
AI.');
```

```

 }
 if(state.curriculumMode === 'manual_both' && !state.selectedOutcomes.length){
 throw new Error('Choose at least one learning outcome or switch to AI-picked
learning outcomes.');
```

```

 }
 const wantAnswers = shouldGenerateAnswers();
 const prompt = buildAIPrompt({ mode: 'generate' });
 showStatus(wantAnswers ? 'Generating worksheet and answer key...' : 'Generating
student sheet only...', 'warn');
```

```

 const result = await callAI({prompt, extraImages: state.promptImages || [],
extraDocuments: state.promptDocuments || []});
 state.worksheet = normaliseWorksheet(result);
 state.qualityCheck = null;
 if(!wantAnswers) stripTaskAnswers(state.worksheet);
 state.worksheet.instructions = '';
 state.worksheet.success_criteria = '';
 state.worksheet.extension = '';
 if(qs('outputMode') && !hasAnswerKeyData(state.worksheet))
qs('outputMode').value = 'student';
 pushHistorySnapshot('Generated worksheet');
 renderEditor();
 updatePreview();
 showStatus(wantAnswers ? 'Worksheet and answer key generated.' : 'Student sheet
generated without answer key.', 'ok');
```

```

 if(typeof goWizardStep === 'function') goWizardStep(4);
 }catch(err){
 showStatus(err.message || 'Generation failed.', 'err');
 }finally{
 qs('generateBtn').disabled = false;
 qs('generateBtn').textContent = 'Generate worksheet';
 }
}

```

```

async function modifyWorksheet(){
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 const request = qs('modifyRequest').value.trim();
 if(!request) return showStatus('Add a modification request first.', 'warn');
 setEditorAiBusy(true, 'Applying AI changes to the worksheet...');
 try{
 showStatus('Modifying worksheet...', 'warn');
 const previousWorksheet = JSON.parse(JSON.stringify(state.worksheet || {}));
 const hadAnswersBefore = hasAnswerKeyData();
 const targetTaskIndexes = getLikelyTargetTaskIndexesFromRequest(request);
 const allowTaskCountChange = requestExplicitlyWantsTaskAdditions(request) ||
requestExplicitlyWantsTaskRemovals(request);
 const explicitAncillary = requestExplicitlyWantsAncillarySections(request);
 const currentTaskCount = Array.isArray(previousWorksheet.tasks) ?
previousWorksheet.tasks.length : 0;
 const targetNote = targetTaskIndexes.length
 ? `Likely target task(s): ${targetTaskIndexes.map(idx => idx + 1).join(',
')}.`
 : 'No task number was clearly referenced.';
 const scopeRule = !allowTaskCountChange && targetTaskIndexes.length === 1
 ? `Modify only task ${targetTaskIndexes[0] + 1}. Keep all other tasks
unchanged.`
 : !allowTaskCountChange && targetTaskIndexes.length > 1
 ? `Modify only tasks ${targetTaskIndexes.map(idx => idx + 1).join(', ')}.
Keep all other tasks unchanged.`
 : 'Keep edits tightly scoped to the teacher request.';
 const taskRule = allowTaskCountChange
 ? 'Only change the number of tasks if the teacher explicitly asked for that.'
 : `Return exactly ${currentTaskCount} tasks. Do not add, remove, split or
append tasks.`;
 const ancillaryRule = explicitAncillary
 ? 'Only change instructions, success criteria or extension if the teacher
explicitly asked for them.'
 : 'Leave "instructions", "success_criteria" and "extension" unchanged. Do not
generate them or expand them.';
 const prompt = `You are revising an existing printable worksheet for
${getCycleLabel()} (${getStudioYearLevel()}) students. Write everything in English
only. ${studioCyclePromptGuidance()}
${difficultyInstruction(qs('difficulty')?.value || 'cm1_moderate')}`

```

The teacher request is:



"\${request}"

Current worksheet JSON:

```
${JSON.stringify(buildWorksheetAiContext(), null, 2)}
```

Image data inside the worksheet JSON has been summarised to keep this request lightweight. If relevant task images were attached with the request, use them as the visual reference.

\${targetNote}

IMPORTANT:

- Return the FULL worksheet JSON, not just a partial patch.
- Preserve existing task numbers.
- \${taskRule}
- \${scopeRule}
- \${ancillaryRule}
- Tasks with image\_enabled:true have images attached. Do NOT remove or replace those tasks unless the teacher explicitly asks you to.
- If the request refers to an image and only one task clearly contains an image, prioritise that task.
- Keep the worksheet coherent, printable and aligned to the current topic and level.

Return ONLY valid JSON using the same worksheet schema.`;

```
const oldTasks = (previousWorksheet.tasks || []).map(t => ({
 number: t.number,
 image_enabled: t.image_enabled,
 image_prompt: t.image_prompt,
 image_size: t.image_size,
 image_width: t.image_width,
 image: t.image ? Object.assign({}, t.image) : null,
 images: Array.isArray(t.images) ? t.images.map(img => Object.assign({}, img))
: (t.image ? [Object.assign({}, t.image)] : []),
 pdf_images: Array.isArray(t.pdf_images) ? t.pdf_images.map(p =>
Object.assign({}, p)) : []
}));
const result = await callAI({prompt, modify:true, extraImages:
collectImagesForRequest(request)});
const merged = mergeModifyResultIntoWorksheet(previousWorksheet, result,
targetTaskIndexes, allowTaskCountChange);
state.worksheet = normaliseWorksheet(merged);
state.qualityCheck = null;
/* Restore image data the AI could not return */
(state.worksheet.tasks || []).forEach((task, idx) => {
 const donor = oldTasks.find(o => o.number === task.number) || oldTasks[idx];
 if(!donor) return;
 if(!task.images || !task.images.length) && donor.images &&
donor.images.length) task.images = donor.images.map(img => Object.assign({}, img));
 if(!task.image && donor.image) task.image = donor.image;
```

```

 if(task.images && task.images.length) task.image = task.images[0];
 if(!task.pdf_images || !task.pdf_images.length) && donor.pdf_images.length)
task.pdf_images = donor.pdf_images;
 if(!task.image_enabled && donor.image_enabled && (task.image ||
(task.pdf_images && task.pdf_images.length))){
 task.image_enabled = true;
 if(!task.image_prompt && donor.image_prompt) task.image_prompt =
donor.image_prompt;
 if(!task.image_size && donor.image_size) task.image_size =
donor.image_size;
 if((task.image_width === undefined || task.image_width === 100) &&
donor.image_width !== undefined) task.image_width = donor.image_width;
 }
 });
 if(!explicitAncillary){
 state.worksheet.instructions = previousWorksheet.instructions || '';
 state.worksheet.success_criteria = previousWorksheet.success_criteria || '';
 state.worksheet.extension = previousWorksheet.extension || '';
 }
 if(!hadAnswersBefore && !shouldGenerateAnswers())
stripTaskAnswers(state.worksheet);
 if(qs('outputMode') && !hasAnswerKeyData(state.worksheet))
qs('outputMode').value = 'student';
 pushHistorySnapshot('AI modified worksheet');
 forceEditorRefresh();
 showStatus('Worksheet updated.', 'ok');
} catch(err){
 showStatus(err.message || 'Could not modify the worksheet.', 'err');
} finally{
 setEditorAiBusy(false);
 forceEditorRefresh();
}
}

function setWorksheetDomain(val){
 if(!state.worksheet) return;
 state.worksheet.domain = val;
 queuePreviewUpdate();
}

function updateOutcome(idx, value){
 if(!state.worksheet) return;
 const old = state.worksheet.learning_outcomes[idx];
 state.worksheet.learning_outcomes[idx] = value;
 if(old !== value){
 state.loStatus[value] = state.loStatus[old] || '';
 delete state.loStatus[old];
 }
 queuePreviewUpdate();
}

function addOutcome(){

```

```

 if(!state.worksheet) return;
 state.worksheet.learning_outcomes.push('New learning outcome');
 state.loStatus['New learning outcome'] = '';
 renderEditor();
 updatePreview();
 }
 function removeOutcome(idx){
 if(!state.worksheet) return;
 const old = state.worksheet.learning_outcomes[idx];
 state.worksheet.learning_outcomes.splice(idx,1);
 delete state.loStatus[old];
 renderEditor();
 updatePreview();
 }

 function toggleEditorOutcomeBrowser(){
 const panel = qs('editorOutcomeBrowser');
 if(!panel) return;
 const isHidden = panel.classList.contains('hidden');
 panel.classList.toggle('hidden', !isHidden);
 if(isHidden) renderEditorOutcomeBrowser();
 }

 function renderEditorOutcomeBrowser(){
 const box = qs('editorOutcomesBox');
 if(!box || !state.worksheet) return;
 const domain = canonicaliseDomain(state.worksheet.domain || '') ||
 defaultStudioDomain();
 const current = new Set(state.worksheet.learning_outcomes || []);
 const grouped = getGroupedDomainOutcomes(domain);
 if(grouped){
 const strands = Object.entries(grouped);
 box.innerHTML = strands.map(([strand, outcomes], sIdx)=>{
 const checkedCount = outcomes.filter(o => current.has(o)).length;
 const badge = checkedCount ? `${checkedCount} selected` : '';
 return `
 <div class="outcome-group" data-strand-idx="${sIdx}">
 <div class="outcome-group-title" style="cursor:pointer;user-select:none"
onclick="toggleEditorStrand(${sIdx})">
 <span
style="display:inline-block;width:16px;text-align:center;margin-right:4px"
id="editorStrandArrow_${sIdx}">>
 ${esc(strand)}${badge}
 </div>
 <div class="outcome-group-list hidden" id="editorStrandList_${sIdx}">
 ${outcomes.map(outcome=>`
 <label class="outcome-option">
 <input type="checkbox" data-outcome="${esc(outcome)}"
${current.has(outcome) ? 'checked' : ''} onchange="toggleEditorOutcome(this)">

```

```

 ${esc(outcome)}
 </label>
 `).join('')
</div>
</div>`;
}).join('');
} else {
 const outcomes = getFlatDomainOutcomes(domain);
 box.innerHTML = outcomes.map(outcome=>`
 <label class="outcome-option">
 <input type="checkbox" data-outcome="${esc(outcome)}"
${current.has(outcome) ? 'checked' : ''} onchange="toggleEditorOutcome(this)">
 ${esc(outcome)}
 </label>
 `).join('');
}
if(!box.innerHTML.trim()) box.innerHTML = '<div class="tiny">No outcomes found
for this domain.</div>';
}

```

```

function toggleEditorStrand(sIdx){
 const list = qs('editorStrandList_' + sIdx);
 const arrow = qs('editorStrandArrow_' + sIdx);
 if(!list) return;
 const opening = list.classList.contains('hidden');
 list.classList.toggle('hidden', !opening);
 if(arrow) arrow.textContent = opening ? '▼' : '►';
}

```

```

function toggleEditorOutcome(el){
 if(!state.worksheet) return;
 const value = el.getAttribute('data-outcome');
 if(!value) return;
 const outcomes = state.worksheet.learning_outcomes;
 if(el.checked){
 if(!outcomes.includes(value)){
 outcomes.push(value);
 state.loStatus[value] = state.loStatus[value] || '';
 }
 } else {
 const idx = outcomes.indexOf(value);
 if(idx !== -1){
 outcomes.splice(idx, 1);
 delete state.loStatus[value];
 }
 }
 state.selectedOutcomes = [...outcomes];
 /* Re-render outcome list but keep browser open */
 const list = qs('editorOutcomeList');
 if(list) list.innerHTML = outcomes.map((outcome, idx)=>`

```

```

 <div class="task-card" style="padding:10px 12px">
 <div class="task-head">
 Outcome ${idx + 1}
 <button class="mini danger" onclick="removeOutcome(${idx})">Remove</button>
 </div>
 <textarea oninput="updateOutcome(${idx}, this.value)"
style="min-height:72px">${esc(outcome)}</textarea>
 </div>
 `).join('');
 updatePreview();
}

```

```

async function autoDetectLearningOutcomes(){
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 const domain = canonicaliseDomain(state.worksheet.domain || '') ||
defaultStudioDomain();
 const allOutcomes = getFlatDomainOutcomes(domain);
 if(!allOutcomes.length) return showStatus('No outcomes available for this
domain.', 'warn');
 const btn = qs('autoDetectLosBtn');
 if(btn){ btn.disabled = true; btn.textContent = 'Detecting...'; }
 setEditorAiBusy(true, 'Analysing worksheet to detect learning outcomes...');
 try{
 const prompt = buildAIPrompt({ mode: 'detect_los' });
 const result = await callAI({prompt, modify:false});
 const indexes = Array.isArray(result?.outcome_indexes) ? result.outcome_indexes
: [];
 const detected = indexes
 .map(i => allOutcomes[parseInt(i, 10) - 1])
 .filter(Boolean);
 if(!detected.length) return showStatus('Could not detect any matching outcomes.
Try manually.', 'warn');
 state.worksheet.learning_outcomes = detected;
 state.selectedOutcomes = [...detected];
 state.loStatus = {};
 detected.forEach(lo => { state.loStatus[lo] = ''; });
 renderEditor();
 updatePreview();
 showStatus(`Detected ${detected.length} learning outcome${detected.length === 1
? '' : 's'}.`, 'ok');
 }catch(err){
 showStatus(err.message || 'Could not auto-detect learning outcomes.', 'err');
 }finally{
 setEditorAiBusy(false);
 forceEditorRefresh();
 if(btn){ btn.disabled = false; btn.textContent = 'Auto-detect LOs'; }
 }
}

```

```

function updateTaskField(idx, field, value){

```

```

 if(!state.worksheet?.tasks[idx]) return;
 state.worksheet.tasks[idx][field] = value;
 queuePreviewUpdate();
 }
 function updateTaskLines(idx, value){
 if(!state.worksheet?.tasks[idx]) return;
 const num = Math.max(0, Math.min(40, parseInt(value,10) || 0));
 state.worksheet.tasks[idx].answer_lines = num;
 queuePreviewUpdate();
 }
 function updateTaskOptions(idx, value){
 if(!state.worksheet?.tasks[idx]) return;
 state.worksheet.tasks[idx].options =
value.split('\n').map(v=>v.trim()).filter(Boolean);
 queuePreviewUpdate();
 }
 function removeTask(idx){
 if(!state.worksheet) return;
 state.worksheet.tasks.splice(idx,1);
 state.worksheet.tasks.forEach((t,i)=>t.number=i+1);
 renderEditor();
 updatePreview();
 }
 function moveTask(idx, dir){
 if(!state.worksheet) return;
 const next = idx + dir;
 if(next < 0 || next >= state.worksheet.tasks.length) return;
 const arr = state.worksheet.tasks;
 [arr[idx], arr[next]] = [arr[next], arr[idx]];
 arr.forEach((t,i)=>t.number=i+1);
 renderEditor();
 updatePreview();
 }
}

function setLOStatus(outcome, status){
 if(state.loStatus[outcome] === status) state.loStatus[outcome] = '';
 else state.loStatus[outcome] = status;
 updatePreview();
}

function renderLOTables(outcomes){
 return `
 <table class="lo-table">
 <thead>
 <tr>
 <th>Learning outcome</th>
 <th class="tick-cell">Met</th>
 <th class="tick-cell">Working towards</th>
 <th class="tick-cell">Not met</th>
 </tr>

```

```

 </thead>
 <tbody>
 ${outcomes.map(outcome=>{
 const current = state.loStatus[outcome] || '';
 return `
 <tr>
 <td>${esc(outcome)}</td>
 <td class="tick-cell ${current==='met'?'active':''}"
onclick="setLOStatus('${jsq(outcome)}', 'met')">${current==='met'?'✓':''}</td>
 <td class="tick-cell ${current==='wt'?'active':''}"
onclick="setLOStatus('${jsq(outcome)}', 'wt')">${current==='wt'?'✓':''}</td>
 <td class="tick-cell ${current==='nm'?'active':''}"
onclick="setLOStatus('${jsq(outcome)}', 'nm')">${current==='nm'?'✓':''}</td>
 </tr>
 `;
 }).join('')}
 </tbody>
 </table>
`;
}

function renderLOList(outcomes){
 return `<ul
class="worksheet-lo-list">${outcomes.map(outcome=>`<li>${esc(outcome)}</li>`).join(
'')}`;
}

function renderLOBlock(outcomes, showAssessmentTable, mode='student'){
 if(!Array.isArray(outcomes) || !outcomes.length) return '';
 const assessmentBlock = showAssessmentTable ? `
 <div style="margin-top:14px">
 <div class="teacher-tag" style="margin-bottom:10px">Assessment table</div>
 ${renderLOTables(outcomes)}
 </div>` : '';
 return `
 <div class="teacher-box">
 <div class="teacher-top">
 <div class="teacher-tag">Learning outcomes</div>
 </div>
 ${renderLOList(outcomes)}
 ${assessmentBlock}
 </div>`;
}

const PAGE_PIXEL_HEIGHT = 1123;

function getPaginationStage(){

```

```

let stage = document.getElementById('paginationMeasureStage');
if(!stage){
 stage = document.createElement('div');
 stage.id = 'paginationMeasureStage';
 stage.style.position = 'fixed';
 stage.style.left = '-10000px';
 stage.style.top = '0';
 stage.style.width = '794px';
 stage.style.visibility = 'hidden';
 stage.style.pointerEvents = 'none';
 stage.style.zIndex = '-1';
 document.body.appendChild(stage);
}
stage.innerHTML = '';
return stage;
}

function fitsSinglePage(pageEl){
 return pageEl.scrollHeight <= PAGE_PIXEL_HEIGHT + 2;
}

function insertBeforeMascot(pageEl, node, containerSelector = null){
 if(containerSelector){
 const target = pageEl.querySelector(containerSelector);
 if(target){
 target.appendChild(node);
 return;
 }
 }
 const mascot = pageEl.querySelector('.mascot');
 if(mascot) pageEl.insertBefore(node, mascot);
 else pageEl.appendChild(node);
}

function buildStudentContinuationPage(originalPage){
 const page = document.createElement('div');
 page.className = 'preview-page';
 const header = originalPage.querySelector('.page-header')?.cloneNode(true);
 if(header){
 const kicker = header.querySelector('.page-kicker');
 if(kicker) kicker.textContent = 'CycleStudio – Continued';
 page.appendChild(header);
 }
 const note = document.createElement('div');
 note.className = 'muted-note';
 note.style.margin = '-4px 0 16px';
 note.textContent = 'Continued from the previous page.';
 page.appendChild(note);
 const mascot = originalPage.querySelector('.mascot')?.cloneNode(true);
 if(mascot) page.appendChild(mascot);
}

```



```

 return page;
}

function paginateStudentHtml(html){
 if(!html) return '';
 const holder = document.createElement('div');
 holder.innerHTML = html.trim();
 const originalPage = holder.firstElementChild;
 if(!originalPage) return html;

 const stage = getPaginationStage();
 const pages = [];
 const fixedSelectors = ['.page-header', '.teacher-box', '.instructions'];
 const fixedNodes = fixedSelectors.map(sel =>
originalPage.querySelector(sel)?.cloneNode(true)).filter(Boolean);
 const flowNodes = Array.from(originalPage.children).filter(el => !(
 el.classList.contains('page-header') ||
 el.classList.contains('teacher-box') ||
 el.classList.contains('instructions') ||
 el.classList.contains('mascot')
));

 let currentPage = document.createElement('div');
 currentPage.className = 'preview-page';
 fixedNodes.forEach(node => currentPage.appendChild(node));
 const mascot = originalPage.querySelector('.mascot)?.cloneNode(true);
 if(mascot) currentPage.appendChild(mascot);
 stage.appendChild(currentPage);

 let blocksOnCurrentPage = 0;
 for(const node of flowNodes){
 const clone = node.cloneNode(true);
 insertBeforeMascot(currentPage, clone);
 if(!fitsSinglePage(currentPage)){
 clone.remove();
 if(blocksOnCurrentPage === 0){
 insertBeforeMascot(currentPage, clone);
 blocksOnCurrentPage++;
 continue;
 }
 }
 pages.push(currentPage.outerHTML);
 stage.innerHTML = '';
 currentPage = buildStudentContinuationPage(originalPage);
 stage.appendChild(currentPage);
 blocksOnCurrentPage = 0;

 const retryClone = node.cloneNode(true);
 insertBeforeMascot(currentPage, retryClone);
 blocksOnCurrentPage++;
 } else {

```

```

 blocksOnCurrentPage++;
 }
}

pages.push(currentPage.outerHTML);
stage.innerHTML = '';
return pages.join('');
}

function buildAnswerContinuationPage(originalPage){
 const page = document.createElement('div');
 page.className = 'preview-page';
 const header = originalPage.querySelector('.page-header')?.cloneNode(true);
 if(header){
 const kicker = header.querySelector('.page-kicker');
 if(kicker) kicker.textContent = 'Teacher copy – Continued';
 page.appendChild(header);
 }
 const note = document.createElement('div');
 note.className = 'muted-note';
 note.style.margin = '-4px 0 16px';
 note.textContent = 'Answer key continued.';
 page.appendChild(note);
 const answerKey = document.createElement('div');
 answerKey.className = 'answer-key';
 answerKey.innerHTML = '<h3>Answers and marking points (continued)</h3>';
 page.appendChild(answerKey);
 const mascot = originalPage.querySelector('.mascot')?.cloneNode(true);
 if(mascot) page.appendChild(mascot);
 return page;
}

function paginateAnswerHtml(html){
 if(!html) return '';
 const holder = document.createElement('div');
 holder.innerHTML = html.trim();
 const originalPage = holder.firstElementChild;
 if(!originalPage) return html;

 const stage = getPaginationStage();
 const pages = [];
 const firstPage = document.createElement('div');
 firstPage.className = 'preview-page';

 const header = originalPage.querySelector('.page-header')?.cloneNode(true);
 if(header) firstPage.appendChild(header);
 const instructions =
originalPage.querySelector('.section-print')?.cloneNode(true);
 if(instructions) firstPage.appendChild(instructions);
 const answerKey = document.createElement('div');

```

```

answerKey.className = 'answer-key';
answerKey.innerHTML = '<h3>Answers and marking points</h3>';
firstPage.appendChild(answerKey);
const mascot = originalPage.querySelector('.mascot')?.cloneNode(true);
if(mascot) firstPage.appendChild(mascot);
stage.appendChild(firstPage);

const items = Array.from(originalPage.querySelectorAll('.answer-key
.answer-item'));
let currentPage = firstPage;
let blocksOnCurrentPage = 0;

for(const item of items){
 const clone = item.cloneNode(true);
 insertBeforeMascot(currentPage, clone, '.answer-key');
 if(!fitsSinglePage(currentPage)){
 clone.remove();
 if(blocksOnCurrentPage === 0){
 insertBeforeMascot(currentPage, clone, '.answer-key');
 blocksOnCurrentPage++;
 continue;
 }
 pages.push(currentPage.outerHTML);
 stage.innerHTML = '';
 currentPage = buildAnswerContinuationPage(originalPage);
 stage.appendChild(currentPage);
 blocksOnCurrentPage = 0;

 const retryClone = item.cloneNode(true);
 insertBeforeMascot(currentPage, retryClone, '.answer-key');
 blocksOnCurrentPage++;
 } else {
 blocksOnCurrentPage++;
 }
}

pages.push(currentPage.outerHTML);
stage.innerHTML = '';
return pages.join('');
}

function fitPreviewToPane(){
 const wrap = document.querySelector('.preview-wrap');
 if(!wrap) return;
 wrap.style.setProperty('--preview-scale', '1');
}

function updatePreview(){
 const root = qs('previewRoot');
 if(!state.worksheet){

```

```

root.innerHTML = `
 <div class="preview-page"
style="display:flex;align-items:center;justify-content:center;text-align:center;color:#587094">
 <div>

 <h3 style="margin:16px 0 8px;font-size:22px;color:#12325f">Resource
preview</h3>
 <p
style="margin:0;font-size:14px;max-width:430px;line-height:1.6">Generate a resource
and it will appear here.</p>
 </div>
 </div>`;
 fitPreviewToPane();
 return;
}

```

```

const mode = qs('outputMode').value;
if(mode !== 'student' && !hasAnswerKeyData()){
 qs('outputMode').value = 'student';
}
if(qs('outputMode').value === 'student'){
 root.innerHTML = paginateStudentHtml(renderStudentPage(false));
} else if(qs('outputMode').value === 'answer'){
 root.innerHTML = paginateStudentHtml(renderStudentPage(true));
} else {
 root.innerHTML = paginateStudentHtml(renderStudentPage(false)) +
paginateAnswerHtml(renderAnswerPage());
}
fitPreviewToPane();
}

function getSavedItems(kind){
 const key = kind === 'draft' ? LS_KEYS.drafts : LS_KEYS.templates;
 try{ return JSON.parse(localStorage.getItem(key) || '[]'); }
 catch(err){ return []; }
}

function setSavedItems(kind, items){
 const key = kind === 'draft' ? LS_KEYS.drafts : LS_KEYS.templates;
 try{
 localStorage.setItem(key, JSON.stringify(items));
 if(window.SylCycle) SylCycle.cloudSave(key, items);
 return true;
 }catch(err){
 console.warn('Full saved item write failed; trying lightweight save:', err);
 const safeItems = makeStorageSafeItems((items || []).slice(0, 12));
 try{
 localStorage.removeItem(key);
 localStorage.setItem(key, JSON.stringify(safeItems));
 if(window.SylCycle) SylCycle.cloudSave(key, safeItems);
 showStatus('Saved without embedded images because browser storage is full.
Text, tasks and settings were kept.', 'warn');
 return true;
 }catch(err2){
 console.warn('Lightweight saved item write failed:', err2);
 showStatus('Could not save locally because browser storage is full. Export
the worksheet/PDF instead.', 'err');
 return false;
 }
 }
}

function saveCurrent(kind){
 const name = prompt(kind === 'draft' ? 'Draft name:' : 'Template name:');
 if(!name) return;
 const items = getSavedItems(kind);
 items.unshift({

```

```

 id: Date.now() + '_' + Math.random().toString(36).slice(2,7),
 name,
 createdAt: new Date().toISOString(),
 payload: snapshotState()
 });
 const saved = setSavedItems(kind, items.slice(0,20));
 renderSavedLists();
 if(kind === 'template') renderTemplateStarter();
 if(saved) showStatus((kind === 'draft' ? 'Draft' : 'Template') + ' saved.',
'ok');
}

function loadSaved(kind, id){
 const items = getSavedItems(kind);
 const item = items.find(x => x.id === id);
 if(!item) return;
 applySnapshot(item.payload);
 showStatus((kind === 'draft' ? 'Draft' : 'Template') + ' loaded.', 'ok');
}
function deleteSaved(kind, id){
 const items = getSavedItems(kind).filter(x => x.id !== id);
 setSavedItems(kind, items);
 renderSavedLists();
 if(kind === 'template') renderTemplateStarter();
}
function formatDate(iso){
 const d = new Date(iso);
 return d.toLocaleString();
}
function renderSavedLists(){
 const draftList = qs('draftList');
 const templateList = qs('templateList');
 if(!draftList || !templateList) return;
 const drafts = getSavedItems('draft');
 const templates = getSavedItems('template');
 draftList.innerHTML = drafts.length ? drafts.map(item => `
 <div class="saved-item">
 <div>
 ${esc(item.name)}
 ${esc(formatDate(item.createdAt))}
 </div>
 <div class="pill-row">
 <button class="mini" onclick="loadSaved('draft',
'${jsq(item.id)}')">Load</button>
 <button class="mini danger" onclick="deleteSaved('draft',
'${jsq(item.id)}')">Delete</button>
 </div>
 </div>
 `).join('') : '<div class="tiny">No drafts saved yet.</div>';

```

```

templateList.innerHTML = templates.length ? templates.map(item => `
 <div class="saved-item">
 <div>
 ${esc(item.name)}
 ${esc(formatDate(item.createdAt))}
 </div>
 <div class="pill-row">
 <button class="mini" onclick="loadSaved('template',
'${jsq(item.id)}')">Load</button>
 <button class="mini danger" onclick="deleteSaved('template',
'${jsq(item.id)}')">Delete</button>
 </div>
 </div>
`).join('') : '<div class="tiny">No templates saved yet.</div>';
}

```

```

async function waitForImages(root){
 const images = Array.from(root.querySelectorAll('img'));
 await Promise.all(images.map(img => {
 if(img.complete && img.naturalWidth > 0) return Promise.resolve();
 return new Promise(resolve => {
 const done = () => resolve();
 img.addEventListener('load', done, { once:true });
 img.addEventListener('error', done, { once:true });
 });
 }));
}

```

```

function getPrintDocumentHtml(){
 const clone = qs('previewRoot').cloneNode(true);
 clone.querySelectorAll('.preview-page').forEach(page => {
 page.style.width = '794px';
 page.style.height = PAGE_PIXEL_HEIGHT + 'px';
 page.style.minHeight = PAGE_PIXEL_HEIGHT + 'px';
 page.style.margin = '0';
 page.style.boxShadow = 'none';
 page.style.border = 'none';
 page.style.background = '#ffffff';
 page.style.overflow = 'hidden';
 page.style.pageBreakAfter = 'always';
 page.style.breakAfter = 'page';
 });
}

```

```

const style = `
 <style>
 @page { size: A4 portrait; margin: 0; }
 :root { ${buildWorksheetFontCssVarBlock()} }
 * {

```

```

 box-sizing: border-box;
 -webkit-print-color-adjust: exact;
 print-color-adjust: exact;
}
html, body {
 margin: 0;
 padding: 0;
 background: #ffffff;
}
body {
 font-family: 'Quicksand', Arial, sans-serif;
 width: 794px;
 margin: 0 auto;
}
#printRoot {
 width: 794px;
 margin: 0 auto;
}
.preview-page {
 width: 794px;
 height: 1123px;
 min-height: 1123px;
 margin: 0 !important;
 background: #ffffff;
 position: relative;
 padding: 42px 48px 68px;
 overflow: hidden;
 page-break-after: always;
 break-after: page;
}
.preview-page:last-child { page-break-after: auto; break-after: auto; }
.page-header{
 display:flex; justify-content:space-between; align-items:flex-start;
gap:16px;
 margin-bottom:18px;
}
.page-kicker{
 display:inline-block;
 font-size:11px; font-weight:800; letter-spacing:.12em;
text-transform:uppercase;
 color:#2f65c8; margin-bottom:8px;
}
.worksheet-title{
 font-family:'Quicksand', Arial, sans-serif;
 font-size:24px; line-height:1.2; margin:0;
 letter-spacing:-.02em;
 max-width:88%;
 color:#12325f;
}
.page-meta{

```



```

 margin-top:8px; font-size:12px; color:#587094;
 display:flex; gap:12px; flex-wrap:wrap;
}
.ifs-logo{
 width:68px; height:auto; object-fit:contain; flex-shrink:0;
}
.student-line{
 display:flex; gap:20px; font-size:13px; margin-bottom:16px; color:#16335d;
 flex-wrap:wrap;
}
.line-fill{
 display:inline-flex; align-items:flex-end; gap:8px;
}
.line-fill span:last-child{
 display:inline-block; min-width:160px; border-bottom:1.5px solid #7d96bb;
height:18px;
}
.teacher-box{
 border:1.5px solid #d4e1f4;
 background:#f8fbff;
 border-radius:18px;
 padding:14px;
 margin-bottom:18px;
 break-inside:avoid-page;
 page-break-inside:avoid;
}
.teacher-top{
 display:flex; justify-content:space-between; gap:12px; margin-bottom:10px;
flex-wrap:wrap;
}
.teacher-tag{
 display:inline-flex; align-items:center; gap:8px;
 border:1px solid #d4e1f4;
 background:#fff;
 padding:8px 10px;
 border-radius:999px;
 font-size:12px;
 font-weight:700;
 color:#16335d;
}
.lo-table{
 width:100%;
 border-collapse:collapse;
 font-size:12px;
 margin-top:6px;
}
.lo-table th, .lo-table td{
 border-top:1px solid #d4e1f4;
 padding:8px 6px;
 vertical-align:top;

```

```

 text-align:left;
 color:#16335d;
 }
 .lo-table th{
 color:#587094;
 font-size:11px;
 font-weight:800;
 }
 .tick-cell{
 width:70px; text-align:center !important;
 }
 .tick-box{
 width:16px; height:16px; display:inline-flex; align-items:center;
justify-content:center;
 border:1.5px solid #7f9bc5; border-radius:4px; font-size:12px;
color:#12325f;
 background:#fff;
 }
 .tick-cell.active .tick-box{
 background:#eaf2ff; border-color:#2f65c8;
 }
 .instructions{
 background:#f3f8ff;
 border-left:5px solid #2f65c8;
 border-radius:14px;
 padding:14px 15px;
 margin-bottom:16px;
 break-inside:avoid-page;
 page-break-inside:avoid;
 }
 .instructions h3, .section-print h3{
 margin:0 0 7px;
 font-size:14px;
 color:#12325f;
 }
 .instructions p, .section-print p{
 margin:0;
 font-size:13px;
 line-height:1.62;
 white-space:pre-wrap;
 color:#16335d;
 }
 .image-note{
 font-size:12px; color:#12325f; font-weight:700; margin:0 0 10px;
 }
 .worksheet-images{
 display:flex;
 flex-wrap:wrap;
 align-items:flex-start;
 gap:12px;

```

```
 margin-bottom:18px;
}
.worksheet-image-item{
 flex:0 0 100%;
 max-width:100%;
}
.worksheet-images img{
 display:block;
 width:100%;
 height:auto;
 background:#fff;
 border:1px solid #d4e1f4;
 border-radius:16px;
 padding:8px;
}
.task-image-wrap{
 margin:0 0 12px;
 display:flex;
 justify-content:center;
}
.task-image-print{
 display:block;
 width:100%;
 max-width:100%;
 height:auto;
 border:1px solid #d4e1f4;
 border-radius:16px;
 padding:10px;
 background:#fff;
}

.intro-text{
 margin:0 0 16px;
 font-size:13px;
 line-height:1.68;
 color:#27456f;
 white-space:pre-wrap;
}
.task-print{
 border:1px solid #d4e1f4;
 border-radius:18px;
 padding:14px 15px;
 margin-bottom:12px;
 break-inside:avoid-page;
 page-break-inside:avoid;
}
.task-number{
 display:inline-flex; align-items:center; justify-content:center;
 width:28px; height:28px; border-radius:50%;
 background:#eaf2ff; color:#12325f; font-weight:800;
```

```

 margin-right:10px; vertical-align:top;
}
.task-prompt{
 display:inline;
 font-size:14px;
 line-height:1.65;
 white-space:pre-wrap;
 color:#16335d;
}
.rich-render{white-space:pre-wrap; overflow-wrap:anywhere}
.mcq-list{
 margin:10px 0 0 38px; padding-left:18px;
}
.mcq-list li{
 margin-bottom:6px;
 font-size:13px;
 line-height:1.5;
 color:#16335d;
}
.answer-lines{margin-top:12px}
.answer-line{
 border-bottom:1.4px solid #bfd0ee;
 height:22px;
 margin-bottom:7px;
}
.extension-box{
 border:1.5px dashed #9cbbe8;
 background:#fafcff;
 border-radius:18px;
 padding:14px 15px;
 margin-top:10px;
}
.answer-key{
 border-top:2px solid #d4e1f4;
 margin-top:22px;
 padding-top:18px;
}
.answer-item{
 margin-bottom:10px; font-size:13px; line-height:1.58; color:#16335d;
 break-inside:avoid-page;
 page-break-inside:avoid;
}
.answer-item strong{color:#12325f}
.section-print{
 margin-bottom: 14px;
}
.worksheet-lo-list{margin:0;padding-left:20px}
.worksheet-lo-list li{margin:0 0
8px;font-size:var(--ws-font-md);line-height:1.55;color:#1f426c;font-weight:500}
.page-kicker{font-size:var(--ws-font-xs)}

```

```

 .worksheet-title{font-size:var(--ws-title-font);line-height:1.12}
 .page-meta,.student-line,.teacher-tag,.lo-table,.instructions
p,.section-print p,.image-note,.intro-text,.task-prompt,.mcq-list
li,.answer-item,.task-passage,.rich-criteria,.worksheet-extension-text,.worksheet-t
eacher-notes{font-size:var(--ws-font-md)}
 .instructions h3,.section-print h3,.success-box h3,.answer-key
h3,.extension-box h3{font-size:var(--ws-font-lg)}
 .answer-question,.image-note,.teacher-tag{font-size:var(--ws-font-sm)}
 .muted-note,.page-kicker,.answer-stimulus{
 font-size:var(--ws-font-xs);
 color:#587094;
 }
 </style>
`;

```

```

return `<!DOCTYPE html>
<html>
<head>
 <meta charset="utf-8">
 <title>${esc(state.worksheet?.title || 'Worksheet')}</title>
 <link rel="preconnect" href="https://fonts.googleapis.com">
 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
 <link
href="https://fonts.googleapis.com/css2?family=Quicksand:wght@400;500;600;700&displ
ay=swap" rel="stylesheet">
 ${style}
 <style>${extraPrintCss}</style>
</head>
<body>
 <div id="printRoot">${clone.innerHTML}</div>

</body>
</html>`;
}

```

```

async function openPrintWindow(autoOpenDialog = true){
 if(!state.worksheet) return showStatus('Generate or load a worksheet first.',
'warn');
 updatePreview();

```

```

 const existing = document.getElementById('worksheetPrintFrame');
 if(existing) existing.remove();

```

```

 const frame = document.createElement('iframe');
 frame.id = 'worksheetPrintFrame';
 frame.style.position = 'fixed';
 frame.style.right = '0';
 frame.style.bottom = '0';
 frame.style.width = '0';
 frame.style.height = '0';

```

```

frame.style.border = '0';
frame.style.opacity = '0';
frame.style.pointerEvents = 'none';
document.body.appendChild(frame);

const doc = frame.contentDocument || frame.contentWindow.document;
doc.open();
doc.write(getPrintDocumentHtml());
doc.close();

const ready = async () => {
 try{
 if(doc.fonts && doc.fonts.ready) await doc.fonts.ready;
 const imgs = Array.from(doc.images || []);
 await Promise.all(imgs.map(img => img.complete ? Promise.resolve() : new
Promise(resolve => {
 img.addEventListener('load', resolve, { once:true });
 img.addEventListener('error', resolve, { once:true });
 })));
 }catch(_err){}

 if(autoOpenDialog){
 frame.contentWindow.focus();
 frame.contentWindow.print();
 }

 setTimeout(() => {
 if(frame.parentNode) frame.remove();
 }, 2000);
};

if(doc.readyState === 'complete') ready();
else frame.addEventListener('load', ready, { once:true });
}

function buildExportClone(){
 const cloneWrap = document.createElement('div');
 cloneWrap.style.position = 'fixed';
 cloneWrap.style.left = '-100000px';
 cloneWrap.style.top = '0';
 cloneWrap.style.width = '794px';
 cloneWrap.style.height = 'auto';
 cloneWrap.style.padding = '0';
 cloneWrap.style.margin = '0';
 cloneWrap.style.background = '#ffffff';
 cloneWrap.style.zIndex = '-1';
 cloneWrap.style.pointerEvents = 'none';
 cloneWrap.style.overflow = 'hidden';

 const clone = qs('previewRoot').cloneNode(true);

```

```

clone.style.width = '794px';
clone.style.margin = '0';
clone.style.padding = '0';
clone.style.background = '#ffffff';

clone.querySelectorAll('.preview-page').forEach(page => {
 page.style.width = '794px';
 page.style.height = PAGE_PIXEL_HEIGHT + 'px';
 page.style.minHeight = PAGE_PIXEL_HEIGHT + 'px';
 page.style.margin = '0';
 page.style.boxShadow = 'none';
 page.style.border = 'none';
 page.style.background = '#ffffff';
 page.style.overflow = 'hidden';
 page.style.pageBreakAfter = 'auto';
 page.style.breakAfter = 'auto';
});

cloneWrap.appendChild(clone);
document.body.appendChild(cloneWrap);
return cloneWrap;
}

function setOutputMode(mode){
 const select = qs('outputMode');
 if(!select) return;
 if(mode !== 'student' && mode !== 'both' && mode !== 'answer') return;
 if(mode !== 'student' && !hasAnswerKeyData()){
 select.value = 'student';
 savePrefs();
 showStatus('Generate the answer key first, then you can preview or export the
teacher copy.', 'warn');
 updatePreview();
 return;
 }
 select.value = mode;
 savePrefs();
 updatePreview();
}

async function exportPdfMode(mode){
 setOutputMode(mode);
 return exportPdf();
}

function studioExportFilename(){
 const title = String(state.worksheet?.title || '')
 .replace(/[\<>:"/\|]?*\x00-\x1F]/+g, ' ')
 .replace(/\\s+/g, ' ')
 .trim()

```

```

 .replace(/[\.]+$/g, '');
 return `Studio - ${title || 'Worksheet'}.pdf`;
}

async function exportPdf(){
 if(!state.worksheet) return showStatus('Generate or load a worksheet first.',
'warn');
 updatePreview();
 showStatus('Building PDF...', 'warn');

 const filename = studioExportFilename();
 let cloneWrap = null;

 try{
 const html2canvasFn = window.html2canvas;
 const jsPdfCtor = window.jspdf && window.jspdf.jsPDF;

 if(!html2canvasFn || !jsPdfCtor){
 showStatus('PDF libraries unavailable here. Opening print view instead.',
'warn');
 return openPrintWindow(true);
 }

 if(document.fonts && document.fonts.ready) await document.fonts.ready;

 cloneWrap = buildExportClone();
 await waitForImages(cloneWrap);

 const exportPages = Array.from(cloneWrap.querySelectorAll('.preview-page'));
 if(!exportPages.length) throw new Error('No printable pages found.');
```

```

 const pdf = new jsPdfCtor({ unit:'mm', format:'a4', orientation:'portrait',
compress:true });

 for(let i = 0; i < exportPages.length; i++){
 const page = exportPages[i];
 const canvas = await html2canvasFn(page, {
 scale: 2,
 useCORS: true,
 allowTaint: true,
 backgroundColor: '#ffffff',
 width: 794,
 height: PAGE_PIXEL_HEIGHT,
 windowWidth: 794,
 windowHeight: PAGE_PIXEL_HEIGHT,
 scrollX: 0,
 scrollY: 0,
 removeContainer: true
 });

```



```

 if(i > 0) pdf.addPage();
 pdf.addImage(canvas.toDataURL('image/jpeg', 0.98), 'JPEG', 0, 0, 210, 297,
undefined, 'FAST');
 }

 pdf.save(filename);
 showStatus('PDF exported.', 'ok');
} catch(err){
 console.error(err);
 showStatus('PDF export failed. Opening print view instead.', 'err');
 openPrintWindow(true);
} finally{
 if(cloneWrap) cloneWrap.remove();
}
}

```

```

function wizardActionButtons(step){
 const left = step === 1
 ? `<span class="wizard-meta">Step 1 of 3</span>`
 : `<button class="wizard-btn ghost" onclick="goWizardStep(${step-1})">←
Back</button>`;
 const nextLabel = step === 2 ? 'Next: Editor and preview →' : `Next:
${WIZARD_STEPS[step + 1].title} →`;
 const right = step === 3
 ? ''
 : `<button class="wizard-btn primary"
onclick="goWizardStep(${step+1})">${nextLabel}</button>`;
 return `<div class="wizard-actions"><div>${left}</div><div
class="wizard-actions-right">${right}</div></div>`;
}

```

```

const WIZARD_STEPS = {
 1: { title:'Curriculum and worksheet brief', subtitle:'' },
 2: { title:'Start and generate', subtitle:'' },
 3: { title:'Editor and preview', subtitle:'' }
};

```

```

function goWizardStep(step){
 const target = Math.max(1, Math.min(3, Number(step) || 1));
 state.currentWizardStep = target;
 qsa('.wizard-step').forEach(el => el.classList.toggle('active',
Number(el.dataset.step) === target));
 qsa('.wizard-tab').forEach(el => el.classList.toggle('active',
Number(el.dataset.step) === target));
 window.scrollTo({ top: 0, behavior: 'smooth' });
}

```

```

function setupGenerateStepForPdfAdapt(assetsSection, generateSection){
 if(!generateSection || generateSection.dataset.pdfAdaptReady === '1') return;

```

```

generateSection.dataset.pdfAdaptReady = '1';

const starterRow = generateSection.querySelector('.generate-starter-row') ||
generateSection.querySelector('.btn-row');
if(starterRow) starterRow.style.gridTemplateColumns = 'repeat(3,1fr)';

const controlsHost = generateSection.querySelector('#adaptPdfControls');
if(assetsSection && controlsHost){
 const pdfField =
assetsSection.querySelector('label[for="templatePdf"]')?.closest('.field');
 const textField =
assetsSection.querySelector('label[for="templateText"]')?.closest('.field');
 if(pdfField && !controlsHost.contains(pdfField))
controlsHost.appendChild(pdfField);
 if(textField && !controlsHost.contains(textField)){
 const label = textField.querySelector('label[for="templateText"]');
 if(label) label.textContent = 'Adaptation notes or pasted text';
 const textarea = textField.querySelector('#templateText');
 if(textarea) textarea.placeholder = 'Optional notes for how the AI should
adapt the PDF into a worksheet.';
 controlsHost.appendChild(textField);
 }
}

function studioHeaderIcon(name){
 const icons = {
 suite: '',
 library: '<svg viewBox="0 0 24 24" aria-hidden="true" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round"
stroke-linejoin="round"><path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20"/><path d="M6.5
2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z"/></svg>',
 settings: '<svg viewBox="0 0 24 24" aria-hidden="true" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round"
stroke-linejoin="round"><path d="M12 15.5A3.5 3.5 0 1 0 12 8a3.5 3.5 0 0 0
7.5z"/><path d="M19.4 15a1.7 1.7 0 0 0 .34 1.87l-.06.06a2 2 0 1 1-2.83
2.83l-.06-.06A1.7 1.7 0 0 0 15 19.4a1.7 1.7 0 0 0-1 .6 1.7 1.7 0 0 0-.4 1.1V21a2 2
0 1 1-4 0v-.09A1.7 1.7 0 0 0 8.6 19.4a1.7 1.7 0 0 0-1.87.34l-.06.06a2 2 0 1
1-2.83-2.83l-.06-.06A1.7 1.7 0 0 0 4.6 15a1.7 1.7 0 0 0-.6-1 1.7 1.7 0 0
0-1.1-.4H3a2 2 0 1 1 0-4h.09A1.7 1.7 0 0 0 4.6 8.6a1.7 1.7 0 0
0-.34-1.87l-.06-.06a2 2 0 1 1 2.83-2.83l.06.06A1.7 1.7 0 0 0 9 4.6a1.7 1.7 0 0 0
1-.6 1.7 1.7 0 0 0 .4-1.1V3a2 2 0 1 1 4 0v.09A1.7 1.7 0 0 0 15.4 4.6a1.7 1.7 0 0 0
1.87-.34l.06-.06a2 2 0 1 1 2.83 2.83l-.06.06A1.7 1.7 0 0 0 19.4 9c.27.4.7.68
1.2.78.18.04.37.06.56.06H21a2 2 0 1 1 0 4h-.09A1.7 1.7 0 0 0 19.4 15z"/></svg>'
 };
 return icons[name] || '';
}

function ensureSettingsModal(settingsSection, topbarRight){

```

```

if(!settingsSection) return;
let modal = document.getElementById('settingsModal');
if(!modal){
 modal = document.createElement('div');
 modal.id = 'settingsModal';
 modal.className = 'settings-modal hidden';
 modal.innerHTML = `
 <div class="settings-modal-card">
 <div class="settings-modal-head">
 <div>
 <h3>Settings</h3>
 <p>Your keys stay local in this browser. This no longer needs its own
wizard step.</p>
 </div>
 <button type="button" class="settings-modal-close"
onclick="closeSettingsModal()">Close</button>
 </div>
 <div class="settings-modal-body" id="settingsModalBody"></div>
 </div>
 `;
 modal.addEventListener('click', (event)=>{ if(event.target === modal)
closeSettingsModal(); });
 document.body.appendChild(modal);
}

const body = document.getElementById('settingsModalBody');
if(body && !body.contains(settingsSection)){
 settingsSection.style.margin = '0';
 body.appendChild(settingsSection);
}

if(topbarRight && !document.getElementById('settingsLauncher')){
 const button = document.createElement('button');
 button.type = 'button';
 button.id = 'settingsLauncher';
 button.className = 'settings-launcher';
 button.innerHTML = `${studioHeaderIcon('settings')}<span>Settings</span>`;
 button.onclick = ()=>openSettingsModal();
 topbarRight.appendChild(button);
}

function ensureHomeLauncher(topbarRight){
 if(!topbarRight || document.getElementById('homeLauncher')) return;
 const link = document.createElement('a');
 link.id = 'homeLauncher';
 link.className = 'settings-launcher';
 link.href = 'index.html';
 link.innerHTML = `${studioHeaderIcon('suite')}<span>Cycle Suite</span>`;
 topbarRight.appendChild(link);
}

```

```

}

function ensureLibraryModal(topbarRight){
 let modal = document.getElementById('libraryModal');
 if(!modal){
 modal = document.createElement('div');
 modal.id = 'libraryModal';
 modal.className = 'settings-modal hidden';
 modal.innerHTML = `
 <div class="settings-modal-card">
 <div class="settings-modal-head">
 <div>
 <h3>Library</h3>
 <p>Saved worksheet snapshots. History keeps text and settings without
heavy embedded images or PDF pages.</p>
 </div>
 <button type="button" class="settings-modal-close"
onclick="closeLibraryModal()">Close</button>
 </div>
 <div class="settings-modal-body" id="libraryModalBody"></div>
 </div>
 `;
 modal.addEventListener('click', (event)=>{ if(event.target === modal)
closeLibraryModal(); });
 document.body.appendChild(modal);
 }

 if(topbarRight && !document.getElementById('libraryLauncher')){
 const button = document.createElement('button');
 button.type = 'button';
 button.id = 'libraryLauncher';
 button.className = 'settings-launcher';
 button.innerHTML = `${studioHeaderIcon('library')}<span>Library</span>`;
 button.onclick = ()=>openLibraryModal();
 topbarRight.appendChild(button);
 }

 refreshLibraryModal();
}

function refreshLibraryModal(){
 const body = document.getElementById('libraryModalBody');
 if(body) body.innerHTML = renderHistoryPanel();
}

function openLibraryModal(){
 refreshLibraryModal();
 document.getElementById('libraryModal')?.classList.remove('hidden');
}

```

```

function closeLibraryModal(){
 document.getElementById('libraryModal')?.classList.add('hidden');
}

function openSettingsModal(){
 document.getElementById('settingsModal')?.classList.remove('hidden');
}

function closeSettingsModal(){
 document.getElementById('settingsModal')?.classList.add('hidden');
}

function initWizardLayout(){
 const layout = document.querySelector('.layout');
 if(!layout || document.querySelector('.wizard-root')) return;

 const leftPanel = layout.querySelector('.panel');
 const workspace = layout.querySelector('.workspace');
 if(!leftPanel || !workspace) return;

 const sections = Array.from(leftPanel.querySelectorAll('.section'));
 const settingsSection = sections[0];
 const curriculumSection = sections[1];
 const assetsSection = sections[2];
 const generateSection = sections[3];
 const stepSections = [curriculumSection, generateSection].filter(Boolean);
 const workspaceTop = workspace.querySelector('.workspace-top');
 const editorPreview = workspace.querySelector('.editor-preview');
 const topbarRight = document.querySelector('.topbar-right');

 if(generateSection) setupGenerateStepForPdfAdapt(assetsSection, generateSection);
 ensureHomeLauncher(topbarRight);
 ensureLibraryModal(topbarRight);
 ensureSettingsModal(settingsSection, topbarRight);

 if(topbarRight){
 const chips = Array.from(topbarRight.querySelectorAll('.chip'));
 if(chips[0]) chips[0].textContent = 'Built by Syl';
 if(chips[1]) chips[1].textContent = 'Built By Syl';
 if(chips[2]) chips[2].textContent = 'Three-step workflow';
 }

 const wizardRoot = document.createElement('div');
 wizardRoot.className = 'wizard-root';

 const progress = document.createElement('div');
 progress.className = 'wizard-progress';
 progress.innerHTML = [1,2,3].map(step => `
 <button type="button" class="wizard-tab ${step===1?'active':''}"
data-step="${step}" onclick="goWizardStep(${step})">

```

```

 ${step}

 ${WIZARD_STEPS[step].title}
 ${WIZARD_STEPS[step].subtitle}

 </button>
`).join('');

const card = document.createElement('div');
card.className = 'wizard-card';

const stepWrap = document.createElement('div');
stepWrap.className = 'wizard-step-wrap';

stepSections.forEach((section, idx) => {
 const step = idx + 1;
 const screen = document.createElement('section');
 screen.className = 'wizard-step' + (step === 1 ? ' active' : '');
 screen.dataset.step = String(step);
 screen.innerHTML = `
 <div class="wizard-heading">
 <div>
 <div class="wizard-kicker">Step ${step}</div>
 <h2>${WIZARD_STEPS[step].title}</h2>
 <p>${WIZARD_STEPS[step].subtitle}</p>
 </div>
 <div class="wizard-meta">${step === 2 ? 'Generate before editing' : 'You
can move between steps at any time'}</div>
 </div>
 `;
 const stack = document.createElement('div');
 stack.className = 'wizard-panel-stack';
 stack.appendChild(section);
 screen.appendChild(stack);
 screen.insertAdjacentHTML('beforeend', wizardActionButtons(step));
 stepWrap.appendChild(screen);
});

const editorScreen = document.createElement('section');
editorScreen.className = 'wizard-step';
editorScreen.dataset.step = '3';
editorScreen.innerHTML = `
 <div class="step5-topbar">
 <div>
 <div class="wizard-kicker">Step 3</div>
 <h2 style="margin:0 0 8px; font-size:34px; line-height:1.05;
letter-spacing:-.04em; color:#12325f">${WIZARD_STEPS[3].title}</h2>
 <p style="margin:0; max-width:760px; font-size:15px; color:#587094;
line-height:1.65">${WIZARD_STEPS[3].subtitle}</p>
 </div>
 </div>

```

```

 <div class="step5-side">
 <button type="button" class="step5-back" onclick="goWizardStep(2)">← Back
to generate</button>
 <button type="button" class="step5-back"
onclick="checkWorksheetBeforeExport()">Check worksheet</button>
 <button type="button" class="step5-back" onclick="startNewResource()">↺
Reset worksheet</button>
 <div class="wizard-step5-note">Export tools live in this screen now.</div>
 </div>
 </div>
`;
 if(workspaceTop) editorScreen.appendChild(workspaceTop);
 if(editorPreview) editorScreen.appendChild(editorPreview);
 stepWrap.appendChild(editorScreen);

 card.appendChild(stepWrap);
 wizardRoot.appendChild(progress);
 wizardRoot.appendChild(card);
 layout.replaceWith(wizardRoot);
}

```

```

/* Initial boot moved to the final runtime layer to avoid duplicate listeners and
double initialisation. */

```

```

// =====
// — Editor, preview, normalisation, images, history, wizard —
// =====

```

```

const DEFAULT_IMAGE_USAGE = 'build_questions';
const DEFAULT_IMAGE_GEN_SIZE = '1024x1024';
const DEFAULT_WORKSHEET_FONT_SIZE = 12;
const ALLOWED_WORKSHEET_FONT_SIZES = [9,10,11,12,13,14,15,16];
const IFS_LOGO_SRC =
'
EsQBqvQXeP/dvNtrb9w8YH6Ad+Md3qk/b/dv73v0A9xXmAfpf0ifMB+sH7H+7v/gPUN+xnsAfsP1kXoD/tF
6Xn7sfCT+3X7Ze0f/7tZz8V/0TtL/tf4y9A57K5xH69fgvyv83Lw594Wgj/Lf8T+Rn7d/7vjxcS8wL2b+xf
478t+ZHvM/z//U8f3QA/M/+09Jr/Y/uvnf+dP+T/ePgJ/lX9c/3vrRey70dv2kM4vUTeDi8qVhBdbDKE3yt
N/wjZlUc6Vf+K2U3VMRwkmTcYh+Azlrv5IkaPalfx40tJyTYCDw66RtqzUD00JBPKRJ2YIXxkf9ClhWrsT0
lDzhmVP6/I8bqL0+I2dovF9FEk/OxjoYLyvU56Syu0Y0DbgzYqQ3wMGMVXx8kk+VBLpxspvtRHHVigB+Icmk
AsraiTwAD+/rQkUHvcLZdwPm+ONVWgcdSzz76twPqrub0khaS9yP0+M8RdHebS75kLePucwfwVbHfbrX2Sy
mwi0984tHUBxynr2QZ2/s/y5HSe0Dc/Q01Yz8kKN88zod1XSyL2Czwt4kRk0Q+3d37i6b+eUppg1LbN0u/8
kqHf9rgxOL54+g+2Z0uV+C04cTeF5MLNeFgqWoGu1FkwHJTpDFv6T3/xIwaWh4SgZp5k1BoQAP2D2Jbh8yH
qS4FZxcYcxrdpxpXDhrG77ajY1oh02z//ksogNBjhpJ2qJWLoKrRK0fpijz0/hVlnJQgq+C6M60MJgZ/4V
o+Z7ynJgoIN7g9X+U5539n/MQ/rF62gr+VWeqpEJhGB3AjjnR/9anNEGUUTXwyYxH2zChFM6nopcdMAwZqw
A4UMZ4LJ7gD/ZeYUAha6DTT7Lb9CsP1L0rO+CtPiBlzXWmpenuQs+NMGhyhHCaw5RG57tj+12TbtUvmq2Wu
uyejzFatXnc11xhiPaTPCJ/2SDTZ/6c48v0aji5Hv19zx1cy00EPovhMUtYu7541azuBYpmbpRF6wK126jV
L3tLoZp04ou6cA+u/805Sv9mpXx2Pq4SQQ4ckuzeSQrnoN++HCjHfDek8oiq51InFQstc/nRF8z8bLfk1jV
h8uV3ub8e0JmYXKUsTgtYlXIfmd9hFuMDjJZV5uff40Yq49ns6GL1IeBJxcXNp0P1UM7J6n2XSScb1090qB

```

naGuA45roKREYqcNXPeP6fb2m91zVuHJooRW2NedsIQQgn/WvtJ03z+KMOFflyG1P6ibR0T4xfnHhaN8TckG  
10vuRWEr1f8h4W8Ye1mLzpx1lEv53iv6HRU1eD9a3A3oe4oS5MSA1MoHUP9Bnf/3reFXKv59lX1Z/ioTncP  
4v34Ewoyc+/P4hlX7RXukS8wL3tSNXHrrTqZwr8Z0oNe6Kt1Uk5X/Vk9BE9SvQRR81by+EK5IMmtYk5w1Zl  
5tGHe9vZ5+UNB66wh8e1654B8YGH8RfH8CuQubBPCfzRvj0K7fuGUJ3JXrf/zTfg5EyvBH1B0dXPnNulv6p  
iKDyVG/oxUcZC14P4VmT4lexITl0fgBvDOCRFG1rDqHFpnPTfAdTN96v6xuaZxjMK/HVBbbSVnz1NOVFNOP  
heqLGjLp8rF0gCnHzfLiVNPwngQH0/Pj2TB8dZR5B5U2FY4uyH0Ha9YCwUSm7iskcnqSQ07ucIVieuy4p/g  
7HHfmE64ecSpn8XcbYy630bJOgm6FAguX2Q0s1LEJELNzkP/xIdAOYT9hfMcJWsuLj25WPSd01NM/egzu6L  
5zP0M38D14faWagGJZrrsKAfaoj2P5faczWGWtjUZZbvW12HurZOGtdBAMUiaLkCz/JWCM1qbD20fNXgqm  
Z/bf1MVMoukWvytyVb4fiEb8rdcITCgc5ZQ90lkiueoTNFqbMdrZQ00QyMwoQXEVz8QoCE9hzZ2t+MFJzxo  
A5/E/fIT7xsyq946b8I7tGDd31xhsI26ReIbyXF9i1AxoWiDjNqxrneCwj6pPrRpeeSljATDI5FHDP605p0  
66yxnxptScS9LY7YYqG1C4KJ4UVICvYj62xmQBUhGupyW1AAycP2WvwJD6/BBwruT6FZuu7G/a4ldp2hHYH  
Zls//4xqFQUwfYhIVDNgzZSjdE0lGeSjzBAjMY0nUEFzienBD/3VQmXRW/mQYgAhKT2ybW+VTbQCrn67ehr  
yGUEoB0wZoDEvAuflbQrdH3GExl0qnI6oQzu0bv7mqWRNaA6DJYhwLd5/aLyWBtu/OXl7y7PJEUzAE19Kjb  
laM/Qa6IvjQKLQKcpqQZwI+C+GpNGUQl7JWgjFvijp0FpW93GbYi/m4+JX3K/4ZRiPev8IC4s+Pa5EFnXpD  
80fim0voZhGz/Tf4xH7Y8CE1d+S3CrT0a0wMMwT75vcBYagSCW4H/DBF7TRaDAKaLxjersfRB5vsXkbnJ/t  
5BRLXsaG96pm1ohDqLTdCt1dskv1ZEoAq90LRCVfOCryV7aEO+GB9tzocWVPwN72JoRmUf4oE1J7GA6h1T  
SsyG7hPrT13mGN8PT9kwyfE/PSH1at4tr9v4kM2dp+SRKTeVKP81/WrjyLXI8cb74ijhKA0x8uNKCxeRez  
gbLl//v/B8Zj7S+HF+bXReiQ08Kq6eKsLbaQAKA2vWKXGF5Nr14x4AAAAAAAAA==';

```
state.imageUsage = state.imageUsage || DEFAULT_IMAGE_USAGE;
state.imageJobs = state.imageJobs || {};
if(window.pdfjsLib && window.pdfjsLib.GlobalWorkerOptions){
 window.pdfjsLib.GlobalWorkerOptions.workerSrc =
 'https://cdnjs.cloudflare.com/ajax/libs/pdf.js/3.11.174/pdf.worker.min.js';
}
state.taskPdfImport = state.taskPdfImport || null;
state.taskPdfCropPanels = state.taskPdfCropPanels || {};

const EDITOR_ACCORDION_LS_KEY = 'cyclestudio_editor_accordions_v2';
function loadEditorAccordionState(){
 try{ return JSON.parse(localStorage.getItem(EDITOR_ACCORDION_LS_KEY) || '{}') ||
 {};}
 catch(err){ return {};}
}
function saveEditorAccordionState(){
 try{ localStorage.setItem(EDITOR_ACCORDION_LS_KEY,
 JSON.stringify(state.editorAccordions || {})); }
 catch(err){}
}
function accordionDomId(key='') {
 return 'editorAccordion_' + String(key || '').replace(/^[^a-z0-9_-]+/gi, '_');
}
state.editorAccordions = state.editorAccordions || loadEditorAccordionState();
function isEditorAccordionOpen(key, defaultOpen=false){
 return Object.prototype.hasOwnProperty.call(state.editorAccordions || {}, key) ?
 !!state.editorAccordions[key] : !!defaultOpen;
}
function setEditorAccordionOpen(key, isOpen){
 state.editorAccordions = state.editorAccordions || {};
 state.editorAccordions[key] = !!isOpen;
```



```

 saveEditorAccordionState();
 const root = document.getElementById(accordionDomId(key));
 if(root) applyAccordionDomState(root, !!isOpen);
}
function applyAccordionDomState(root, isOpen){
 root.classList.toggle('open', !!isOpen);
 root.setAttribute('data-accordion-open', isOpen ? 'true' : 'false');
 const body = root.querySelector('.editor-accordion-body');
 if(body) body.classList.toggle('hidden', !isOpen);
 const head = root.querySelector('.editor-accordion-head');
 if(head) head.setAttribute('aria-expanded', isOpen ? 'true' : 'false');
 const toggle = root.querySelector('.editor-accordion-toggle');
 if(toggle){
 toggle.setAttribute('aria-expanded', isOpen ? 'true' : 'false');
 toggle.textContent = isOpen ? '▼' : '►';
 toggle.setAttribute('title', isOpen ? 'Collapse section' : 'Expand section');
 }
}
function accordionPlainText(value=''){
 return cleanText(String(value || '').replace(/<br\s*\/*>(?!=)/gi,
'\n').replace(/<[^\>]+>/g, ' ')).replace(/\\s+/g, ' ').trim();
}
function accordionSnippet(value='', fallback=''){
 const plain = accordionPlainText(value);
 if(!plain) return fallback || 'Collapsed to save your scrolling finger.';
 return plain.length > 96 ? plain.slice(0, 93).trim() + '...' : plain;
}
function editorAccordionHtml(config={}){
 const key = String(config.key || 'accordion');
 const open = isEditorAccordionOpen(key, !!config.defaultOpen);
 const actions = config.actions ? `<div class="editor-accordion-actions"
onclick="event.stopPropagation()">${config.actions}</div>` : '';
 const subtitle = config.showSubtitle && config.subtitle ? `<div
class="editor-accordion-sub">${config.subtitle}</div>` : '';
 return `
 <div class="editor-accordion task-card ${open ? 'open' : ''}
${config.extraClass || ''}" id="${accordionDomId(key)}"
data-accordion-key="${esc(key)}">
 <div class="editor-accordion-head" role="button" tabindex="0"
aria-expanded="${open ? 'true' : 'false'}"
onclick="toggleEditorAccordion('${jsq(key)}')" onkeydown="if(event.key === 'Enter'
|| event.key === ' '){ event.preventDefault();
toggleEditorAccordion('${jsq(key)}'); }">
 <div class="editor-accordion-title-wrap">
 <button type="button" class="editor-accordion-toggle"
aria-expanded="${open ? 'true' : 'false'}" onclick="event.stopPropagation();
toggleEditorAccordion('${jsq(key)}')">${open ? '▼' : '►'}</button>
 <div>
 ${config.title ||
'Section'}

```

```

 ${subtitle}
 </div>
</div>
 ${actions}
</div>
 <div class="editor-accordion-body ${open ? '' : 'hidden'}">${config.body ||
''}</div>
</div>`;
}
window.toggleEditorAccordion = function(key, force){
 const next = typeof force === 'boolean' ? force : !isEditorAccordionOpen(key,
false);
 setEditorAccordionOpen(key, next);
};
window.setAllTaskAccordions = function(isOpen){
 if(!Array.isArray(state.worksheet?.tasks)) return;
 state.worksheet.tasks.forEach((task, idx) =>
setEditorAccordionOpen(`task_${idx}`, !!isOpen));
};

function cleanText(val='') {
 if(Array.isArray(val)) return val.filter(Boolean).join('\n');
 return String(val || '').trim();
}

function hasHtmlMarkup(val='') {
 return /<\/?[a-z][\s\S]*>/i.test(String(val || ''));
}

function trimRichBreaks(html='') {
 let value = String(html || '')
 .replace(/<br\s*\/?>(?:\s*<br\s*\/?>)+/gi, match =>
match.replace(/<br\s*\/?>/gi, '
'))
 .replace(/ /gi, ' ');
 const plain = value
 .replace(/<br\s*\/?>(?:\s*)/gi, '')
 .replace(/<\/?(?:strong|em|u)>/gi, '')
 .replace(/ /gi, ' ')
 .trim();
 if(!plain) return '';
 value = value.replace(/^(?:\s*
\s*)+/i, '');
 value = value.replace(/\s+(
)/gi, '$1');
 return value;
}

function plainTextToRichHtml(val='') {
 return esc(decodeXmlEntities(String(val || ''))).replace(/\n/g, '
');
}

function sanitiseRichTextHtml(value='') {
 const raw = String(value || '');
 if(!raw.trim()) return '';
 if(!hasHtmlMarkup(raw)) return trimRichBreaks(plainTextToRichHtml(raw));
 const root = document.createElement('div');

```

```

root.innerHTML = raw;
const walk = (node) => {
 if(node.nodeType === Node.TEXT_NODE) return esc(node.textContent || '');
 if(node.nodeType !== Node.ELEMENT_NODE) return '';
 const tag = node.tagName.toLowerCase();
 if(tag === 'br') return '
';

 let inner = Array.from(node.childNodes).map(walk).join('');

 if(!inner &&
!['div','p','li','ul','ol','section','article','header','footer'].includes(tag))
return '';

 const style = (node.getAttribute('style') || '').toLowerCase();
 const computedTextDecor = String(node.style?.textDecoration ||
node.style?.textDecorationLine || '').toLowerCase();
 const isBold = ['b','strong'].includes(tag)
 || /font-weight\s*:\s*(bold|[6-9]00)/i.test(style)
 || (node.style?.fontWeight && (node.style.fontWeight === 'bold' ||
parseInt(node.style.fontWeight, 10) >= 600));
 const isItalic = ['i','em'].includes(tag)
 || /font-style\s*:\s*italic/i.test(style)
 || String(node.style?.fontStyle || '').toLowerCase() === 'italic';
 const isUnderline = tag === 'u'
 || /text-decoration[^\;]*underline/i.test(style)
 || computedTextDecor.includes('underline');

 if(isBold) inner = `<strong>${inner}</strong>`;
 if(isItalic) inner = `<em>${inner}</em>`;
 if(isUnderline) inner = `<u>${inner}</u>`;

 if(['div','p','li','ul','ol','section','article','header','footer'].includes(tag))
return inner ? `${inner}<br>` : '
';
 return inner;
};
return trimRichBreaks(Array.from(root.childNodes).map(walk).join(''));
}
function renderRichText(value='') {
 return sanitiseRichTextHtml(value);
}
function richEditorContent(value='') {
 return renderRichText(value);
}
function richEditorHtml(config={}){
 const id = config.id || `rich_${Math.random().toString(36).slice(2,8)}`;
 const placeholder = config.placeholder || '';
 const note = config.note || '';
 const minHeight = Number(config.minHeight) || 96;
 const toolbar = config.toolbar !== false;

```

```

const dataset = config.dataset || {};
const dataAttrs = Object.entries(dataset).map(([key, val]) => {
 const attrKey = String(key).replace(/([a-z0-9])([A-Z])/g,
'$1-$2').toLowerCase();
 return `data-${esc(attrKey)}="${esc(String(val))}"`;
}).join(' ');
return `
 <div class="rich-editor-wrap">
 ${toolbar ? `<div class="rich-toolbar">
 <button class="rich-toolbar-btn" type="button" title="Bold (Ctrl/Cmd+B)"
onmousedown="event.preventDefault();
applyRichCommand('${jsq(id)}', 'bold')">B</button>
 <button class="rich-toolbar-btn" type="button" title="Italic (Ctrl/Cmd+I)"
onmousedown="event.preventDefault();
applyRichCommand('${jsq(id)}', 'italic')">I</button>
 <button class="rich-toolbar-btn" type="button" title="Underline
(Ctrl/Cmd+U)" onmousedown="event.preventDefault();
applyRichCommand('${jsq(id)}', 'underline')"><u>U</u></button>
 </div>` : ''}
 <div id="${esc(id)}" class="rich-editor" contenteditable="true"
spellcheck="true" data-placeholder="${esc(placeholder)}"
style="min-height:${minHeight}px" ${dataAttrs}
oninput="handleRichEditorInput(this)"
onkeydown="handleRichEditorKeydown(event, this)"
onpaste="handleRichEditorPaste(event, this)">${richEditorContent(config.value ||
'')}</div>
 ${note ? `<div class="field-note">${esc(note)}</div>` : ''}
 </div>`;
}
function updateRichStateFromEditor(el){
 if(!el || !state.worksheet) return;
 const value = sanitiseRichTextHtml(el.innerHTML || '');
 const target = el.dataset.richTarget || el.dataset.richtarget || '';
 const field = el.dataset.field || '';
 const idx = parseInt(el.dataset.idx || '-1', 10);
 if(target === 'worksheetField' && field){
 state.worksheet[field] = value;
 }else if(target === 'worksheetSection' && field){
 state.worksheet[field] = value;
 }else if(target === 'taskField' && field && state.worksheet.tasks?.[idx]){
 state.worksheet.tasks[idx][field] = value;
 }
 queuePreviewUpdate();
}
window.handleRichEditorInput = function(el){
 updateRichStateFromEditor(el);
};
function insertRichHtmlAtCaret(html='') {
 const sel = window.getSelection() && window.getSelection();
 if(!sel || !sel.rangeCount) return false;

```

```

const range = sel.getRangeAt(0);
range.deleteContents();
const temp = document.createElement('div');
temp.innerHTML = html;
const frag = document.createDocumentFragment();
let node = null;
let lastNode = null;
while((node = temp.firstChild)){
 lastNode = frag.appendChild(node);
}
range.insertNode(frag);
if(lastNode){
 const nextRange = document.createRange();
 nextRange.setStartAfter(lastNode);
 nextRange.collapse(true);
 sel.removeAllRanges();
 sel.addRange(nextRange);
}
return true;
}
window.insertRichLineBreak = function(el){
 if(!el) return;
 el.focus();
 let inserted = false;
 try{ inserted = document.execCommand('insertLineBreak', false, null);
}catch(err){}
 if(!inserted){
 try{ inserted = document.execCommand('insertHTML', false, '
');
}catch(err){}
 }
 if(!inserted) inserted = insertRichHtmlAtCaret('
');
 updateRichStateFromEditor(el);
};
window.handleRichEditorKeydown = function(event, el){
 if((event.ctrlKey || event.metaKey) && !event.shiftKey){
 const key = String(event.key || '').toLowerCase();
 const map = { b:'bold', i:'italic', u:'underline' };
 if(map[key]){
 event.preventDefault();
 document.execCommand(map[key], false, null);
 updateRichStateFromEditor(el);
 return;
 }
 }
 if(String(event.key || '') === 'Enter' && !event.shiftKey){
 event.preventDefault();
 window.insertRichLineBreak(el);
 }
};
window.handleRichEditorPaste = function(event, el){

```

```

 if(!event) return;
 event.preventDefault();
 const plain = event.clipboardData?.getData('text/plain') || '';
 const html = plainTextToRichHtml(plain);
 if(document.queryCommandSupported '&&
document.queryCommandSupported('insertHTML')){
 document.execCommand('insertHTML', false, html);
 }else if(document.queryCommandSupported '&&
document.queryCommandSupported('insertText')){
 document.execCommand('insertText', false, plain);
 }
 updateRichStateFromEditor(e1);
};
window.applyRichCommand = function(editorId, command){
 const editor = document.getElementById(editorId);
 if(!editor) return;
 editor.focus();
 document.execCommand(command, false, null);
 updateRichStateFromEditor(editor);
};
function parseAnswerLines(value){
 return Math.max(0, Math.min(40, parseInt(value,10) || 0));
}
function parseImageWidth(value){
 return Math.max(30, Math.min(100, parseInt(value,10) || 100));
}
function parseImageGenSize(value){
 return ['1024x1024','1024x1536','1536x1024'].includes(value) ? value :
(state.imageGenSize || DEFAULT_IMAGE_GEN_SIZE);
}
function imageSizeLabel(value){
 const map = { '1024x1024':'Square', '1024x1536':'Portrait',
'1536x1024':'Landscape' };
 return map[parseImageGenSize(value)] || 'Square';
}
function normaliseTaskImage(image){
 if(!image || !image.dataUrl) return null;
 return {
 source: image.source || 'upload',
 provider: image.provider || '',
 prompt: cleanText(image.prompt || ''),
 mimeType: image.mimeType || 'image/png',
 dataUrl: image.dataUrl,
 name: cleanText(image.name || 'task-image.png') || 'task-image.png'
 };
}
function parseCropPercent(value){
 const num = parseInt(value, 10);
 if(!Number.isFinite(num)) return 0;
 return Math.max(0, Math.min(80, num));
}

```

```

}
function enforceImageCropBounds(crop={}, changedSide='') {
 const next = {
 left: parseCropPercent(crop?.left),
 right: parseCropPercent(crop?.right),
 top: parseCropPercent(crop?.top),
 bottom: parseCropPercent(crop?.bottom)
 };
 const maxPair = 90;
 if(next.left + next.right > maxPair){
 if(changedSide === 'left') next.left = Math.max(0, maxPair - next.right);
 else if(changedSide === 'right') next.right = Math.max(0, maxPair - next.left);
 else {
 const scale = maxPair / (next.left + next.right);
 next.left = Math.round(next.left * scale);
 next.right = Math.round(next.right * scale);
 }
 }
 if(next.top + next.bottom > maxPair){
 if(changedSide === 'top') next.top = Math.max(0, maxPair - next.bottom);
 else if(changedSide === 'bottom') next.bottom = Math.max(0, maxPair -
next.top);
 else {
 const scale = maxPair / (next.top + next.bottom);
 next.top = Math.round(next.top * scale);
 next.bottom = Math.round(next.bottom * scale);
 }
 }
 return next;
}
function cropSummaryText(crop={}){
 const safe = enforceImageCropBounds(crop);
 if(!safe.left && !safe.right && !safe.top && !safe.bottom) return '';
 return `Crop: L${safe.left}% · R${safe.right}% · T${safe.top}% ·
B${safe.bottom}%`;
}
function loadImageFromDataUrl(dataUrl){
 return new Promise((resolve, reject) => {
 const img = new Image();
 img.onload = () => resolve(img);
 img.onerror = () => reject(new Error('Could not read the imported page
image.'));
 img.src = dataUrl;
 });
}
async function cropImageDataUrl(dataUrl, crop={}){
 if(!dataUrl) return '';
 const safe = enforceImageCropBounds(crop);
 if(!safe.left && !safe.right && !safe.top && !safe.bottom) return dataUrl;
 const img = await loadImageFromDataUrl(dataUrl);

```

```

const naturalWidth = img.naturalWidth || img.width || 1;
const naturalHeight = img.naturalHeight || img.height || 1;
const cropLeft = Math.round(naturalWidth * (safe.left / 100));
const cropRight = Math.round(naturalWidth * (safe.right / 100));
const cropTop = Math.round(naturalHeight * (safe.top / 100));
const cropBottom = Math.round(naturalHeight * (safe.bottom / 100));
const sx = Math.max(0, Math.min(naturalWidth - 1, cropLeft));
const sy = Math.max(0, Math.min(naturalHeight - 1, cropTop));
const sw = Math.max(1, naturalWidth - sx - cropRight);
const sh = Math.max(1, naturalHeight - sy - cropBottom);
const canvas = document.createElement('canvas');
canvas.width = sw;
canvas.height = sh;
const ctx = canvas.getContext('2d');
ctx.drawImage(img, sx, sy, sw, sh, 0, 0, sw, sh);
return canvas.toDataURL('image/png');
}
function normaliseTaskPdfImage(image, idx=0){
 if(!image || !image.dataUrl) return null;
 const crop = enforceImageCropBounds(image.crop || {});
 return {
 source: image.source || 'pdf',
 mimeType: image.mimeType || 'image/png',
 dataUrl: image.dataUrl,
 originalDataUrl: image.originalDataUrl || image.dataUrl,
 crop,
 name: cleanText(image.name || `task-pdf-page-${idx + 1}.png`) ||
`task-pdf-page-${idx + 1}.png`,
 file_name: cleanText(image.file_name || ''),
 page_number: Math.max(1, parseInt(image.page_number, 10) || (idx + 1))
 };
}
function resolveTaskPdfImages(candidateImages, previousImages=[]){
 const explicit = Array.isArray(candidateImages) ? candidateImages.map((img,
idx)=>normaliseTaskPdfImage(img, idx)).filter(Boolean) : [];
 if(explicit.length) return explicit;
 return Array.isArray(previousImages) ? previousImages.map((img,
idx)=>normaliseTaskPdfImage(img, idx)).filter(Boolean) : [];
}
function taskPdfImageSourceLabel(image){
 const page = Math.max(1, parseInt(image?.page_number, 10) || 1);
 const fileName = cleanText(image?.file_name || 'Imported PDF');
 return `${fileName} · page ${page}.${cropSummaryText(image?.crop || {})}`;
}
function taskPdfCropPanelKey(taskIdx, pdfIdx){
 return `task_pdf_crop_${taskIdx}_${pdfIdx}`;
}
function isTaskPdfCropPanelOpen(taskIdx, pdfIdx){
 state.taskPdfCropPanels = state.taskPdfCropPanels || {};
 return !!state.taskPdfCropPanels[taskPdfCropPanelKey(taskIdx, pdfIdx)];
}

```



```

}
function taskImageSourceLabel(taskImage, imageSize){
 if(!taskImage?.dataUrl) return '';
 if(taskImage.source === 'ai') return `AI generated with OpenAI. Shape setting:
 ${imageSizeLabel(imageSize)}.`;
 if(taskImage.source === 'clipboard') return 'Pasted from clipboard.';
 return 'Uploaded from your device.';
}
function setTaskImageFromDataUrl(idx, dataUrl, options={}){
 const task = state.worksheet?.tasks?.[idx];
 if(!task || !dataUrl) return false;
 task.image_enabled = true;
 task.image = {
 source: options.source || 'upload',
 provider: options.provider || '',
 prompt: cleanText(task.image_prompt || ''),
 mimeType: options.mimeType || 'image/png',
 dataUrl,
 name: cleanText(options.name || `task-${idx + 1}-image.png`) || `task-${idx +
1}-image.png`;
 };
 if(!task.image_prompt) task.image_prompt = buildTaskImagePrompt(task, idx);
 return true;
}
function normaliseWorksheetImage(image, idx, fallback=null){
 const source = image || fallback || {};
 const dataUrl = source.dataUrl || fallback?.dataUrl || '';
 if(!dataUrl) return null;
 return {
 id: cleanText(source.id || fallback?.id || `worksheet-image-${idx + 1}`) ||
`worksheet-image-${idx + 1}`,
 name: cleanText(source.name || fallback?.name || `worksheet-image-${idx +
1}.png`) || `worksheet-image-${idx + 1}.png`,
 mimeType: source.mimeType || fallback?.mimeType || 'image/png',
 dataUrl,
 base64: source.base64 || fallback?.base64 || (String(dataUrl).includes(',') ?
String(dataUrl).split(',')[1] : ''),
 width: parseImageWidth(source.width ?? fallback?.width ?? 100)
 };
}
function resolveWorksheetImages(candidateImages, previousImages=[]){
 const explicit = Array.isArray(candidateImages) ? candidateImages.map((img,
idx)=>normaliseWorksheetImage(img, idx)).filter(Boolean) : [];
 if(explicit.length) return explicit;
 const previous = Array.isArray(previousImages) ? previousImages.map((img,
idx)=>normaliseWorksheetImage(img, idx)).filter(Boolean) : [];
 if(previous.length) return previous;
 return (state.images || []).map((img, idx)=>normaliseWorksheetImage(img, idx,
img)).filter(Boolean);
}

```

```

function syncWorksheetImagesFromUploadsInState(){
 if(!state.worksheet) return [];
 const current = Array.isArray(state.worksheet.worksheet_images) ?
state.worksheet.worksheet_images : [];
 const existingByDataUrl = new Map(current.map(img => [img.dataUrl, img]));
 state.worksheet.worksheet_images = (state.images || []).map((img,
idx)=>normaliseWorksheetImage(existingByDataUrl.get(img.dataUrl) || img, idx,
img)).filter(Boolean);
 return state.worksheet.worksheet_images;
}
function worksheetImageEditorHtml(img, idx){
 const width = parseImageWidth(img?.width ?? 100);
 return `
 <div class="task-card">
 <div class="task-head">
 Worksheet image ${idx + 1}
 <div class="task-actions">
 <button class="mini" onclick="moveWorksheetImage(${idx}, -1)">↑</button>
 <button class="mini" onclick="moveWorksheetImage(${idx}, 1)">↓</button>
 <button class="mini danger"
onclick="removeWorksheetImage(${idx})">Delete</button>
 </div>
 </div>
 <div style="display:grid;grid-template-columns:120px
minmax(0,1fr);gap:12px;align-items:start">
 <div style="border:1px solid
var(--line);border-radius:14px;padding:8px;background:#fff">

 </div>
 <div>
 <div class="tiny"
style="margin-bottom:10px;white-space:nowrap;overflow:hidden;text-overflow:ellipsis
">${esc(img.name)}</div>
 <label>Display width <span id="worksheetImageWidthValue-${idx}"
class="tiny">${width}%</label>
 <input type="range" min="35" max="100" step="5" value="${width}"
oninput="updateWorksheetImageWidth(${idx}, this.value)">
 <div class="field-note" style="margin-top:8px">100% stacks it on its own
line. Smaller widths let images sit side by side.</div>
 </div>
 </div>
 </div>
 `;
}
function buildTaskImagePrompt(task, idx){
 const title = cleanText(state.worksheet?.title || 'Worksheet activity');
 const level = studioAgeGroupText();
 const parts = [

```

```

 'Create a clean, child-friendly educational illustration for a primary school
 worksheet.',
 `Worksheet title: ${title}.`,
 `Age group: ${level}.`,
 task?.passage ? `Stimulus or source text: ${cleanText(task.passage)}` : '',
 task?.prompt ? `Activity focus: ${cleanText(task.prompt)}` : '',
 task?.options?.length ? `Useful context from the answer options:
 ${task.options.join('; ')}.` : '',
 'Show a single clear scene or subject that helps students understand the
 activity.',
 'Keep it school-appropriate, visually clear, and easy to read at worksheet
 size.',
 'No labels, no captions, no speech bubbles, no worksheet borders, no
 watermarks, no visible answer text.'
].filter(Boolean);
 return parts.join(' ');
}

function getImageUsage(){
 return qs('imageUsage')?.value || state.imageUsage || DEFAULT_IMAGE_USAGE;
}

function setImageUsage(val){
 state.imageUsage = val || DEFAULT_IMAGE_USAGE;
 if(qs('imageUsage')) qs('imageUsage').value = state.imageUsage;
}

function getImageGenSize(){
 return qs('imageGenSize')?.value || state.imageGenSize || DEFAULT_IMAGE_GEN_SIZE;
}

function setImageGenSize(val){
 const next = ['1024x1024','1024x1536','1536x1024'].includes(val) ? val :
 DEFAULT_IMAGE_GEN_SIZE;
 state.imageGenSize = next;
 if(qs('imageGenSize')) qs('imageGenSize').value = next;
}

function imageUsageInstruction(mode, count){
 if(!count) return 'No worksheet images were uploaded. Set "image_note" to an
 empty string and do not invent image-based tasks.';
 switch(mode){
 case 'decorative':
 return `The teacher uploaded ${count} image(s). Display them on the worksheet
 as visual support only. Do not make them the centre of the questions. "image_note"
 may be brief or empty.`;
 case 'writing_prompt':
 return `The teacher uploaded ${count} image(s). Use them as the writing
 prompt or stimulus. Make the image(s) central to the worksheet. Provide a clear
 student-facing "image_note" explaining how to use them.`;
 case 'compare_contrast':
 return `The teacher uploaded ${count} image(s). Use them for
 compare-and-contrast, observation, or connection-making tasks. If fewer than two
 images are available, still use the uploaded material sensibly without inventing
 extra images. Provide a clear student-facing "image_note".`;
 }
}

```

```

 default:
 return `The teacher uploaded ${count} image(s). Build meaningful questions
from them. At least one substantial task should refer directly to the uploaded
image material. Provide a clear student-facing "image_note".`;
 }
}
function inferMimeTypeFromDataUrl(dataUrl, fallback='image/png'){
 const match = String(dataUrl || '').match(/^data:([^\s;]+)([;,\s]*/i);
 return match?.[1] || fallback;
}
function dataUrlToAttachment(dataUrl, fallbackMimeType='image/png'){
 const value = String(dataUrl || '');
 const parts = value.split(',');
 if(parts.length < 2) return null;
 return {
 mimeType: inferMimeTypeFromDataUrl(value, fallbackMimeType),
 base64: parts[1]
 };
}
function summariseWorksheetImageForAI(image, idx=0){
 if(!image) return null;
 return {
 name: cleanText(image.name || `worksheet-image-${idx + 1}.png`) ||
`worksheet-image-${idx + 1}.png`,
 source: image.source || 'upload',
 mimeType: image.mimeType || inferMimeTypeFromDataUrl(image.dataUrl || '',
'image/png'),
 display_width: parseImageWidth(image.width ?? 100)
 };
}
function summariseTaskImageForAI(task, idx=0){
 if(!task?.image?.dataUrl) return null;
 return {
 name: cleanText(task.image.name || `task-${idx + 1}-image.png`) || `task-${idx
+ 1}-image.png`,
 source: task.image.source || 'upload',
 mimeType: task.image.mimeType || inferMimeTypeFromDataUrl(task.image.dataUrl ||
'', 'image/png'),
 prompt: cleanText(task.image.prompt || task.image_prompt || ''),
 display_width: parseImageWidth(task.image_width ?? 100),
 shape: parseImageGenSize(task.image_size || state.imageGenSize ||
DEFAULT_IMAGE_GEN_SIZE)
 };
}
function summariseTaskPdfImageForAI(image, idx=0, displayWidth=100){
 if(!image?.dataUrl) return null;
 return {
 name: cleanText(image.name || `task-pdf-page-${idx + 1}.png`) ||
`task-pdf-page-${idx + 1}.png`,
 file_name: cleanText(image.file_name || 'Imported PDF') || 'Imported PDF',

```

```

 page_number: Math.max(1, parseInt(image.page_number, 10) || (idx + 1)),
 crop: enforceImageCropBounds(image.crop || {}),
 mimeType: image.mimeType || inferMimeTypeFromDataUrl(image.dataUrl || '',
'image/png'),
 display_width: parseImageWidth(displayWidth ?? 100)
 };
}
buildWorksheetAiContext = function(){
 const worksheet = state.worksheet || {};
 const tasks = Array.isArray(worksheet.tasks) ? worksheet.tasks : [];
 const worksheetImages = Array.isArray(worksheet.worksheet_images) ?
worksheet.worksheet_images : [];
 const totalTaskPdfPages = tasks.reduce((sum, task) => sum +
(Array.isArray(task?.pdf_images) ? task.pdf_images.length : 0), 0);
 return {
 meta: {
 note: 'Large image data has been removed from this AI context to keep edit
requests reliable. Use the image summaries below, plus any attached images in the
request, as the visual context.',
 step3_upload_count: Array.isArray(state.images) ? state.images.length : 0,
 worksheet_image_count: worksheetImages.length,
 task_pdf_page_count: totalTaskPdfPages
 },
 domain: worksheet.domain || '',
 learning_outcomes: Array.isArray(worksheet.learning_outcomes) ?
[...worksheet.learning_outcomes] : [],
 title: worksheet.title || '',
 instructions: worksheet.instructions || '',
 success_criteria: worksheet.success_criteria || '',
 intro_text: worksheet.intro_text || '',
 image_note: worksheet.image_note || '',
 extension: worksheet.extension || '',
 worksheet_images: worksheetImages.map((image, idx) =>
summariseWorksheetImageForAI(image, idx)).filter(Boolean),
 tasks: tasks.map((task, idx) => ({
 number: task.number || (idx + 1),
 passage: task.passage || '',
 prompt: task.prompt || '',
 options: Array.isArray(task.options) ? [...task.options] : [],
 answer_lines: Math.max(0, parseInt(task.answer_lines, 10) || 0),
 answer: task.answer || '',
 image_enabled: !!task.image_enabled,
 image_prompt: cleanText(task.image_prompt || ''),
 image_width: parseImageWidth(task.image_width ?? 100),
 image_size: parseImageGenSize(task.image_size || state.imageGenSize ||
DEFAULT_IMAGE_GEN_SIZE),
 image_summary: summariseTaskImageForAI(task, idx),
 pdf_image_summaries: Array.isArray(task.pdf_images) ?
task.pdf_images.map((image, pdfIdx) => summariseTaskPdfImageForAI(image, pdfIdx,
task.image_width)).filter(Boolean) : []

```

```

 }));
 };
};
function getReferencedTaskIndexesFromText(text='') {
 const indexes = new Set();
 const value = String(text || '');
 const explicitMatches = value.match(/task\s*\d+/gi) || [];
 explicitMatches.forEach(match => {
 const num = parseInt(match.replace(/^[^d]/g, ''), 10);
 if (Number.isFinite(num) && num > 0) indexes.add(num - 1);
 });
 const groupedMatches = value.match(/tasks?\s+[\d\s,;&and-]+/gi) || [];
 groupedMatches.forEach(match => {
 (match.match(/\d+/g) || []).forEach(numText => {
 const num = parseInt(numText, 10);
 if (Number.isFinite(num) && num > 0) indexes.add(num - 1);
 });
 });
 return [...indexes].filter(idx => idx >= 0 && idx <
(state.worksheet?.tasks?.length || 0));
}
function requestMentionsVisuals(text='') {
 return
/^(image|images|picture|pictures|photo|photos|map|maps|diagram|illustration|pdf|page|pages|figure|visual|stimulus)$/i.test(String(text || ''));
}
function requestExplicitlyWantsAncillarySections(text='') {
 return
/^(instructions?|success\s*criteria|extension|extension\s*task|challenge\s*task|bonus\s*task)$/i.test(String(text || ''));
}
function requestExplicitlyWantsTaskAdditions(text='') {
 const value = String(text || '');
 return
/^(add|append|insert|include|create|make|generate|give|write)[\s\S]{0,40}^(task|tasks|question|questions|activity|activities|exercise|exercises|section|sections|prompt|prompts)$/i.test(value)
||
/^(new|another|extra|more)\s+(task|tasks|question|questions|activity|activities|exercise|exercises|prompt|prompts)$/i.test(value);
}
function requestExplicitlyWantsTaskRemovals(text='') {
 return
/^(remove|delete|drop|cut)[\s\S]{0,30}^(task|tasks|question|questions|activity|activities|exercise|exercises|section|sections)$/i.test(String(text || ''));
}
function getTasksWithVisuals() {
 const tasks = state.worksheet?.tasks || [];
 return tasks.map((task, idx) => ({ task, idx })).filter(({ task }) =>
!!(task?.image?.dataUrl || (Array.isArray(task?.pdf_images) &&

```

```

task.pdf_images.some(image => image?.dataUrl))));
}
function getLikelyTargetTaskIndexesFromRequest(request='') {
 const referenced = getReferencedTaskIndexesFromText(request);
 if(referenced.length) return referenced;
 const visualTasks = getTasksWithVisuals().map(entry => entry.idx);
 if(requestMentionsVisuals(request) && visualTasks.length === 1) return
[visualTasks[0]];
 return [];
}
function collectTaskImageAttachments(taskIndexes=[], limit=6, seen=new Set()){
 const attachments = [];
 const tasks = state.worksheet?.tasks || [];
 (Array.isArray(taskIndexes) ? taskIndexes : []).forEach(idx => {
 if(attachments.length >= limit) return;
 const task = tasks[idx];
 if(!task) return;
 const candidates = [];
 if(task.image?.dataUrl) candidates.push({ dataUrl: task.image.dataUrl,
mimeType: task.image.mimeType || 'image/png' });
 (Array.isArray(task.pdf_images) ? task.pdf_images : []).forEach(image => {
 if(image?.dataUrl) candidates.push({ dataUrl: image.dataUrl, mimeType:
image.mimeType || 'image/png' });
 });
 candidates.forEach(candidate => {
 if(attachments.length >= limit || !candidate?.dataUrl ||
seen.has(candidate.dataUrl)) return;
 const attachment = dataUrlToAttachment(candidate.dataUrl, candidate.mimeType
|| 'image/png');
 if(!attachment?.base64) return;
 seen.add(candidate.dataUrl);
 attachments.push(attachment);
 });
 });
 return attachments;
}
function collectWorksheetImageAttachments(limit=6, seen=new Set()){
 const attachments = [];
 const worksheetImages = Array.isArray(state.worksheet?.worksheet_images) ?
state.worksheet.worksheet_images : [];
 worksheetImages.forEach(image => {
 if(attachments.length >= limit || !image?.dataUrl || seen.has(image.dataUrl))
return;
 const attachment = dataUrlToAttachment(image.dataUrl, image.mimeType ||
'image/png');
 if(!attachment?.base64) return;
 seen.add(image.dataUrl);
 attachments.push(attachment);
 });
 return attachments;
}

```

```

}
collectImagesForRequest = function(request='') {
 const seen = new Set();
 const targetTaskIndexes = getLikelyTargetTaskIndexesFromRequest(request);
 let attachments = collectTaskImageAttachments(targetTaskIndexes, 6, seen);
 if(!attachments.length){
 const referenced = getReferencedTaskIndexesFromText(request);
 attachments = collectTaskImageAttachments(referenced, 6, seen);
 }
 if(requestMentionsVisuals(request) && attachments.length < 6){
 attachments = attachments.concat(collectWorksheetImageAttachments(6 -
attachments.length, seen));
 }
 if(requestMentionsVisuals(request) && attachments.length < 6){
 const visualTaskIndexes = getTasksWithVisuals().map(entry => entry.idx);
 attachments = attachments.concat(collectTaskImageAttachments(visualTaskIndexes,
6 - attachments.length, seen));
 }
 if(requestMentionsVisuals(request) && attachments.length < 6){
 const allTaskIndexes = (state.worksheet?.tasks || []).map((_, idx) => idx);
 attachments = attachments.concat(collectTaskImageAttachments(allTaskIndexes, 6
- attachments.length, seen));
 }
 return attachments.slice(0, 6);
};

function collectQualityCheckAttachments(limit=8){
 const seen = new Set();
 let attachments = [];
 attachments = attachments.concat(collectWorksheetImageAttachments(limit, seen));
 if(attachments.length < limit){
 const visualTaskIndexes = getTasksWithVisuals().map(entry => entry.idx);
 attachments = attachments.concat(collectTaskImageAttachments(visualTaskIndexes,
limit - attachments.length, seen));
 }
 return attachments.slice(0, limit);
}

function qualityStatusLabel(status='') {
 const value = String(status || '').toLowerCase();
 if(value === 'ready') return 'Ready';
 if(value === 'minor') return 'Minor issues';
 if(value === 'needs_work') return 'Needs work';
 return 'Not checked';
}

function qualityStatusClass(status='') {
 const value = String(status || '').toLowerCase();
 return ['ready', 'minor', 'needs_work'].includes(value) ? value : '';
}

function qualitySeverityLabel(severity='') {
 const value = String(severity || '').toLowerCase();

```



```

 if(value === 'issue') return 'Issue';
 if(value === 'warning') return 'Warning';
 if(value === 'ok') return 'OK';
 return 'Note';
 }
 function normaliseQualityCheckResult(result={}){
 const rawItems = Array.isArray(result?.checks) ? result.checks :
 Array.isArray(result?.items) ? result.items : [];
 const checks = rawItems.slice(0, 10).map(item => ({
 severity: ['ok','warning','issue'].includes(String(item?.severity ||
 '').toLowerCase()) ? String(item.severity).toLowerCase() : 'warning',
 area: cleanText(item?.area || 'General') || 'General',
 location: cleanText(item?.location || '') || 'Whole worksheet',
 finding: cleanText(item?.finding || item?.issue || '') || 'No detail
provided.',
 suggestion: cleanText(item?.suggestion || '') || '',
 fix_request: cleanText(item?.fix_request || item?.fixPrompt || '') || ''
 }));
 let status = String(result?.status || result?.overall || '').toLowerCase();
 if(!['ready','minor','needs_work'].includes(status)){
 status = checks.some(item => item.severity === 'issue') ? 'needs_work' :
checks.some(item => item.severity === 'warning') ? 'minor' : 'ready';
 }
 return {
 status,
 summary: cleanText(result?.summary || '') || (checks.length ? 'Worksheet check
complete.' : 'No issues found.'),
 checkedAt: new Date().toISOString(),
 checks,
 quick_fix_request: cleanText(result?.quick_fix_request || '') ||
checks.filter(item => item.severity !== 'ok').map(item => item.fix_request ||
`${item.location}: ${item.suggestion || item.finding}`).filter(Boolean).join('\n')
 };
 }
 function qualityCheckSubtitle(){
 if(!state.qualityCheck) return 'Run a quick AI sanity check before export.';
 const count = (state.qualityCheck.checks || []).filter(item => item.severity !==
'ok').length;
 return `${qualityStatusLabel(state.qualityCheck.status)} · ${count} flagged
item${count === 1 ? '' : 's'}.`;
 }
 function renderQualityCheckBody(){
 const check = state.qualityCheck;
 if(!check){
 return `
 <div class="quality-empty">
 Run this after you have generated or edited the worksheet. It checks
 clarity, age level, task-answer alignment, image usefulness and obvious
 print/export risks. It is not magic, but it is better than sending Claude's
 nonsense straight to the photocopier.

```

```

 </div>
 <div class="pill-row" style="margin-top:10px">
 <button class="small-btn primary" type="button"
onclick="checkWorksheetBeforeExport()">Check worksheet now</button>
 </div>`;
 }
 const flagged = (check.checks || []).filter(item => item.severity !==
'ok').length;
 const checkedAt = check.checkedAt ? new Date(check.checkedAt).toLocaleString() :
'';
 return `
 <div class="quality-panel">
 <div class="quality-score-row">
 <div>
 <span class="quality-badge
${qualityStatusClass(check.status)}">${qualityStatusLabel(check.status)}
 ${checkedAt ? `Checked
${esc(checkedAt)}` : ''}
 </div>
 <div class="pill-row">
 <button class="small-btn" type="button"
onclick="checkWorksheetBeforeExport()">Run again</button>
 ${flagged && check.quick_fix_request ? `<button class="small-btn primary"
type="button" onclick="sendQualityCheckToModify()">Send issues to Modify</button>`
: ''}
 <button class="small-btn" type="button"
onclick="clearQualityCheck()">Clear</button>
 </div>
 </div>
 <div class="tiny" style="line-height:1.55">${esc(check.summary)}</div>
 <div class="quality-list">
 ${((check.checks || []).length ? check.checks.map(item => `
 <div class="quality-item">
 <div class="quality-item-head">
 <span class="quality-pill
${esc(item.severity)}">${qualitySeverityLabel(item.severity)}
 ${esc(item.area)}
 <strong style="font-size:12px">${esc(item.location)}
 </div>
 <p><strong style="color:var(--ink)">Finding:
${esc(item.finding)}</p>
 ${item.suggestion ? `<p><strong
style="color:var(--ink)">Suggestion: ${esc(item.suggestion)}</p>` : ''}
 </div>
 `).join('') : '<div class="quality-empty">No issues flagged. Suspiciously
wholesome.</div>'}
 </div>
 </div>`;
}
function buildQualityCheckPrompt(){

```

```

const worksheetContext = JSON.stringify(buildWorksheetAiContext(), null, 2);
const hasAnswers = hasAnswerKeyData();
const exportMode = qs('outputMode')?.value || 'student';
return `You are doing a final quality check for a printable ${getCycleLabel()}
(${getStudioYearLevel()}) worksheet before a teacher exports it.
${studioCyclePromptGuidance()}`

```

Current preview/export mode: \${exportMode}.  
 Answer key present: \${hasAnswers ? 'yes' : 'no'}.  
 Difficulty setting: \${qs('difficulty')?.value || 'cm1\_moderate'}.

Worksheet JSON:  
 \${worksheetContext}

Check for issues that would actually matter to a busy teacher using this selected cycle and year level:

- unclear instructions or vague task wording
- tasks that seem too easy, too hard, or not age-appropriate
- missing answer lines where students clearly need space to write
- multiple-choice tasks with poor or obvious options
- answer key mismatch or missing answer guidance, but only if an answer key exists or the export mode requires one
- learning outcomes that do not match the actual worksheet
- uploaded/generated images that are not clearly useful, based on image summaries and any attached visuals
- likely print/layout concerns: too much text, overly long task, image-heavy page, confusing structure
- spelling/grammar/formatting issues in student-facing text

Be practical, not precious. Do not complain that optional instructions, success criteria or extension are blank unless the worksheet genuinely needs them. Do not invent problems just to look busy.

Return ONLY valid JSON with this exact schema:

```

{
 "status": "ready | minor | needs_work",
 "summary": "<one short teacher-facing summary>",
 "checks": [
 {
 "severity": "ok | warning | issue",
 "area": "Layout | Instructions | Tasks | Answers | Learning outcomes | Images | Language",
 "location": "Whole worksheet | Task 2 | Learning outcome 1 | etc.",
 "finding": "<specific finding>",
 "suggestion": "<specific practical suggestion>",
 "fix_request": "<short instruction that could be sent to an AI editor to fix this>"
 }
],
 "quick_fix_request": "<combined concise instruction to fix the important

```

```
warnings/issues>"
}
```

Rules:

- Include at most 8 checks.
- Include OK checks only when they are useful; otherwise focus on warnings/issues.
- Use "ready" only if there are no meaningful problems.
- Use "minor" for small fixes.
- Use "needs\_work" if students would likely be confused or the resource is not export-ready.`;

```
}
async function checkWorksheetBeforeExport(){
 if(!state.worksheet) return showStatus('Generate or load a worksheet first.',
'warn');
 setEditorAiBusy(true, 'Checking worksheet before export...');
 try{
 showStatus('Checking worksheet...', 'warn');
 const result = await callAI({
 prompt: buildQualityCheckPrompt(),
 modify: true,
 extraImages: collectQualityCheckAttachments(8)
 });
 state.qualityCheck = normaliseQualityCheckResult(result);
 setEditorAccordionOpen('editor_quality_check', true);
 renderEditor();
 showStatus(state.qualityCheck.status === 'ready' ? 'Worksheet check complete:
ready.' : 'Worksheet check complete. Review the flagged items.',
state.qualityCheck.status === 'needs_work' ? 'warn' : 'ok');
 }catch(err){
 showStatus(err.message || 'Could not check the worksheet.', 'err');
 }finally{
 setEditorAiBusy(false);
 forceEditorRefresh();
 }
}
function clearQualityCheck(){
 state.qualityCheck = null;
 renderEditor();
 showStatus('Worksheet check cleared.', 'ok');
}
function sendQualityCheckToModify(){
 if(!state.qualityCheck?.quick_fix_request) return showStatus('No fix request
available.', 'warn');
 setEditorAccordionOpen('editor_modify', true);
 renderEditor();
 const box = qs('modifyRequest');
 if(box){
 box.value = `Fix the issues from the worksheet
check:\n\n${state.qualityCheck.quick_fix_request}`;
 box.focus();
 }
}
```

```

 }
 showStatus('Issues copied into Modify with AI. Review them, then run the
modification if they make sense.', 'ok');
}
function mergeModifyResultIntoWorksheet(previousWorksheet, aiResult,
targetTaskIndexes=[], allowTaskCountChange=false){
 const previous = previousWorksheet && typeof previousWorksheet === 'object' ?
previousWorksheet : {};
 const result = aiResult && typeof aiResult === 'object' ? aiResult : {};
 const merged = { ...previous, ...result };
 const previousTasks = Array.isArray(previous.tasks) ? previous.tasks.map(task =>
task ? { ...task } : task) : [];
 const resultTasks = Array.isArray(result.tasks) ? result.tasks.filter(Boolean) :
[];
 const nonEmptyResultTasks = resultTasks.filter(task => cleanText(task?.prompt ||
task?.passage || task?.answer || '').trim());

 if(!allowTaskCountChange && previousTasks.length){
 if(resultTasks.length === previousTasks.length){
 merged.tasks = resultTasks;
 } else if(targetTaskIndexes.length === 1 && nonEmptyResultTasks.length){
 const idx = targetTaskIndexes[0];
 const targetNumber = idx + 1;
 const candidate = nonEmptyResultTasks.find(task => parseInt(task?.number, 10)
=== targetNumber) || nonEmptyResultTasks[0];
 previousTasks[idx] = { ...previousTasks[idx], ...candidate, number:
targetNumber };
 merged.tasks = previousTasks;
 } else if(nonEmptyResultTasks.length === 1 && previousTasks.length){
 const idx = targetTaskIndexes[0] ?? 0;
 previousTasks[idx] = { ...previousTasks[idx], ...nonEmptyResultTasks[0],
number: idx + 1 };
 merged.tasks = previousTasks;
 } else {
 merged.tasks = previousTasks;
 }
 } else {
 merged.tasks = resultTasks.length ? resultTasks : previousTasks;
 }

 if(!cleanText(result.title || '')) merged.title = previous.title || '';
 if(!cleanText(result.domain || '')) merged.domain = previous.domain || '';
 if(!Array.isArray(result.learning_outcomes) || !result.learning_outcomes.length){
 merged.learning_outcomes = Array.isArray(previous.learning_outcomes) ?
previous.learning_outcomes : [];
 }
 if(!Array.isArray(result.worksheet_images) || !result.worksheet_images.length){
 merged.worksheet_images = Array.isArray(previous.worksheet_images) ?
previous.worksheet_images : [];
 }
}

```

```

 if(result.instructions === undefined) merged.instructions = previous.instructions
 || '';
 if(result.success_criteria === undefined) merged.success_criteria =
previous.success_criteria || '';
 if(result.extension === undefined) merged.extension = previous.extension || '';
 if(result.intro_text === undefined) merged.intro_text = previous.intro_text ||
'';
 if(result.image_note === undefined) merged.image_note = previous.image_note ||
'';
 return merged;
}
function forceEditorRefresh(){
 renderEditor();
 updatePreview();
 requestAnimationFrame(() => {
 renderEditor();
 updatePreview();
 });
 setTimeout(() => {
 renderEditor();
 updatePreview();
 }, 0);
 setTimeout(() => {
 renderEditor();
 updatePreview();
 }, 80);
 setTimeout(() => {
 renderEditor();
 updatePreview();
 }, 180);
}
function schemaHint(taskCount, includeAnswers = (hasAnswerKeyData() ||
shouldGenerateAnswers()), aiSuggestedTaskCount = false){
 const taskCountRule = aiSuggestedTaskCount
 ? '- "tasks" must contain between 1 and 8 tasks. Choose the count that best
fits the worksheet brief, level, and source material.'
 : '- "tasks" must contain between ${Math.max(1, taskCount-1)} and ${Math.min(8,
taskCount+1)} tasks.`;
 if(includeAnswers){
 return `Return ONLY valid JSON with this exact schema:
{
"domain": "<one exact domain string from the allowed list>",
"learning_outcomes": ["<outcome 1>", "<outcome 2>"],
"title": "<worksheet title>",
"instructions": "<short student-facing instructions>",
"success_criteria": "<short success criteria block, may be empty>",
"intro_text": "<optional short intro paragraph, may be empty>",
"image_note": "<student-facing note about the uploaded image(s), may be empty>",
"tasks": [
{

```

```

 "number": 1,
 "passage": "<optional separate passage, sentence set, word bank or stimulus
text>",
 "prompt": "<task text>",
 "options": ["<option A>", "<option B>"],
 "answer_lines": 0,
 "answer": "<teacher answer or marking point>"
 }
],
"extension": "<optional extension task, may be empty>"
}

```

#### Rules:

- "domain" must be one exact domain string from the allowed list only.
- "learning\_outcomes" must contain 2 to 4 outcomes copied VERBATIM from the allowed curriculum bank for the chosen domain only. Do not invent, paraphrase, merge or rewrite outcomes.

`${taskCountRule}`

- Each task needs a clear prompt.
- Use "passage" only when a task genuinely needs a separate reading extract, sentence set, word bank, data set or other stimulus. Otherwise return an empty string.
- Use "options" only when multiple-choice or selection options genuinely help. Otherwise return an empty array.
- Set "answer\_lines" to 0 by default. Only use a non-zero value if the teacher explicitly asks for answer lines in the brief or template.
- Every task must include an "answer".
- Keep the worksheet suitable for the selected level, and do not drift into secondary-school expectations.
- Unless the teacher explicitly asks for them, return empty strings for "instructions", "success\_criteria" and "extension".
- Keep the writing clear, precise and practical.`;

return `Return ONLY valid JSON with this exact schema:

```

{
 "domain": "<one exact domain string from the allowed list>",
 "learning_outcomes": ["<outcome 1>", "<outcome 2>"],
 "title": "<worksheet title>",
 "instructions": "<short student-facing instructions>",
 "success_criteria": "<short success criteria block, may be empty>",
 "intro_text": "<optional short intro paragraph, may be empty>",
 "image_note": "<student-facing note about the uploaded image(s), may be empty>",
 "tasks": [
 {
 "number": 1,
 "passage": "<optional separate passage, sentence set, word bank or stimulus
text>",
 "prompt": "<task text>",
 "options": ["<option A>", "<option B>"],
 "answer_lines": 0
 }
]
}

```

```

 }
],
 "extension": "<optional extension task, may be empty>"
}

```

#### Rules:

- "domain" must be one exact domain string from the allowed list only.
- "learning\_outcomes" must contain 2 to 4 outcomes copied VERBATIM from the allowed curriculum bank for the chosen domain only. Do not invent, paraphrase, merge or rewrite outcomes.

```

${taskCountRule}

```

- Each task needs a clear prompt.
- Use "passage" only when a task genuinely needs a separate reading extract, sentence set, word bank, data set or other stimulus. Otherwise return an empty string.
- Use "options" only when multiple-choice or selection options genuinely help. Otherwise return an empty array.
- Set "answer\_lines" to 0 by default. Only use a non-zero value if the teacher explicitly asks for answer lines in the brief or template.
- Do NOT generate teacher answers yet. Omit the "answer" field or leave it as an empty string for every task.
- Keep the worksheet suitable for the selected level, and do not drift into secondary-school expectations.
- Unless the teacher explicitly asks for them, return empty strings for "instructions", "success\_criteria" and "extension".
- Keep the writing clear, precise and practical.`;

```

}
function normaliseTask(task, idx, previousTask=null){
 const base = {
 number: idx + 1,
 passage: cleanText(task?.passage || ''),
 prompt: cleanText(task?.prompt || ''),
 options: Array.isArray(task?.options) ?
task.options.filter(Boolean).map(v=>String(v).trim()) : [],
 answer_lines: parseAnswerLines(task?.answer_lines),
 answer: cleanText(task?.answer || ''),
 image_enabled: !(task?.image_enabled ?? previousTask?.image_enabled ?? false),
 image_prompt: cleanText(task?.image_prompt || previousTask?.image_prompt ||
''),
 image_size: parseImageGenSize(task?.image_size ?? previousTask?.image_size ??
state.imageGenSize ?? DEFAULT_IMAGE_GEN_SIZE),
 image_width: parseImageWidth(task?.image_width ?? previousTask?.image_width ??
100),
 image: normaliseTaskImage(task?.image || previousTask?.image || null),
 pdf_images: resolveTaskPdfImages(task?.pdf_images, previousTask?.pdf_images ||
[]),
 };
 if(base.image_enabled && !base.image_prompt){
 base.image_prompt = buildTaskImagePrompt(base, idx);
 }
}

```



```

 return base;
}
function renderCriteriaBlock(text){
 const rendered = renderRichText(text);
 if(!rendered) return '';
 return `

{ state.imageUsage = imageUsage.value;
savePrefs(); });
 imageUsage.addEventListener('change', ()=>{ state.imageUsage =
imageUsage.value; savePrefs(); });
 }
}
function ensureImageGenerationUi(){
 if(qs('imageApiKey')) return;
 const outputModeField = qs('modelName')?.closest('.field') ||
qs('outputMode')?.closest('.grid-2');
 if(!outputModeField) return;
 const block = document.createElement('div');
 block.className = 'field';
 block.innerHTML = `
 <div class="image-gen-badge"></div>
 `;
}


```

```

 <div class="image-gen-grid">
 <div class="field">
 <label for="imageApiKey">OpenAI image API key</label>
 <input id="imageApiKey" type="password" placeholder="Paste your OpenAI API
key">
 <div class="tiny" style="margin-top:6px"></div>
 </div>
 <div class="field">
 <label for="imageModel">Image model</label>
 <input id="imageModel" type="text" value="gpt-image-1.5"
placeholder="gpt-image-1.5">
 <div class="tiny" style="margin-top:6px"></div>
 </div>
 </div>
 `;
 outputModeField.insertAdjacentElement('afterend', block);
}
function wireImageGenerationControls(){
 ['imageApiKey','imageModel'].forEach(id => {
 const el = qs(id);
 if(!el || el.dataset.bound) return;
 el.dataset.bound = '1';
 el.addEventListener('input', savePrefs);
 el.addEventListener('change', savePrefs);
 });
}
function ensureTaskImageUploadInput(){
 let input = qs('taskImageUploadInput');
 if(input) return input;
 input = document.createElement('input');
 input.type = 'file';
 input.accept = 'image/*';
 input.id = 'taskImageUploadInput';
 input.className = 'hidden';
 input.addEventListener('change', async (e)=>{
 const idx = parseInt(input.dataset.taskIdx || '-1', 10);
 const file = e.target.files?.[0];
 if(!file || !state.worksheet?.tasks[idx]) return;
 try{
 const dataUrl = await fileToDataUrl(file);
 setTaskImageFromDataUrl(idx, dataUrl, {
 source: 'upload',
 mimeType: file.type || 'image/png',
 name: file.name || `task-${idx + 1}-image.png`
 });
 renderEditor();
 updatePreview();
 showStatus(`Image added to task ${idx + 1}.`, 'ok');
 }catch(err){
 showStatus('Could not load that image from disk.', 'err');
 }
 });
}

```

```

 }finally{
 input.value = '';
 }
});
document.body.appendChild(input);
return input;
}

function ensureTaskPdfUploadInput(){
 let input = qs('taskPdfUploadInput');
 if(input) return input;
 input = document.createElement('input');
 input.type = 'file';
 input.accept = 'application/pdf,.pdf';
 input.id = 'taskPdfUploadInput';
 input.className = 'hidden';
 input.addEventListener('change', async (e)=>{
 const idx = parseInt(input.dataset.taskIdx || '-1', 10);
 const file = e.target.files?.[0];
 if(!file || !state.worksheet?.tasks?.[idx]) return;
 try{
 await openTaskPdfImport(idx, file);
 }catch(err){
 console.error(err);
 showStatus(err.message || 'Could not open that PDF.', 'err');
 }finally{
 input.value = '';
 }
 });
 document.body.appendChild(input);
 return input;
}

async function ensurePdfJsReady(){
 if(!window.pdfjsLib) throw new Error('PDF import library failed to load. Reload
the page and try again.');
```

if(window.pdfjsLib.GlobalWorkerOptions &&  
!window.pdfjsLib.GlobalWorkerOptions.workerSrc){  
 window.pdfjsLib.GlobalWorkerOptions.workerSrc =  
'https://cdnjs.cloudflare.com/ajax/libs/pdf.js/3.11.174/pdf.worker.min.js';  
}

```

 return window.pdfjsLib;
}

async function renderPdfPageToDataURL(page, targetWidth=1400){
 const baseViewport = page.getViewport({ scale: 1 });
 const scale = Math.max(0.35, Math.min(2.2, targetWidth / Math.max(1,
baseViewport.width)));
 const viewport = page.getViewport({ scale });
 const canvas = document.createElement('canvas');
 const context = canvas.getContext('2d', { alpha: false });
 canvas.width = Math.round(viewport.width);

```

```

 canvas.height = Math.round(viewport.height);
 await page.render({ canvasContext: context, viewport }).promise;
 return canvas.toDataURL('image/png');
}

function rotateImageDataUrl(dataUrl, degrees=90){
 return new Promise((resolve, reject) => {
 const img = new Image();
 img.onload = () => {
 const angle = ((degrees % 360) + 360) % 360;
 const swap = angle === 90 || angle === 270;
 const canvas = document.createElement('canvas');
 canvas.width = swap ? img.height : img.width;
 canvas.height = swap ? img.width : img.height;
 const ctx = canvas.getContext('2d', { alpha: false });
 ctx.fillStyle = 'rgba(0,0,0,0)';
 ctx.fillRect(0, 0, canvas.width, canvas.height);
 ctx.translate(canvas.width / 2, canvas.height / 2);
 ctx.rotate(angle * Math.PI / 180);
 ctx.drawImage(img, -img.width / 2, -img.height / 2);
 resolve(canvas.toDataURL('image/png'));
 };
 img.onerror = () => reject(new Error('Could not load image for rotation.'));
 img.src = dataUrl;
 });
}

function ensureTaskPdfImportModal(){
 let modal = qs('taskPdfImportModal');
 if(modal) return modal;
 modal = document.createElement('div');
 modal.id = 'taskPdfImportModal';
 modal.className = 'task-pdf-modal hidden';
 modal.innerHTML = `
 <div class="task-pdf-modal-card" role="dialog" aria-modal="true"
aria-labelledby="taskPdfImportTitle">
 <div class="task-pdf-modal-head">
 <div>
 <h3 id="taskPdfImportTitle">Import PDF pages into task</h3>
 <p id="taskPdfImportSubtitle">Pick the pages you want to keep, then
they'll be dropped into the task as PNGs.</p>
 </div>
 <button class="small-btn" type="button"
data-task-pdf-action="close">Close</button>
 </div>
 <div class="task-pdf-modal-body" id="taskPdfImportBody"></div>
 <div class="task-pdf-modal-actions">
 <div class="left">
 <button class="small-btn" type="button" data-task-pdf-action="select-all"
onclick="setAllTaskPdfPages(true); return false;">Select all</button>
 <button class="small-btn" type="button" data-task-pdf-action="clear-all"

```

```

onclick="setAllTaskPdfPages(false); return false;">Clear all</button>
 </div>
 <div class="right">
 <button class="small-btn" type="button"
data-task-pdf-action="close">Cancel</button>
 <button class="small-btn primary" id="taskPdfImportConfirmBtn"
type="button" data-task-pdf-action="confirm" onclick="confirmTaskPdfImport();
return false;">Insert selected pages</button>
 </div>
</div>
</div>`;
modal.addEventListener('click', (event)=>{
 if(event.target === modal){
 closeTaskPdfImport();
 return;
 }
 const actionBtn = event.target.closest('[data-task-pdf-action]');
 if(actionBtn){
 const action = actionBtn.getAttribute('data-task-pdf-action');
 if(action === 'close') closeTaskPdfImport();
 if(action === 'select-all') setAllTaskPdfPages(true);
 if(action === 'clear-all') setAllTaskPdfPages(false);
 if(action === 'confirm') confirmTaskPdfImport();
 return;
 }
 const checkbox = event.target.closest('[data-task-pdf-checkbox]');
 if(checkbox){
 const pageIdx = parseInt(checkbox.getAttribute('data-task-pdf-checkbox'),
10);
 if(Number.isFinite(pageIdx)) toggleTaskPdfPage(pageIdx, !!checkbox.checked);
 }
});
modal.addEventListener('change', (event)=>{
 const checkbox = event.target.closest('[data-task-pdf-checkbox]');
 if(!checkbox) return;
 const pageIdx = parseInt(checkbox.getAttribute('data-task-pdf-checkbox'), 10);
 if(Number.isFinite(pageIdx)) toggleTaskPdfPage(pageIdx, !!checkbox.checked);
});
document.body.appendChild(modal);
return modal;
}
function renderTaskPdfImportModal(){
 const modal = ensureTaskPdfImportModal();
 const subtitle = qs('taskPdfImportSubtitle');
 const body = qs('taskPdfImportBody');
 const confirmBtn = qs('taskPdfImportConfirmBtn');
 const session = state.taskPdfImport;
 if(!session){
 modal.classList.add('hidden');
 return;
 }

```

```

 }
 modal.classList.remove('hidden');
 subtitle.textContent = `${session.fileName} · choose the
page${session.pages.length === 1 ? '' : 's'} you want to insert into task
${session.taskIdx + 1}.`;
 const selectedCount = session.pages.filter(page => page.selected).length;
 confirmBtn.textContent = session.inserting
 ? `Inserting ${selectedCount} || ''} page${selectedCount === 1 ? '' :
's'}...`.replace(/\\s+...$/, '...')
 : (selectedCount ? `Insert ${selectedCount} page${selectedCount === 1 ? '' :
's'}` : 'Insert selected pages');
 confirmBtn.disabled = !selectedCount || !!session.inserting;
 if(session.loading){
 body.innerHTML = `

Reading
PDF...Building the page previews now. They may appear progressively as they
load.</div>`;
 return;
 }
 if(!session.pages.length){
 body.innerHTML = `

const pdfjs = await ensurePdfJsReady();

const modal = ensureTaskPdfImportModal();

modal.classList.remove('hidden');

state.taskPdfImport = {

 taskIdx: idx,

 fileName: cleanText(file.name || 'Imported PDF') || 'Imported PDF',

 loading: true,

 inserting: false,

 pages: [],

 pdfBytes: null

};

renderTaskPdfImportModal();


```

```

const bytes = new Uint8Array(await file.arrayBuffer());
state.taskPdfImport.pdfBytes = bytes;
const doc = await pdfjs.getDocument({ data: bytes }).promise;
const pages = [];
for(let pageNumber = 1; pageNumber <= doc.numPages; pageNumber++){
 const page = await doc.getPage(pageNumber);
 const thumbDataUrl = await renderPdfPageToDataUrl(page, 220);
 let insertDataUrl = thumbDataUrl;
 try{
 insertDataUrl = await renderPdfPageToDataUrl(page, 980);
 }catch(pageErr){
 console.warn('Using thumbnail-sized PNG for PDF page', pageNumber, pageErr);
 }
 pages.push({ pageNumber, thumbDataUrl, insertDataUrl, selected: true });
 if(state.taskPdfImport){
 state.taskPdfImport.pages = [...pages];
 renderTaskPdfImportModal();
 }
}
if(state.taskPdfImport){
 state.taskPdfImport.loading = false;
 state.taskPdfImport.pages = pages;
 renderTaskPdfImportModal();
}
}
window.closeTaskPdfImport = function(){
 state.taskPdfImport = null;
 const modal = qs('taskPdfImportModal');
 if(modal) modal.classList.add('hidden');
};
window.toggleTaskPdfPage = function(pageIdx, checked){
 const session = state.taskPdfImport;
 if(!session?.pages?.[pageIdx]) return;
 session.pages[pageIdx].selected = !!checked;
 renderTaskPdfImportModal();
};
window.setAllTaskPdfPages = function(checked){
 const session = state.taskPdfImport;
 if(!session?.pages?.length) return;
 session.pages.forEach(page => { page.selected = !!checked; });
 renderTaskPdfImportModal();
};
window.confirmTaskPdfImport = async function(){
 const session = state.taskPdfImport;
 if(!session || session.inserting) return;
 const task = state.worksheet?.tasks?.[session.taskIdx];
 if(!task) return closeTaskPdfImport();
 const selectedItems = session.pages.filter(page => page.selected);
 if(!selectedItems.length) return showStatus('Choose at least one page to
import.', 'warn');

```

```

try{
 session.inserting = true;
 renderTaskPdfImportModal();
 const imported = selectedItems.map(page => {
 const sourceDataUrl = page.insertDataUrl || page.thumbDataUrl;
 return {
 source: 'pdf',
 mimeType: 'image/png',
 dataUrl: sourceDataUrl,
 originalDataUrl: sourceDataUrl,
 crop: { left:0, right:0, top:0, bottom:0 },
 name: `task-${session.taskIdx + 1}-pdf-page-${page.pageNumber}.png`,
 file_name: session.fileName,
 page_number: page.pageNumber
 };
 });
 task.pdf_images = imported;
 closeTaskPdfImport();
 renderEditor();
 updatePreview();
 showStatus(`${imported.length} PDF page${imported.length === 1 ? '' : 's'}`
inserted into task ${session.taskIdx + 1}.`, 'ok');
} catch(err){
 console.error(err);
 showStatus(err.message || 'Could not import those PDF pages.', 'err');
 if(state.taskPdfImport){
 state.taskPdfImport.inserting = false;
 renderTaskPdfImportModal();
 }
}
};
window.triggerTaskPdfUpload = function(idx){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return;
 const input = ensureTaskPdfUploadInput();
 input.dataset.taskIdx = String(idx);
 input.click();
};
window.removeTaskPdfImage = function(idx, pdfIdx){
 const task = state.worksheet?.tasks?.[idx];
 if(!task?.pdf_images?.[pdfIdx]) return;
 task.pdf_images.splice(pdfIdx, 1);
 renderEditor();
 updatePreview();
};
window.rotateTaskPdfImage = async function(idx, pdfIdx){
 const task = state.worksheet?.tasks?.[idx];
 const image = task?.pdf_images?.[pdfIdx];
 if(!image?.dataUrl) return;
 try{

```



```

 image.dataUrl = await rotateImageDataUrl(image.dataUrl, 90);
 if(image.originalDataUrl) image.originalDataUrl = image.dataUrl;
 image.crop = { left:0, right:0, top:0, bottom:0 };
 renderEditor();
 updatePreview();
 }catch(err){
 showStatus('Could not rotate that image.', 'err');
 }
};
window.rotateTaskPdfImportPage = async function(pageIdx){
 const session = state.taskPdfImport;
 const page = session?.pages?.[pageIdx];
 if(!page) return;
 try{
 page.thumbDataUrl = await rotateImageDataUrl(page.thumbDataUrl, 90);
 page.insertDataUrl = await rotateImageDataUrl(page.insertDataUrl, 90);
 renderTaskPdfImportModal();
 }catch(err){
 showStatus('Could not rotate that page.', 'err');
 }
};
window.clearTaskPdfImages = function(idx){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return;
 task.pdf_images = [];
 renderEditor();
 updatePreview();
 showStatus(`PDF pages removed from task ${idx + 1}.`, 'ok');
};
window.toggleTaskPdfCropPanel = function(taskIdx, pdfIdx){
 state.taskPdfCropPanels = state.taskPdfCropPanels || {};
 const key = taskPdfCropPanelKey(taskIdx, pdfIdx);
 state.taskPdfCropPanels[key] = !state.taskPdfCropPanels[key];
 renderEditor();
};
window.resetTaskPdfCrop = function(taskIdx, pdfIdx){
 const image = state.worksheet?.tasks?.[taskIdx]?.pdf_images?.[pdfIdx];
 if(!image) return;
 const baseDataUrl = image.originalDataUrl || image.dataUrl;
 image.originalDataUrl = baseDataUrl;
 image.crop = { left:0, right:0, top:0, bottom:0 };
 image.dataUrl = baseDataUrl;
 renderEditor();
 updatePreview();
 showStatus(`Crop reset for PDF page ${Math.max(1, parseInt(image.page_number, 10) || (pdfIdx + 1))}.`, 'ok');
};
window.updateTaskPdfCropValueLabel = function(taskIdx, pdfIdx, side, value){
 const el = qs(`taskPdfCropValue-${taskIdx}-${pdfIdx}-${side}`);
 if(el) el.textContent = `${parseFloat(value)}%`;
};

```

```

};
window.updateTaskPdfCrop = async function(taskIdx, pdfIdx, side, value){
 const image = state.worksheet?.tasks?.[taskIdx]?.pdf_images?.[pdfIdx];
 if(!image) return;
 const baseDataUrl = image.originalDataUrl || image.dataUrl;
 image.originalDataUrl = baseDataUrl;
 const nextCrop = enforceImageCropBounds({ ...(image.crop || {}), [side]:
parseCropPercent(value) }, side);
 image.crop = nextCrop;
 try{
 image.dataUrl = await cropImageDataUrl(baseDataUrl, nextCrop);
 renderEditor();
 updatePreview();
 }catch(err){
 console.error(err);
 showStatus(err.message || 'Could not crop that imported page.', 'err');
 }
};

async function pasteTaskImageFromClipboardItems(idx, items){
 if(!state.worksheet?.tasks?.[idx]) return 0;
 for(const item of [...items || []]){
 if(item.kind === 'file' && (item.type || '').startsWith('image/')){
 const file = item.getAsFile?();
 if(!file) continue;
 const dataUrl = await fileToDataUrl(file);
 if(setTaskImageFromDataUrl(idx, dataUrl, {
 source: 'clipboard',
 mimeType: file.type || 'image/png',
 name: file.name || `task-${idx + 1}-pasted-image.png`
 }))) return 1;
 }
 }
 return 0;
}

async function pasteTaskImageFromClipboardApi(idx, items){
 if(!state.worksheet?.tasks?.[idx]) return 0;
 for(const item of [...items || []]){
 const imageType = (item.types || []).find(type => type.startsWith('image/'));
 if(!imageType) continue;
 const blob = await item.getType(imageType);
 const dataUrl = await fileToDataUrl(new File([blob], `task-${idx +
1}-pasted-image.${imageType.split('/')[1] || 'png'}`, { type:imageType }));
 if(setTaskImageFromDataUrl(idx, dataUrl, {
 source: 'clipboard',
 mimeType: imageType,
 name: `task-${idx + 1}-pasted-image.${imageType.split('/')[1] || 'png'}`
 }))) return 1;
 }
 return 0;
}

```

```

}
async function callOpenAIImage(prompt, size){
 const apiKey = qs('imageApiKey')?.value.trim();
 const model = qs('imageModel')?.value.trim() || 'gpt-image-1.5';
 const finalSize = parseImageGenSize(size);
 if(!apiKey) throw new Error('Add your OpenAI image API key first.');
```

```

 const res = await fetch('https://api.openai.com/v1/images/generations', {
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 'Authorization': `Bearer ${apiKey}`
 },
 body: JSON.stringify({
 model,
 prompt,
 size: finalSize
 })
 });
 let data;
 try{ data = await res.json(); }
 catch(err){ throw new Error('Could not read the image API response.');
```

```

 if(!res.ok || data.error){
 throw new Error(data.error?.message || (`Image API error ${res.status}`));
 }
 const b64 = data?.data?.[0]?.b64_json;
 if(!b64) throw new Error('The image API did not return an image.');
```

```

 return {
 dataUrl: `data:image/png;base64,${b64}`,
 mimeType: 'image/png',
 revisedPrompt: data?.data?.[0]?.revised_prompt || ''
 };
}

```

```

window.toggleTaskImage = function(idx, checked){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return;
 task.image_enabled = !!checked;
 if(task.image_enabled && !task.image_prompt) task.image_prompt =
buildTaskImagePrompt(task, idx);
 renderEditor();
 updatePreview();
};
window.updateTaskImagePrompt = function(idx, value){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return;
 task.image_prompt = value;
};
window.updateTaskImageSize = function(idx, value){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return;

```

```

 task.image_size = parseImageGenSize(value);
 renderEditor();
 };
 window.updateTaskImageWidth = function(idx, value){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return;
 task.image_width = parseImageWidth(value);
 const valueEls = [
 document.getElementById(`taskImageWidthValue-${idx}`),
 document.getElementById(`taskPdfWidthValue-${idx}`)
].filter(Boolean);
 valueEls.forEach(el => { el.textContent = `${task.image_width}%`; });
 updatePreview();
 };
 window.suggestTaskImagePrompt = function(idx){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return;
 task.image_prompt = buildTaskImagePrompt(task, idx);
 renderEditor();
 };
 window.triggerTaskImageUpload = function(idx){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return;
 task.image_enabled = true;
 const input = ensureTaskImageUploadInput();
 input.dataset.taskIdx = String(idx);
 input.click();
 };
 window.handleTaskImagePasteEvent = async function(idx, event){
 if(!state.worksheet?.tasks?.[idx]) return;
 try{
 const count = await pasteTaskImageFromClipboardItems(idx,
event?.clipboardData?.items || []);
 if(count > 0){
 if(event?.preventDefault) event.preventDefault();
 renderEditor();
 updatePreview();
 showStatus(`Clipboard image added to task ${idx + 1}.`, 'ok');
 }else{
 showStatus('No image found in the clipboard. Copy the screenshot first, then
paste it here.', 'warn');
 }
 }catch(err){
 showStatus('Could not paste that image into the task.', 'err');
 }
 };
 window.pasteTaskImageFromClipboard = async function(idx){
 if(!state.worksheet?.tasks?.[idx]) return;
 try{
 if(navigator.clipboard && navigator.clipboard.read){

```

```

 const items = await navigator.clipboard.read();
 const count = await pasteTaskImageFromClipboardApi(idx, items);
 if(count > 0){
 renderEditor();
 updatePreview();
 showStatus(`Clipboard image added to task ${idx + 1}.`, 'ok');
 return;
 }
}
const zone = document.getElementById(`taskImagePasteZone-${idx}`);
zone?.focus();
showStatus('Click the paste box and press Ctrl/Cmd + V to paste your screenshot
into the task.', 'warn');
}catch(err){
 const zone = document.getElementById(`taskImagePasteZone-${idx}`);
 zone?.focus();
 showStatus('Clipboard access was blocked, so use Ctrl/Cmd + V in the paste box
instead.', 'warn');
}
};
window.removeTaskImage = function(idx){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return;
 task.image = null;
 renderEditor();
 updatePreview();
 showStatus(`Image removed from task ${idx + 1}.`, 'ok');
};
window.generateTaskImage = async function(idx){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return showStatus('No task to illustrate.', 'warn');
 if(state.imageJobs[idx]) return;
 setEditorAiBusy(true, `Generating image for task ${Number(idx) + 1}...`);
 try{
 task.image_enabled = true;
 const prompt = cleanText(task.image_prompt || buildTaskImagePrompt(task, idx));
 const size = parseImageGenSize(task.image_size || state.imageGenSize ||
DEFAULT_IMAGE_GEN_SIZE);
 task.image_prompt = prompt;
 task.image_size = size;
 state.imageJobs[idx] = true;
 renderEditor();
 showStatus(`Generating image for task ${idx + 1}...`, 'warn');
 const result = await callOpenAIImage(prompt, size);
 task.image = {
 source: 'ai',
 provider: 'openai',
 prompt,
 mimeType: result.mimeType || 'image/png',
 dataUrl: result.dataUrl,

```

```

 name: `task-${idx + 1}-image.png`,
 size
 };
 updatePreview();
 showStatus(`Image ready for task ${idx + 1}.`, 'ok');
} catch (err) {
 showStatus(err.message || 'Could not generate that image.', 'err');
} finally {
 delete state.imageJobs[idx];
 setEditorAiBusy(false);
 forceEditorRefresh();
}
};

window.generateAnswerKeyNow = async function() {
 if (!state.worksheet?.tasks?.length) return showStatus('Generate a worksheet first.', 'warn');
 setEditorAiBusy(true, 'Generating the answer key...');
 try {
 showStatus('Generating answer key...', 'warn');
 const prompt = buildAIPrompt({ mode: 'answer_key' });
 const result = await callAI({ prompt, modify: false });
 const answers = Array.isArray(result?.tasks) ? result.tasks : [];
 const byNumber = new Map();
 answers.forEach(item => {
 const num = parseInt(item?.number, 10);
 if (Number.isFinite(num)) byNumber.set(num, cleanText(item?.answer || ''));
 });
 state.worksheet.tasks.forEach((task, idx) => {
 const next = byNumber.get(task.number) || byNumber.get(idx + 1) || '';
 task.answer = cleanText(next || task.answer || '');
 });
 if (qs('outputMode')) qs('outputMode').value = 'both';
 renderEditor();
 updatePreview();
 savePrefs();
 showStatus('Answer key generated.', 'ok');
 } catch (err) {
 showStatus(err.message || 'Could not generate the answer key.', 'err');
 } finally {
 setEditorAiBusy(false);
 forceEditorRefresh();
 }
};

function sanitiseWorksheetFontSize(value) {
 const num = parseInt(value, 10);
 return ALLOWED_WORKSHEET_FONT_SIZES.includes(num) ? num :
 DEFAULT_WORKSHEET_FONT_SIZE;
}

```

```

}

function getWorksheetFontSize(){
 return sanitiseWorksheetFontSize(state.fontSize || readStoredPrefs().fontSize ||
 DEFAULT_WORKSHEET_FONT_SIZE);
}
window.getWorksheetFontSize = getWorksheetFontSize;

function worksheetFontVarsObject(size=getWorksheetFontSize()){
 const base = sanitiseWorksheetFontSize(size);
 return {
 '--ws-font': `${base}px`,
 '--ws-font-xs': `${Math.max(base - 2, 7)}px`,
 '--ws-font-sm': `${Math.max(base - 1, 8)}px`,
 '--ws-font-md': `${base}px`,
 '--ws-font-lg': `${base + 1}px`,
 '--ws-title-font': `${base + 13}px`
 };
}

function applyWorksheetFontCssVars(size=getWorksheetFontSize()){
 const vars = worksheetFontVarsObject(size);
 Object.entries(vars).forEach(([key, value]) =>
document.documentElement.style.setProperty(key, value));
}

function buildWorksheetFontCssVarBlock(size=getWorksheetFontSize()){
 return Object.entries(worksheetFontVarsObject(size)).map(([key, value]) =>
`${key}:${value};`).join(' ');
}
window.buildWorksheetFontCssVarBlock = buildWorksheetFontCssVarBlock;

window.setWorksheetFontSize = function(value){
 state.fontSize = sanitiseWorksheetFontSize(value);
 const select = qs('worksheetFontSize');
 if(select) select.value = String(state.fontSize);
 applyWorksheetFontCssVars(state.fontSize);
 savePrefs();
 updatePreview();
};

function ensureWorksheetFontSizeUi(){
 const row = document.querySelector('.workspace-top .pill-row');
 if(!row || qs('worksheetFontSize')) return;
 const control = document.createElement('label');
 control.className = 'font-size-control';
 control.innerHTML = `
 Font size
 <select id="worksheetFontSize" aria-label="Worksheet font size">
 ${ALLOWED_WORKSHEET_FONT_SIZES.map(size => `<option value="${size}">${size}`

```

```

pt</option>`).join('')
 </select>`;
 row.insertBefore(control, row.firstChild);
 const select = control.querySelector('select');
 select.value = String(getWorksheetFontSize());
 select.addEventListener('change', (event) =>
setWorksheetFontSize(event.target.value));
}

ensureImageUsageUi();
ensureImageGenerationUi();

loadPrefs = function(){
 const prefs = readStoredPrefs();
 applyStudioCycle(prefs.preferredCycle || 'cycle3', prefs.yearLevel || '');
 if(qs('apiKey')) qs('apiKey').value = prefs.apiKey || '';
 if(qs('modelName')) qs('modelName').value = prefs.modelName || '';
 if(qs('outputMode')) qs('outputMode').value = prefs.outputMode || 'student';
 if(qs('difficulty')) qs('difficulty').value =
normaliseDifficulty(prefs.difficulty || 'cm1_moderate');
 if(qs('questionCount')) qs('questionCount').value = prefs.questionCount || '6';
 if(qs('answerGenMode')) qs('answerGenMode').value = prefs.answerGenMode ||
'student_only';
 state.answerGenMode = prefs.answerGenMode || 'student_only';
 if(qs('worksheetBrief')) qs('worksheetBrief').value = prefs.worksheetBrief || '';
 if(qs('templateText')) qs('templateText').value = prefs.templateText || '';
 if(qs('imageApiKey')) qs('imageApiKey').value = prefs.imageApiKey || '';
 if(qs('imageModel')) qs('imageModel').value = prefs.imageModel ||
'gpt-image-1.5';
 setImageGenSize(prefs.imageGenSize || DEFAULT_IMAGE_GEN_SIZE);
 state.curriculumMode = prefs.curriculumMode || (prefs.useAICurriculum !==
undefined ? (prefs.useAICurriculum ? 'ai_both' : 'manual_both') : 'ai_both');
 state.displayOptions = {
 instructions: prefs.showInstructions !== false,
 successCriteria: prefs.showSuccessCriteria !== false,
 extension: prefs.showExtension !== false,
 learningObjectives: prefs.showLearningObjectives === true
 };
 state.fontSize = sanitiseWorksheetFontSize(prefs.fontSize ||
DEFAULT_WORKSHEET_FONT_SIZE);
 applyWorksheetFontCssVars(state.fontSize);
 setImageUsage(prefs.imageUsage || DEFAULT_IMAGE_USAGE);
 syncToggles();
};

savePrefs = function(){
 writeStoredPrefs(buildPrefsObject());
};

```



```

updateDisplayOption = function(key, checked){
 if(!state.displayOptions) state.displayOptions = { instructions:false,
successCriteria:false, extension:false, learningObjectives:false };
 state.displayOptions[key] = !!checked;
 savePrefs();
 renderEditor();
 updatePreview();
}

normaliseWorksheet = function(result){
 const curriculumMode = state.curriculumMode || 'ai_both';
 const selectedDomain = canonicaliseDomain(qs('domainSelect')?.value.trim() ||
'');
 const evidenceText = buildCurriculumEvidenceText(result);

 const fallbackDomain = defaultStudioDomain();
 let resolvedDomain = curriculumMode === 'ai_both'
 ? (canonicaliseDomain(result?.domain) || inferDomainFromText(evidenceText) ||
fallbackDomain)
 : (selectedDomain || fallbackDomain);

 let learningOutcomes = [];
 if(curriculumMode === 'manual_both'){
 learningOutcomes = uniqueList((state.selectedOutcomes || []).map(outcome =>
canonicaliseOutcome(outcome, resolvedDomain)).filter(Boolean));
 }else{
 const rawOutcomes = Array.isArray(result?.learning_outcomes) ?
result.learning_outcomes : [];
 learningOutcomes = chooseAllowedOutcomes(resolvedDomain, evidenceText, 3,
rawOutcomes);
 }

 if(!learningOutcomes.length){
 learningOutcomes = chooseAllowedOutcomes(resolvedDomain, evidenceText, 3,
state.selectedOutcomes || []);
 }
 if(!learningOutcomes.length){
 learningOutcomes = getFlatDomainOutcomes(resolvedDomain).slice(0, 3);
 }

 const worksheet = {
 domain: resolvedDomain,
 learning_outcomes: learningOutcomes,
 title: cleanText(result.title) || 'Untitled worksheet',
 instructions: cleanText(result.instructions),
 success_criteria: cleanText(result.success_criteria),
 intro_text: '',
 image_note: cleanText(result.image_note),
 worksheet_images: resolveWorksheetImages(result?.worksheet_images,
state.worksheet?.worksheet_images || []),
 }

```

```

 tasks: Array.isArray(result.tasks) ? result.tasks.slice(0,12).map((task,
idx)=>normaliseTask(task, idx)) : [],
 extension: cleanText(result.extension)
 };

```

```

 if(!worksheet.tasks.length){
 worksheet.tasks = [{ number:1, passage:'', prompt:'Task text goes here.',
options:[], answer_lines:0, answer:'', image_enabled:false, image_prompt:'',
image_size: state.imageGenSize || DEFAULT_IMAGE_GEN_SIZE, image_width:100,
image:null }]];
 }

```

```

 worksheet.tasks.forEach(task => { task.answer_lines = 0; });

```

```

 if(qs('domainSelect')) qs('domainSelect').value = worksheet.domain;
 state.selectedOutcomes = [...worksheet.learning_outcomes];
 renderOutcomeOptions();

```

```

 state.loStatus = {};
 worksheet.learning_outcomes.forEach(lo => { state.loStatus[lo] =
state.loStatus[lo] || ''; });
 return worksheet;
};

```

```

function buildWorksheetHeaderImagePrompt(){
 const title = cleanText(state.worksheet?.title || 'Worksheet activity');
 const level = studioAgeGroupText();
 return [
 'Create a clean, child-friendly educational illustration for a primary school
worksheet header.',
 `Worksheet title: ${title}.`,
 `Age group: ${level}.`,
 'The image should support the overall worksheet topic and look clear when
printed on an A4 worksheet.',
 'No labels, no captions, no speech bubbles, no worksheet borders, no
watermarks, no visible answer text.'
].filter(Boolean).join(' ');
}

```

```

function taskAssetSummaryHtml(task, idx){
 const hasImage = !!task?.image?.dataUrl;
 const pdfCount = Array.isArray(task?.pdf_images) ? task.pdf_images.length : 0;
 if(!hasImage && !pdfCount){
 return `
```

```

attached`);
return `
 <div class="task-card" style="padding:10px 12px;margin-bottom:12px">
 <div class="task-head" style="margin-bottom:6px">
 Attached assets
 <button class="mini secondary" type="button"
onclick="toggleEditorAccordion('editor_assets');"
setEditorAssetDestination('task_${idx}')">Open Assets</button>
 </div>
 <div class="field-note">${bits.join(' · ')}> Manage visuals, AI image
generation and PDF pages from the Assets section below instead of stuffing this
task card full of buttons.</div>
 </div>`;
}
function taskAssetManagerCardHtml(task, idx){
 const hasImage = !!task?.image?.dataUrl;
 const pdfImages = Array.isArray(task?.pdf_images) ? task.pdf_images : [];
 if(!hasImage && !pdfImages.length) return '';
 const imageBusy = !!state.imageJobs?.[idx];
 const imageSize = parseImageGenSize(task.image_size || state.imageGenSize ||
DEFAULT_IMAGE_GEN_SIZE);
 const imagePreview = hasImage ? `
 <div class="task-image-preview">

 <div class="task-image-meta">${taskImageSourceLabel(task.image, imageSize)}
Display width: ${task.image_width || 100}%</div>
 </div>` : '';
 const pdfGallery = pdfImages.length ? `
 <div class="task-pdf-gallery">
 ${pdfImages.map((image, pdfIdx)=>{
 const crop = enforceImageCropBounds(image.crop || {});
 const cropOpen = isTaskPdfCropPanelOpen(idx, pdfIdx);
 const hasCrop = !(crop.left || crop.right || crop.top || crop.bottom);
 return `
 <div class="task-pdf-card">
 <div class="task-pdf-card-head">
 <div class="task-pdf-card-title">PDF page ${Math.max(1,
parseInt(image.page_number, 10) || (pdfIdx + 1))}</div>
 <div class="pill-row">
 <button class="mini secondary" type="button"
onclick="rotateTaskPdfImage(${idx}, ${pdfIdx})">Rotate</button>
 <button class="mini secondary" type="button"
onclick="toggleTaskPdfCropPanel(${idx}, ${pdfIdx})">${cropOpen ? 'Hide crop' :
'Crop'}</button>
 <button class="mini danger" type="button"
onclick="removeTaskPdfImage(${idx}, ${pdfIdx})">Remove</button>
 </div>

 <div class="task-pdf-meta">${taskPdfImageSourceLabel(image)} Display
width: ${task.image_width || 100}%${hasCrop ? ' Cropped version shown below and on
the worksheet.' : ''}</div>
 ${cropOpen ? `
 <div class="task-pdf-crop-panel">
 <div class="task-pdf-crop-grid">
 ${['left', 'right', 'top', 'bottom'].map(side=>`
 <div class="field">
 <label>${side.charAt(0).toUpperCase() + side.slice(1)} crop
<span class="tiny"
id="taskPdfCropValue-${idx}-${pdfIdx}-${side}">${crop[side]}%</label>
 <input type="range" min="0" max="40" step="1"
value="${crop[side]}" oninput="updateTaskPdfCropValueLabel(${idx}, ${pdfIdx},
'${side}', this.value)" onchange="updateTaskPdfCrop(${idx}, ${pdfIdx}, '${side}',
this.value)">
 </div>`
).join('')}
 </div>
 <div class="task-pdf-crop-note">Crop trims the imported PNG from each
edge after insertion. Release the slider to apply it.</div>
 <div class="task-pdf-crop-actions">
 <button class="small-btn" type="button"
onclick="resetTaskPdfCrop(${idx}, ${pdfIdx})">Reset crop</button>
 </div>
 </div>` : ''}
 </div>`;
 }).join('')}
 </div>` : '';
 return `
 <div class="task-card" style="padding:12px">
 <div class="task-head" style="margin-bottom:8px">
 Task ${idx + 1} assets
 <div class="pill-row">
 <button class="small-btn" type="button"
onclick="setEditorAssetDestination('task_${idx}')">Target in add controls</button>
 ${hasImage ? `<button class="small-btn ghost-danger" type="button"
onclick="removeTaskImage(${idx})" ${imageBusy ? 'disabled' : ''}>Remove
image</button>` : ''}
 ${pdfImages.length ? `<button class="small-btn ghost-danger"
type="button" onclick="clearTaskPdfImages(${idx})">Clear PDF pages</button>` : ''}
 </div>
 </div>
 <div class="field-note" style="margin-bottom:10px">Use the add controls above
to upload, paste, generate or import new assets straight into this task without
opening the task card itself.</div>
 ${((hasImage || pdfImages.length) ? `
 <div class="field" style="margin-bottom:10px">
 <label>Display size in worksheet <span id="taskAssetWidthValue-${idx}"
class="tiny">${task.image_width || 100}%</label>

```

```

 <input type="range" min="30" max="100" step="5" value="${task.image_width
|| 100}" oninput="updateTaskImageWidth(${idx}, this.value);
syncTaskAssetWidthLabels(${idx}, this.value)">
 </div>` : ''}
 ${hasImage ? `
 <div class="field" style="margin-bottom:12px">
 <label>Task image</label>
 ${imagePreview}
 </div>` : ''}
 ${pdfImages.length ? `
 <div class="field">
 <label>Imported PDF pages</label>
 ${pdfGallery}
 </div>` : ''}
</div>`;
}

```

```

taskEditorHtml = function(task, idx){
 const imagePrompt = esc(task.image_prompt || buildTaskImagePrompt(task, idx));
 const imageSize = parseImageGenSize(task.image_size || state.imageGenSize ||
DEFAULT_IMAGE_GEN_SIZE);
 const imageBusy = !!state.imageJobs?.[idx];
 const pdfImages = Array.isArray(task.pdf_images) ? task.pdf_images : [];
 const imagePreview = task.image?.dataUrl ? `
 <div class="task-image-preview">

 <div class="task-image-meta">${taskImageSourceLabel(task.image, imageSize)}
Display width: ${task.image_width || 100}%</div>
 </div>` : '';
 const pdfGallery = pdfImages.length ? `
 <div class="task-pdf-gallery">
 ${pdfImages.map((image, pdfIdx)=>{
 const crop = enforceImageCropBounds(image.crop || {});
 const cropOpen = isTaskPdfCropPanelOpen(idx, pdfIdx);
 const hasCrop = !(crop.left || crop.right || crop.top || crop.bottom);
 return `
 <div class="task-pdf-card">
 <div class="task-pdf-card-head">
 <div class="task-pdf-card-title">PDF page ${Math.max(1,
parseInt(image.page_number, 10) || (pdfIdx + 1))}</div>
 <div class="pill-row">
 <button class="mini secondary" type="button"
onclick="rotateTaskPdfImage(${idx}, ${pdfIdx})">Rotate</button>
 <button class="mini secondary" type="button"
onclick="toggleTaskPdfCropPanel(${idx}, ${pdfIdx})">${cropOpen ? 'Hide crop' :
'Crop'}</button>
 <button class="mini danger" type="button"
onclick="removeTaskPdfImage(${idx}, ${pdfIdx})">Remove</button>

```

```

 </div>
 </div>

 <div class="task-pdf-meta">${taskPdfImageSourceLabel(image)} Display
 width: ${task.image_width || 100}%.${(hasCrop ? ' Cropped version shown below and on
 the worksheet.' : '')}</div>
 ${cropOpen ? `
 <div class="task-pdf-crop-panel">
 <div class="task-pdf-crop-grid">
 ${[['left', 'right', 'top', 'bottom']].map(side=>`
 <div class="field">
 <label>${side.charAt(0).toUpperCase() + side.slice(1)} crop
 <span class="tiny"
 id="taskPdfCropValue-${idx}-${pdfIdx}-${side}">${crop[side]}%</label>
 <input type="range" min="0" max="40" step="1"
 value="${crop[side]}" oninput="updateTaskPdfCropValueLabel(${idx}, ${pdfIdx},
 '${side}', this.value)" onchange="updateTaskPdfCrop(${idx}, ${pdfIdx}, '${side}',
 this.value)">
 </div>`
).join('')}
 </div>
 <div class="task-pdf-crop-note">Crop trims the imported PNG from each
 edge after insertion. Release the slider to apply it. No drama, no re-upload.</div>
 <div class="task-pdf-crop-actions">
 <button class="small-btn" type="button"
 onclick="resetTaskPdfCrop(${idx}, ${pdfIdx})">Reset crop</button>
 </div>
 </div>` : ''}
 </div>`;
 }).join('')}
 </div>` : '';
 const snippetSource = accordionPlainText(task.prompt || task.passage ||
 task.answer || '');
 const actions = `
 <div class="task-actions">
 <button class="mini secondary" onclick="event.stopPropagation();
 toggleEditorAccordion('task_${idx}')">Toggle</button>
 <button class="mini" onclick="event.stopPropagation(); moveTask(${idx}, -
 1)">↑</button>
 <button class="mini" onclick="event.stopPropagation(); moveTask(${idx},
 1)">↓</button>
 <button class="mini secondary" onclick="event.stopPropagation();
 duplicateTask(${idx})">Duplicate</button>
 <button class="mini secondary" onclick="event.stopPropagation();
 regenerateTask(${idx})">Regenerate</button>
 <button class="mini danger" onclick="event.stopPropagation();
 removeTask(${idx})">Delete</button>
 </div>`;

```

```

const stimulusBody = `
 <div class="field">
 <label>Passage / stimulus / word bank</label>
 ${richEditorHtml({
 id: `task_${idx}_passage`,
 value: task.passage || '',
 placeholder: 'Optional separate passage, sentence set, word bank or
stimulus text.',
 note: 'Shortcuts work here: Ctrl/Cmd+B, I and U.',
 dataset: { richTarget:'taskField', idx, field:'passage' },
 minHeight: 100
 })}
 </div>`;

const promptBody = `
 <div class="field">
 <label>Prompt</label>
 ${richEditorHtml({
 id: `task_${idx}_prompt`,
 value: task.prompt || '',
 placeholder: 'Write the task prompt here.',
 dataset: { richTarget:'taskField', idx, field:'prompt' },
 minHeight: 110
 })}
 </div>
 <div class="field">
 <label>Options (one per line)</label>
 <textarea style="min-height:96px" oninput="updateTaskOptions(${idx},
this.value)">${esc((task.options||[]).join('\n'))}</textarea>
 </div>`;

const answerLinesBody = `
 <div class="field">
 <label>Answer lines</label>
 <input type="number" min="0" max="40" step="1"
value="${esc(task.answer_lines)}" oninput="updateTaskLines(${idx}, this.value)">
 <div class="field-note">Starts at 0. Click to add lines.</div>
 </div>`;

const importPdfBody = `
 <div class="field task-pdf-tools">
 <label>Existing PDF pages</label>
 <div class="pill-row">
 <button class="small-btn" type="button"
onclick="triggerTaskPdfUpload(${idx})">${pdfImages.length ? 'Replace PDF pages' :
'Import PDF pages'}</button>
 ${pdfImages.length ? `<button class="small-btn ghost-danger" type="button"
onclick="clearTaskPdfImages(${idx})">Clear PDF pages</button>` : ''}
 </div>
 <div class="task-pdf-inline-note">Upload a PDF, tick the pages you want, and

```

this task will insert those pages as PNGs. Handy for dropping existing worksheets straight into one task without the app trying to get clever.</div>

```
 ${pdfImages.length ? `
 <div class="field" style="margin-top:10px">
 <label>Display size in worksheet <span id="taskPdfWidthValue-${idx}"
class="tiny">${task.image_width || 100}%</label>
 <input type="range" min="30" max="100" step="5" value="${task.image_width
|| 100}" oninput="updateTaskImageWidth(${idx}, this.value)">
 <div class="field-note">This changes how large the imported PDF pages
appear on the worksheet.</div>
 </div>` : ''}
 ${pdfGallery}
 </div>`;
```

```
const importImageBody = `
 <div class="field">
 <div class="pill-row" style="margin-bottom:8px">
 <button class="small-btn" type="button"
onclick="triggerTaskImageUpload(${idx})" ${imageBusy ? 'disabled' :
''}>${task.image?.dataUrl ? 'Replace with upload' : 'Upload image'}</button>
 <button class="small-btn" type="button"
onclick="pasteTaskImageFromClipboard(${idx})" ${imageBusy ? 'disabled' :
''}>${task.image?.dataUrl ? 'Replace with paste' : 'Paste image'}</button>
 ${task.image?.dataUrl ? `<button class="small-btn ghost-danger"
type="button" onclick="removeTaskImage(${idx})" ${imageBusy ? 'disabled' :
''}>Remove image</button>` : ''}
 </div>
 ${task.image?.dataUrl ? `
 <div class="field">
 <label>Display size in worksheet <span id="taskImageWidthValue-${idx}"
class="tiny">${task.image_width || 100}%</label>
 <input type="range" min="30" max="100" step="5" value="${task.image_width
|| 100}" oninput="updateTaskImageWidth(${idx}, this.value)">
 </div>
 ${imagePreview}` : ''}
 <div aria-label="Paste image into task ${idx + 1}" class="paste-zone"
id="taskImagePasteZone-${idx}" tabindex="0"
onpaste="handleTaskImagePasteEvent(${idx}, event)">
 Paste screenshot here
 Click here, then Ctrl/Cmd + V.
 </div>
 ${editorAccordionHtml({ key:`task_${idx}_ai_image`, title:'AI image
generation', subtitle:'Generate an illustration using OpenAI.', showSubtitle:true,
body:`
 <div class="field">
 <label>Image prompt</label>
 <textarea oninput="updateTaskImagePrompt(${idx}, this.value)"
placeholder="Describe the image you want for this task.">${imagePrompt}</textarea>
 <div class="task-editor-grid" style="margin-top:10px">
 <div class="field">
```



```

 <label>Generated image shape</label>
 <select onchange="updateTaskImageSize(${idx}, this.value)">
 <option value="1024x1024" ${imageSize === '1024x1024' ? 'selected'
: ''}>Square</option>
 <option value="1024x1536" ${imageSize === '1024x1536' ? 'selected'
: ''}>Portrait</option>
 <option value="1536x1024" ${imageSize === '1536x1024' ? 'selected'
: ''}>Landscape</option>
 </select>
 </div>
 <div class="field">
 <label>Display size <span id="taskAiImageWidthValue-${idx}"
class="tiny">${task.image_width || 100}%</label>
 <input type="range" min="30" max="100" step="5"
value="${task.image_width || 100}" oninput="updateTaskImageWidth(${idx},
this.value)">
 </div>
</div>
<div class="pill-row" style="margin-top:8px">
 <button class="small-btn" type="button"
onclick="suggestTaskImagePrompt(${idx})" ${imageBusy ? 'disabled' : ''}>Suggest
prompt</button>
 <button class="small-btn primary" type="button"
onclick="generateTaskImage(${idx})" ${imageBusy ? 'disabled' : ''}>${imageBusy ?
'Generating...' : (task.image?.dataUrl ? 'Regenerate' : 'Generate')}</button>
</div>
 ${imageBusy ? '<div class="task-image-status">Generating image...</div>' :
''}
</div>`, defaultOpen:false, extraClass:'task-inner-accordion')}}
</div>`;

```

```
const answerKeyBody = `
```

```

 <div class="field">
 <label>Answer / marking point</label>
 ${richEditorHtml({
 id: `task_${idx}_answer`,
 value: task.answer || '',
 placeholder: 'Teacher answer or marking point.',
 dataset: { richTarget:'taskField', idx, field:'answer' },
 minHeight: 96
 })}
 </div>`;

```

```

const nestedSections = [
 taskAssetSummaryHtml(task, idx),
 editorAccordionHtml({ key:`task_${idx}_stimulus`, title:'Stimulus',
subtitle:accordionSnippet(task.passage, 'Optional passage, word bank or source
text.'), showSubtitle:true, body:stimulusBody, defaultOpen:false,
extraClass:'task-inner-accordion' }),
 editorAccordionHtml({ key:`task_${idx}_prompt_section`, title:'Prompt',

```

```

subtitle:accordionSnippet(task.prompt || (task.options || []).join(' '), 'Main
question and any options.'), showSubtitle:true, body:promptBody, defaultOpen:true,
extraClass:'task-inner-accordion' }),
 editorAccordionHtml({ key:`task_${idx}_answer_lines`, title:'Answer Lines',
subtitle:(task.answer_lines > 0 ? `${task.answer_lines} line${task.answer_lines ===
1 ? '' : 's'} currently set.` : 'No writing lines yet.'), showSubtitle:true,
body:answerLinesBody, defaultOpen:false, extraClass:'task-inner-accordion' }),
 editorAccordionHtml({ key:`task_${idx}_answer_key`, title:'Answer Key',
subtitle:accordionSnippet(task.answer, 'Teacher answer or marking guidance.'),
showSubtitle:true, body:answerKeyBody, defaultOpen:false,
extraClass:'task-inner-accordion' })
].join('');

```

```

return editorAccordionHtml({
 key: `task_${idx}`,
 title: `Task ${idx + 1}`,
 subtitle: accordionSnippet(snippetSource, 'Collapsed to save your scrolling
finger.'),
 showSubtitle: true,
 body: nestedSections,
 actions,
 defaultOpen: idx === 0,
 extraClass: 'task-accordion'
});
};

```

```

addTask = function(){
 if(!state.worksheet){
 state.worksheet = {
 domain: canonicaliseDomain(qs('domainSelect').value || '') ||
defaultStudioDomain(),
 learning_outcomes: state.selectedOutcomes.length ?
[...state.selectedOutcomes] : ['Learning outcome to be confirmed'],
 title: 'Untitled worksheet',
 instructions: '',
 success_criteria: '',
 intro_text: '',
 image_note: '',
 tasks: [],
 extension: ''
 };
 }
 state.worksheet.tasks.push({
 number: state.worksheet.tasks.length + 1,
 passage: '',
 prompt: '',
 options: [],
 answer_lines: 0,
 answer: '',

```

```

 image_enabled: false,
 image_prompt: '',
 image_size: state.imageGenSize || DEFAULT_IMAGE_GEN_SIZE,
 image_width: 100,
 image: null,
 pdf_images: []
 });
 renderEditor();
 updatePreview();
};

window.duplicateTask = function(idx){
 if(!state.worksheet?.tasks[idx]) return;
 const source = state.worksheet.tasks[idx];
 const clone = JSON.parse(JSON.stringify(source));
 state.worksheet.tasks.splice(idx + 1, 0, clone);
 state.worksheet.tasks.forEach((t,i)=>t.number=i+1);
 renderEditor();
 updatePreview();
 showStatus(`Task ${idx + 1} duplicated.`, 'ok');
};

window.regenerateTask = async function(idx){
 if(!state.worksheet?.tasks[idx]) return showStatus('No task to regenerate.',
'warn');
 setEditorAiBusy(true, `Regenerating task ${Number(idx) + 1}...`);
 try{
 showStatus(`Regenerating task ${idx + 1}...`, 'warn');
 const prompt = buildAIPrompt({ mode: 'regenerate_task', taskId: idx });
 const result = await callAI({prompt, modify:false, extraImages:
collectTaskImageAttachments([idx], 6)});
 const previousTask = state.worksheet.tasks[idx];
 const nextTask = normaliseTask(result, idx, previousTask);
 if(!hasAnswerKeyData() && !shouldGenerateAnswers()) nextTask.answer = '';
 nextTask.answer_lines = 0;
 if(previousTask?.image_enabled){
 nextTask.image_enabled = true;
 nextTask.image_prompt = previousTask.image_prompt ||
buildTaskImagePrompt(nextTask, idx);
 nextTask.image = null;
 }
 state.worksheet.tasks[idx] = nextTask;
 pushHistorySnapshot(`Regenerated task ${Number(idx) + 1}`);
 renderEditor();
 updatePreview();
 showStatus(`Task ${idx + 1} regenerated.`, 'ok');
 }catch(err){
 showStatus(err.message || 'Could not regenerate that task.', 'err');
 }finally{
 setEditorAiBusy(false);
 }
};

```

```

 forceEditorRefresh();
 }
};

function variationInstruction(mode, extraTopic='') {
 switch(mode) {
 case 'easier': return 'Create an easier version. Keep the same topic, overall structure, domain and learning outcomes wherever sensible, but reduce complexity, increase scaffolding and make wording clearer.';
 case 'harder': return 'Create a harder version. Keep the same topic, overall structure, domain and learning outcomes wherever sensible, but increase depth, challenge and independence.';
 case 'different_questions': return hasAnswerKeyData() || shouldGenerateAnswers() ? 'Keep the same topic, domain, learning outcomes and overall worksheet identity, but replace the tasks with fresh questions and fresh answers.' : 'Keep the same topic, domain, learning outcomes and overall worksheet identity, but replace the tasks with fresh questions only. Leave teacher answers blank for now.';
 case 'same_lo_new_topic': return `Keep the domain and learning outcomes aligned to the current worksheet, but rebuild the worksheet around this new topic exactly: "${extraTopic}".`;
 case 'homework': return 'Turn this into a homework version. Make it self-contained, crystal clear, and suitable for completion without live teacher explanation.';
 case 'classwork': return 'Turn this into a classwork version. Keep it printable, but make it feel more suitable for guided classroom completion and teacher-supported discussion.';
 default: return 'Create a sensible fresh variation of this worksheet while keeping it printable and aligned.';
 }
}

window.generateVariation = async function(mode) {
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 let extraTopic = '';
 if(mode === 'same_lo_new_topic') {
 extraTopic = prompt('New topic for this variation:');
 if(extraTopic === null) return;
 if(!String(extraTopic).trim()) return showStatus('Add a new topic for that variation.', 'warn');
 }
 const variationLabels = { easier: 'Creating an easier variation...', harder: 'Creating a harder variation...', different_questions: 'Creating a fresh question set...', same_lo_new_topic: 'Creating a same-LO variation with a new topic...', homework: 'Creating a homework version...', classwork: 'Creating a classwork version...' };
 setEditorAiBusy(true, variationLabels[mode] || 'Creating a worksheet variation...');
 try {
 showStatus('Generating variation...', 'warn');
 }

```

```

 const prompt = buildAIPrompt({ mode: 'variation', variationMode: mode,
extraTopic: String(extraTopic).trim() });
 const hadAnswersBefore = hasAnswerKeyData();
 const result = await callAI({prompt, modify:false});
 state.worksheet = normaliseWorksheet(result);
 if(!hadAnswersBefore && !shouldGenerateAnswers())
stripTaskAnswers(state.worksheet);
 state.worksheet.instructions = '';
 state.worksheet.success_criteria = '';
 state.worksheet.extension = '';
 pushHistorySnapshot('Variation: ' + String(mode || 'updated'));
 renderEditor();
 updatePreview();
 showStatus('Variation ready.', 'ok');
 }catch(err){
 showStatus(err.message || 'Could not generate that variation.', 'err');
 }finally{
 setEditorAiBusy(false);
 forceEditorRefresh();
 }
};

renderTaskPrint = function(task){
 const imageWidth = parseImageWidth(task.image_width ?? 100);
 const aiImage = task.image_enabled && task.image?.dataUrl ? `<div
class="task-image-wrap"></div>` : '';
 const pdfImages = Array.isArray(task.pdf_images) ? task.pdf_images.map((image,
pdfIdx)=>`<div class="task-image-wrap"></div>`).join('') : '';
 const image = aiImage + pdfImages;
 const passage = task.passage ? `<div class="task-passage
rich-render">${renderRichText(task.passage)}</div>` : '';
 const prompt = String(task.prompt || '').trim() ? `<span class="task-prompt
rich-render">${renderRichText(task.prompt)}` : '';
 const options = task.options?.length ? `
 <ol class="mcq-list" type="A">
 ${task.options.map(opt=>`<li>${esc(opt)}</li>`).join('')}
 ` : '';
 const lines = task.answer_lines > 0 ? `
 <div class="answer-lines">
 ${Array.from({length: task.answer_lines}).map(()==>'<div
class="answer-line"></div>').join('')}
 </div>` : '';

 const writtenTask = `
 <div class="task-print ${image ? 'task-print-after-media' : ''}">

```

```

 ${passage}
 <div>${task.number}${prompt}</div>
 ${options}
 ${lines}
 </div>
`;

if(!image) return writtenTask;

return `
 <div class="task-media-frame">
 ${image}
 </div>
 ${writtenTask}
`;
};

renderAnswerItems = function(tasks){
 return tasks.map(task=>`
 <div class="answer-item">
 ${task.number}
 <div class="answer-question rich-render">${task.prompt ?
renderRichText(task.prompt) : 'Task'}</div>
 ${task.passage ? `<div class="rich-render answer-stimulus" style="margin:0 0
6px; color:#6a84a7">Stimulus:
${renderRichText(task.passage)}</div>` : ''}
 <div class="rich-render">${renderRichText(task.answer || 'No answer
supplied.')}</div>
 </div>
 `).join('');
};

function makeStorageSafePayload(payload){
 let safe;
 try{
 safe = JSON.parse(JSON.stringify(payload || {}));
 }catch(err){
 safe = {};
 }

 /* Browser localStorage is tiny. Base64 images/PDF pages will blow it up.
 Keep history/template snapshots text-first and strip heavy embedded assets. */
 safe.images = [];
 safe.promptImages = [];
 safe.promptDocuments = [];
 safe.templatePdf = null;

 if(safe.worksheet){
 safe.worksheet.worksheet_images = [];
 }

```

```

 if(Array.isArray(safe.worksheet.tasks)){
 safe.worksheet.tasks.forEach(task => {
 task.image = null;
 task.images = [];
 task.pdf_images = [];
 if(task.image_enabled) task.image_enabled = false;
 });
 }
}
return safe;
}

function makeStorageSafeItems(items){
 return (items || []).map(item => ({
 ...item,
 payload: makeStorageSafePayload(item.payload)
 }));
}

function safeSetLocalStorage(key, value, fallbackValue=null){
 try{
 localStorage.setItem(key, value);
 return true;
 }catch(err){
 console.warn('Local storage write failed:', err);
 try{
 localStorage.removeItem(key);
 if(fallbackValue !== null) localStorage.setItem(key, fallbackValue);
 return fallbackValue !== null;
 }catch(err2){
 console.warn('Fallback local storage write failed:', err2);
 return false;
 }
 }
}

function getHistoryItems(){
 try{ return JSON.parse(localStorage.getItem(LS_KEYS.history) || '[]'); }
 catch(err){ return []; }
}

function setHistoryItems(items){
 const safeItems = makeStorageSafeItems((items || []).slice(0, 12));
 const full = JSON.stringify(safeItems);
 if(safeSetLocalStorage(LS_KEYS.history, full)){
 if(window.SylCycle) SylCycle.cloudSave(LS_KEYS.history, safeItems);
 return true;
 }

 const smaller = JSON.stringify(safeItems.slice(0, 4));
 if(safeSetLocalStorage(LS_KEYS.history, smaller)){

```

```

 if(window.SylCycle) SylCycle.cloudSave(LS_KEYS.history, safeItems.slice(0, 4));
 return true;
}

localStorage.removeItem(LS_KEYS.history);
return false;
}
function buildHistorySignature(snapshot){
 const worksheet = snapshot?.worksheet || {};
 const tasks = Array.isArray(worksheet.tasks) ? worksheet.tasks.map(task => ({
 prompt: task.prompt || '',
 passage: task.passage || '',
 lines: task.answer_lines || 0,
 options: Array.isArray(task.options) ? task.options : []
 })) : [];
 return JSON.stringify({
 preferredCycle: snapshot?.preferredCycle || snapshot?.fields?.preferredCycle ||
'cycle3',
 yearLevel: snapshot?.yearLevel || snapshot?.fields?.yearLevel || '',
 title: worksheet.title || '',
 domain: worksheet.domain || '',
 outcomes: worksheet.learning_outcomes || [],
 tasks,
 sections: {
 instructions: worksheet.instructions || '',
 success_criteria: worksheet.success_criteria || '',
 extension: worksheet.extension || ''
 },
 displayOptions: snapshot?.displayOptions || {}
 });
}

function getSnapshotCycle(saved){
 const f = saved?.fields || {};
 return normaliseCycle(saved?.preferredCycle || saved?.cycle || f.preferredCycle
|| f.cycle || 'cycle3');
}

function getSnapshotYearLevel(saved, cycle){
 const f = saved?.fields || {};
 const years = getCycleYears(cycle);
 const preferred = saved?.yearLevel || f.yearLevel || '';
 return years.includes(preferred) ? preferred : years[0];
}

function pushHistorySnapshot(label='Snapshot'){
 if(!state.worksheet) return false;
 try{
 const payload = snapshotState();
 const signature = buildHistorySignature(payload);

```



```

 const items = getHistoryItems();
 if(items[0] && items[0].signature === signature && items[0].label === label)
return true;
 items.unshift({
 id: Date.now() + '_' + Math.random().toString(36).slice(2,8),
 label,
 title: cleanText(state.worksheet?.title || 'Untitled worksheet') || 'Untitled
worksheet',
 createdAt: new Date().toISOString(),
 signature,
 payload
 });
 const saved = setHistoryItems(items);
 if(saved) refreshLibraryModal();
 return saved;
 }catch(err){
 console.warn('History snapshot skipped:', err);
 return false;
 }
}
function renderHistoryPanel(){
 const items = getHistoryItems();
 return `
 <div class="field">
 <label>Session history</label>
 <div class="field-note" style="margin-bottom:10px">Auto-saved locally when
you generate, modify or create variations. To avoid browser storage limits, history
keeps text/settings but not embedded images or PDF pages.</div>
 <div class="pill-row" style="margin-bottom:10px">
 <button class="small-btn" onclick="saveHistorySnapshot()">Save current
snapshot</button>
 <button class="small-btn" onclick="clearHistorySnapshots()">Clear
history</button>
 </div>
 <div class="library-list">
 ${items.length ? items.map(item => `
 <div class="saved-item">
 <div>
 ${esc(item.label)}
 ${esc(item.title)} • ${esc(formatDate(item.createdAt))}
 </div>
 <div class="pill-row">
 <button class="mini"
onclick="restoreHistorySnapshot('${jsq(item.id)}')">Load</button>
 <button class="mini danger"
onclick="deleteHistorySnapshot('${jsq(item.id)}')">Delete</button>
 </div>
 </div>
 `).join('') : '<div class="tiny">No history yet.</div>'}
 </div>

```

```

 </div>
 `;
}
window.saveHistorySnapshot = function(){
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 const saved = pushHistorySnapshot('Manual snapshot');
 refreshLibraryModal();
 renderEditor();
 showStatus(saved ? 'Snapshot saved.' : 'Snapshot could not be saved because
browser storage is full. The current worksheet is still usable.', saved ? 'ok' :
'warn');
};
window.restoreHistorySnapshot = function(id){
 const item = getHistoryItems().find(entry => entry.id === id);
 if(!item) return showStatus('Snapshot not found.', 'warn');
 applySnapshot(item.payload);
 refreshLibraryModal();
 showStatus('Snapshot restored.', 'ok');
};
window.deleteHistorySnapshot = function(id){
 setHistoryItems(getHistoryItems().filter(entry => entry.id !== id));
 refreshLibraryModal();
 renderEditor();
 showStatus('Snapshot deleted.', 'ok');
};
window.clearHistorySnapshots = function(){
 localStorage.removeItem(LS_KEYS.history);
 if(window.SylCycle) SylCycle.cloudSave(LS_KEYS.history, []);
 refreshLibraryModal();
 renderEditor();
 showStatus('History cleared.', 'ok');
};

snapshotState = function(){
 const snapshotCycle = currentCycle;
 const snapshotYear = qs('studioYearLevel')?.value ||
getCycleYears(snapshotCycle)[0] || 'CM1';
 return {
 preferredCycle: snapshotCycle,
 yearLevel: snapshotYear,
 curriculumMode: state.curriculumMode,
 images: state.images,
 promptImages: state.promptImages,
 promptDocuments: state.promptDocuments,
 imageUsage: getImageUsage(),
 templatePdf: state.templatePdf,
 worksheet: state.worksheet,
 qualityCheck: state.qualityCheck,
 selectedOutcomes: state.selectedOutcomes,
 loStatus: state.loStatus,
 }
}

```

```

displayOptions: {
 instructions: state.displayOptions?.instructions !== false,
 successCriteria: state.displayOptions?.successCriteria !== false,
 extension: state.displayOptions?.extension !== false,
 learningObjectives: state.displayOptions?.learningObjectives === true
},
editorAccordions: state.editorAccordions || {},
fields: {
 apiKey: '',
 modelName: qs('modelName').value.trim(),
 outputMode: qs('outputMode').value,
 worksheetBrief: qs('worksheetBrief').value,
 difficulty: qs('difficulty').value,
 questionCount: qs('questionCount').value,
 preferredCycle: snapshotCycle,
 yearLevel: snapshotYear,
 domainSelect: qs('domainSelect').value,
 templateText: qs('templateText').value,
 imageApiKey: '',
 imageModel: qs('imageModel')?.value.trim() || 'gpt-image-1.5',
 imageUsage: getImageUsage(),
}
};
};

applySnapshot = function(saved){
 if(!saved) return;
 const f = saved.fields || {};
 const snapshotCycle = getSnapshotCycle(saved);
 const snapshotYear = getSnapshotYearLevel(saved, snapshotCycle);
 const savedOutcomes = saved.selectedOutcomes ||
saved.worksheet?.learning_outcomes || [];
 state.curriculumMode = saved.curriculumMode || (saved.useAICurriculum ? 'ai_both'
: 'manual_both') || 'ai_both';
 state.images = saved.images || [];
 state.promptImages = saved.promptImages || [];
 state.promptDocuments = saved.promptDocuments || [];
 state.imageUsage = saved.imageUsage || saved.fields?.imageUsage ||
DEFAULT_IMAGE_USAGE;
 state.templatePdf = saved.templatePdf || null;
 state.worksheet = saved.worksheet || null;
 state.qualityCheck = saved.qualityCheck || null;
 if(state.worksheet) state.worksheet.worksheet_images =
resolveWorksheetImages(state.worksheet.worksheet_images, []);
 state.selectedOutcomes = savedOutcomes;
 state.loStatus = saved.loStatus || {};
 state.displayOptions = {
 instructions: saved.displayOptions?.instructions !== false,
 successCriteria: saved.displayOptions?.successCriteria !== false,
 extension: saved.displayOptions?.extension !== false,

```

```

 learningObjectives: saved.displayOptions?.learningObjectives === true
 };
 state.editorAccordions = saved.editorAccordions || state.editorAccordions || {};
 saveEditorAccordionState();
 qs('apiKey').value = f.apiKey || qs('apiKey').value;
 qs('modelName').value = f.modelName || qs('modelName').value;
 qs('outputMode').value = f.outputMode || 'student';
 qs('worksheetBrief').value = f.worksheetBrief || '';
 qs('questionCount').value = f.questionCount || '6';
 applyStudioCycle(snapshotCycle, snapshotYear);
 qs('difficulty').value = normaliseDifficulty(f.difficulty || saved.difficulty ||
'moderate');
 const savedDomain = canonicaliseDomain(f.domainSelect || state.worksheet?.domain
|| '') || defaultStudioDomain();
 qs('domainSelect').value = savedDomain;
 if(state.worksheet) state.worksheet.domain =
canonicaliseDomain(state.worksheet.domain || savedDomain) || savedDomain;
 state.selectedOutcomes = uniqueList(savedOutcomes);
 if(state.worksheet && state.selectedOutcomes.length)
state.worksheet.learning_outcomes = [...state.selectedOutcomes];
 qs('templateText').value = f.templateText || '';
 if(qs('imageApiKey')) qs('imageApiKey').value = f.imageApiKey || '';
 if(qs('imageModel')) qs('imageModel').value = f.imageModel || 'gpt-image-1.5';
 setImageUsage(f.imageUsage || state.imageUsage || DEFAULT_IMAGE_USAGE);
 if(qs('templatePdfInfo')) qs('templatePdfInfo').textContent = state.templatePdf ?
('Loaded: ' + state.templatePdf.name) : 'No PDF loaded.';
 syncToggles();
 renderOutcomeOptions();
 renderImageThumbs();
 renderPromptImageThumbs();
 renderEditor();
 updatePreview();
 savePrefs();
};

```

```

startNewResource = function(){
 state.worksheet = null;
 state.images = [];
 state.promptImages = [];
 state.promptDocuments = [];
 state.templatePdf = null;
 state.qualityCheck = null;
 state.selectedOutcomes = [];
 state.loStatus = {};
 state.curriculumMode = 'ai_both';
 state.imageUsage = DEFAULT_IMAGE_USAGE;
 state.displayOptions = { instructions:false, successCriteria:false,
extension:false, learningObjectives:false };
 state.editorAccordions = {};

```

```

saveEditorAccordionState();
qs('worksheetBrief').value = '';
qs('templateText').value = '';
populateDifficultyOptions('moderate');
qs('questionCount').value = '6';
qs('domainSelect').selectedIndex = 0;
if(qs('imageUsage')) qs('imageUsage').value = DEFAULT_IMAGE_USAGE;
if(qs('imageUpload')) qs('imageUpload').value = '';
if(qs('promptImageUpload')) qs('promptImageUpload').value = '';
if(qs('templatePdf')) qs('templatePdf').value = '';
if(qs('templatePdfInfo')) qs('templatePdfInfo').textContent = 'No PDF loaded.';
if(qs('imageThumbs')) qs('imageThumbs').innerHTML = '';
if(qs('promptImageThumbs')) qs('promptImageThumbs').innerHTML = '';
if(qs('promptImageFileStatus')) qs('promptImageFileStatus').textContent = 'No
source files added.';
clearStatus();
syncToggles();
populateDomains();
renderEditor();
updatePreview();
savePrefs();
goWizardStep(1);
};

```

```

function getWorksheetSectionMeta(key){
 const cycleContext = `${getCycleLabel()} (${getStudioYearLevel()})`;
 const map = {
 instructions: {
 title: 'Instructions',
 placeholder: 'Add short student-facing instructions.',
 emptyNote: 'Leave this blank and it stays off the sheet.',
 success: 'Instructions updated.',
 regenPrompt: `Regenerate ONLY the student-facing instructions for this
printable ${cycleContext} worksheet. Keep them short, clear and classroom-ready.
Usually 1 to 3 brief sentences. Do not include a title. Do not mention answer keys,
teachers, or internal notes.`
 },
 success_criteria: {
 title: 'Success criteria',
 placeholder: 'Add success criteria. One per line works best.',
 emptyNote: 'Leave this blank and it stays off the sheet.',
 success: 'Success criteria updated.',
 regenPrompt: `Regenerate ONLY the success criteria for this printable
${cycleContext} worksheet. Write 2 to 4 short student-friendly criteria, separated
by line breaks. Do not include bullets, numbering, or a title.`
 },
 extension: {
 title: 'Extension task',

```

```

 placeholder: 'Add an optional extension task.',
 emptyNote: 'Leave this blank and it stays off the sheet.',
 success: 'Extension task updated.',
 regenPrompt: `Regenerate ONLY the extension task for this printable
 ${cycleContext} worksheet. Make it a single concise but worthwhile extension that
 fits the worksheet topic and level. Do not include a title.`
 }
};
return map[key];
}

function sectionEditorHtml(key){
 const meta = getWorksheetSectionMeta(key);
 const value = state.worksheet?.[key] || '';
 const hasValue = !!String(cleanText(String(value).replace(/<br\s*\/?>(?!.=)/gi,
 '\n')).replace(/<[^>]+>/g, ''')).trim();
 const editorId = `editSection_${key}`;
 return `
 <div class="task-card">
 <div class="task-head">
 ${meta.title}
 <div class="task-actions">
 <button class="mini"
onclick="regenerateWorksheetSection('${key}')">${hasValue ? 'Regenerate' :
'Generate'}</button>
 <button class="mini danger" onclick="deleteWorksheetSection('${key}')"
${hasValue ? '' : 'disabled'}>Delete</button>
 </div>
 </div>
 ${richEditorHtml({
 id: editorId,
 value,
 placeholder: meta.placeholder,
 note: hasValue ? 'Delete clears it again. Blank means hidden.' : 'Blank by
default. Generate if needed for the worksheet.',
 dataset: { richTarget:'worksheetSection', field:key },
 minHeight: 110
 })}
 </div>
 `;
}

window.updateWorksheetSection = function(key, value){
 if(!state.worksheet) return;
 state.worksheet[key] = value;
 queuePreviewUpdate();
};

window.deleteWorksheetSection = function(key){
 if(!state.worksheet) return;

```

```

 const meta = getWorksheetSectionMeta(key);
 state.worksheet[key] = '';
 renderEditor();
 updatePreview();
 showStatus(`${meta.title} removed from the worksheet.`, 'ok');
 };

window.regenerateWorksheetSection = async function(key){
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 const meta = getWorksheetSectionMeta(key);
 setEditorAiBusy(true, meta ? `Generating ${meta.title.toLowerCase()}...` :
'Generating worksheet section...');
 try{
 showStatus(`Regenerating ${meta.title.toLowerCase()}...`, 'warn');
 const prompt = buildAIPrompt({ mode: 'regenerate_section', sectionKey: key });
 const result = await callAI({prompt, modify:false});
 state.worksheet[key] = cleanText(result?.text || '');
 pushHistorySnapshot('Regenerated section: ' + String(key || 'section'));
 renderEditor();
 updatePreview();
 showStatus(meta.success, 'ok');
 }catch(err){
 showStatus(err.message || `Could not regenerate ${meta.title.toLowerCase()}.`,
'err');
 }finally{
 setEditorAiBusy(false);
 forceEditorRefresh();
 }
};

window.resetWorksheetImagesFromUploads = function(){
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 syncWorksheetImagesFromUploadsInState();
 renderEditor();
 updatePreview();
 showStatus('Worksheet images reset from Step 3 uploads.', 'ok');
};

window.moveWorksheetImage = function(idx, dir){
 if(!state.worksheet?.worksheet_images?.length) return;
 const next = idx + dir;
 if(next < 0 || next >= state.worksheet.worksheet_images.length) return;
 const list = state.worksheet.worksheet_images;
 [list[idx], list[next]] = [list[next], list[idx]];
 renderEditor();
 updatePreview();
};

window.updateWorksheetImageWidth = function(idx, value){
 const image = state.worksheet?.worksheet_images?.[idx];
 if(!image) return;
 image.width = parseImageWidth(value);
};

```

```

 const valueEl = qs(`worksheetImageWidthValue-${idx}`);
 if(valueEl) valueEl.textContent = `${image.width}%`;
 updatePreview();
 };
 window.removeWorksheetImage = function(idx){
 if(!state.worksheet?.worksheet_images?.length) return;
 state.worksheet.worksheet_images.splice(idx, 1);
 renderEditor();
 updatePreview();
 showStatus('Worksheet image removed.', 'ok');
 };

 window.uploadEditorImage = function(){
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 const dest = qs('imageDestination')?.value || 'header';
 const input = document.createElement('input');
 input.type = 'file';
 input.accept = 'image/*';
 input.multiple = dest === 'header';
 input.style.display = 'none';
 input.addEventListener('change', async () => {
 const files = Array.from(input.files || []);
 if(!files.length) return;
 for(const file of files){
 try{
 const dataUrl = await fileToDataURL(file);
 addImageToDestination(dest, dataUrl, file.name || 'Uploaded image',
file.type || 'image/png');
 }catch(err){ console.warn('Could not read image', file.name, err); }
 }
 input.remove();
 renderEditor();
 updatePreview();
 const label = dest === 'header' ? 'worksheet header' : `task
${parseInt(dest.split('_')[1],10) + 1}`;
 showStatus(`${files.length} image${files.length === 1 ? '' : 's'} added to
${label}.`, 'ok');
 });
 document.body.appendChild(input);
 input.click();
 };

 window.pasteEditorImage = async function(){
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 const dest = qs('imageDestination')?.value || 'header';
 try{
 const items = await navigator.clipboard.read();
 let found = false;
 for(const item of items){
 const imageType = item.types.find(t => t.startsWith('image/'));

```



```

 if(!imageType) continue;
 const blob = await item.getType(imageType);
 const dataUrl = await new Promise((res, rej) => {
 const r = new FileReader();
 r.onload = () => res(r.result);
 r.onerror = rej;
 r.readAsDataURL(blob);
 });
 addImageToDestination(dest, dataUrl, 'Pasted image', imageType);
 found = true;
 break;
}
if(!found) return showStatus('No image found on clipboard.', 'warn');
renderEditor();
updatePreview();
const label = dest === 'header' ? 'worksheet header' : `task
${parseInt(dest.split('_')[1],10) + 1}`;
showStatus(`Image pasted into ${label}.`, 'ok');
} catch(err){
 showStatus('Could not read clipboard. Use Upload instead, or allow clipboard
access and try again.', 'warn');
}
};

```

```

window.setEditorAssetDestination = function(dest){
 const select = qs('imageDestination');
 if(select) select.value = dest;
 const promptEl = qs('assetAiPrompt');
 if(!promptEl) return;
 if(dest === 'header'){
 if(!cleanText(promptEl.value || '')) promptEl.value =
buildWorksheetHeaderImagePrompt();
 return;
 }
 const idx = parseInt(String(dest).split('_')[1], 10);
 const task = state.worksheet?.tasks?.[idx];
 if(task && !cleanText(promptEl.value || '')) promptEl.value =
cleanText(task.image_prompt || buildTaskImagePrompt(task, idx));
};

window.triggerEditorAssetPdfImport = function(){
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 const dest = qs('imageDestination')?.value || 'header';
 if(dest === 'header') return showStatus('PDF page import still works exactly as
before, but it only attaches to tasks, not the worksheet header.', 'warn');
 const idx = parseInt(dest.split('_')[1], 10);
 if(!Number.isFinite(idx) || !state.worksheet?.tasks?.[idx]) return
showStatus('Choose a task destination first.', 'warn');
 triggerTaskPdfUpload(idx);
};

```

```

window.generateEditorAssetImage = async function(){
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 const dest = qs('imageDestination')?.value || 'header';
 const promptEl = qs('assetAiPrompt');
 const size = parseImageGenSize(qs('assetAiSize')?.value || state.imageGenSize ||
DEFAULT_IMAGE_GEN_SIZE);
 let prompt = cleanText(promptEl?.value || '');
 if(!prompt){
 if(dest === 'header') prompt = buildWorksheetHeaderImagePrompt();
 else {
 const idx = parseInt(dest.split('_')[1], 10);
 const task = state.worksheet?.tasks?.[idx];
 prompt = cleanText(task?.image_prompt || buildTaskImagePrompt(task, idx));
 }
 if(promptEl) promptEl.value = prompt;
 }
 setEditorAiBusy(true, dest === 'header' ? 'Generating worksheet asset...' :
`Generating asset for task ${parseInt(dest.split('_')[1], 10) + 1}...`);
 try{
 showStatus('Generating image asset...', 'warn');
 const result = await callOpenAIImage(prompt, size);
 if(dest === 'header'){
 if(!Array.isArray(state.worksheet.worksheet_images))
state.worksheet.worksheet_images = [];
 state.worksheet.worksheet_images.push({
 dataUrl: result.dataUrl,
 name: `worksheet-header-image-${state.worksheet.worksheet_images.length +
1}.png`,
 mimeType: result.mimeType || 'image/png',
 source: 'ai',
 provider: 'openai',
 prompt,
 size,
 width: 100
 });
 }else{
 const idx = parseInt(dest.split('_')[1], 10);
 const task = state.worksheet?.tasks?.[idx];
 if(!task) throw new Error('Choose a valid task destination first.');
```

```

 };
 }
 renderEditor();
 updatePreview();
 showStatus(dest === 'header' ? 'Worksheet asset generated.' : `Image generated
for task ${parseInt(dest.split('_')[1], 10) + 1}.`, 'ok');
} catch (err) {
 showStatus(err.message || 'Could not generate that asset image.', 'err');
} finally {
 setEditorAiBusy(false);
 forceEditorRefresh();
}
};
window.syncTaskAssetWidthLabels = function (idx, value) {
 const label = qs(`taskAssetWidthValue-${idx}`);
 if (label) label.textContent = `${parseInt(value)}%`;
};

function addImageToDestination(dest, dataUrl, name, mimeType) {
 if (dest === 'header') {
 if (!Array.isArray(state.worksheet.worksheet_images))
state.worksheet.worksheet_images = [];
 state.worksheet.worksheet_images.push({ dataUrl, name, mimeType,
source: 'upload', width: 100 });
 } else {
 const taskIdx = parseInt(dest.split('_')[1], 10);
 const task = state.worksheet?.tasks?.[taskIdx];
 if (!task) return;
 task.image_enabled = true;
 task.image = { dataUrl, name, mimeType, source: 'upload' };
 if (!task.image_prompt) task.image_prompt = buildTaskImagePrompt(task, taskIdx);
 }
}

renderEditor = function () {
 const wrap = qs('editorPane');
 if (!state.worksheet) {
 wrap.innerHTML = `<div class="editor-empty">No resource yetUse
Start Blank, Start from Template, Adapt PDF, or Generate with AI and it will appear
here for editing.</div>`;
 return;
 }
 const w = state.worksheet;
 const worksheetImages = resolveWorksheetImages(w.worksheet_images, []);
 w.worksheet_images = worksheetImages;

 const basicsBody = `
 <div class="field">
 <label for="editTitle">Worksheet title</label>

```

```

 <input id="editTitle" type="text" value="{esc(w.title)}"
oninput="state.worksheet.title=this.value; queuePreviewUpdate()">
 </div>
 <div class="field">
 <label for="editOutputMode">Preview / export mode</label>
 <select id="editOutputMode" onchange="setOutputMode(this.value)">
 <option value="student"
${qs('outputMode').value==='student'? 'selected':''}>Student sheet</option>
 <option value="both" ${qs('outputMode').value==='both'? 'selected':''}
${hasAnswerKeyData(w)? '' : 'disabled'}>${hasAnswerKeyData(w)? 'Student sheet + answer
key' : 'Student sheet + answer key (generate first)'}</option>
 </select>
 <div class="field-note">${hasAnswerKeyData(w) ? 'Answer key already exists
for this worksheet.' : 'No answer key yet. Generate it only if you actually need
it.'}</div>
 </div>`;

const answerBody = `
 <div class="field">
 <label>Answer key</label>
 <div class="pill-row">
 <button class="small-btn" type="button"
onclick="generateAnswerKeyNow()">${hasAnswerKeyData(w) ? 'Regenerate answer key
now' : 'Generate answer key now'}</button>
 </div>
 <div class="field-note">${hasAnswerKeyData(w) ? 'Useful when you want to
refresh the teacher copy after changing tasks.' : 'This makes a separate AI call
later, so you only pay for answer content when you need it.'}</div>
 </div>`;

const sectionsBody = `
 <div class="field-note" style="margin-bottom:10px">Blank by default. Generate
them only when you want them. Rich text works here too, with Ctrl/Cmd+B, I and
U.</div>
 ${sectionEditorHtml('instructions')}
 ${sectionEditorHtml('success_criteria')}`;

const taskAssets = w.tasks.map((task, idx) => ({ task, idx })).filter(({ task })
=> !(task?.image?.dataUrl || (Array.isArray(task?.pdf_images) &&
task.pdf_images.length)));
const headerAssetCount = worksheetImages.length;
const taskAssetCount = taskAssets.reduce((sum, entry) => sum +
(entry.task?.image?.dataUrl ? 1 : 0) + ((entry.task?.pdf_images || []).length), 0);
const totalAssetCount = headerAssetCount + taskAssetCount;
const assetCountLabel = totalAssetCount ? `${totalAssetCount}
asset${totalAssetCount === 1 ? '' : 's'} across the worksheet.` : 'Upload, paste,
generate or import PDF pages here.';

const assetsBody = `
 <div class="field">

```

```

<label>Add or generate asset</label>
<div
style="display:flex;gap:8px;align-items:center;flex-wrap:wrap;margin-bottom:8px">
 <select id="imageDestination" style="min-width:180px"
onChange="setEditorAssetDestination(this.value)">
 <option value="header">Worksheet header</option>
 ${w.tasks.map((t, i) => `<option value="task_${i}">Task ${i +
1}</option>`).join('')}
 </select>
 <button class="small-btn" type="button"
onclick="uploadEditorImage()">Upload</button>
 <button class="small-btn" type="button"
onclick="pasteEditorImage()">Paste</button>
 <button class="small-btn primary" type="button"
onclick="generateEditorAssetImage()">Generate with AI</button>
 <button class="small-btn" type="button"
onclick="triggerEditorAssetPdfImport()">Import PDF pages</button>
</div>
 <div class="field-note">Choose the destination first. Images can go to the
header or a task. PDF page import stays fully intact, but still attaches to tasks
only because that is the sane way to do it.</div>
</div>
 <div class="field">
 <label for="assetAiPrompt">AI image prompt</label>
 <textarea id="assetAiPrompt" placeholder="Describe the image you want. Leave
blank and the app will suggest one based on the worksheet or selected
task."></textarea>
 <div class="task-editor-grid" style="margin-top:10px">
 <div class="field">
 <label for="assetAiSize">Generated image shape</label>
 <select id="assetAiSize">
 <option value="1024x1024" ${parseImageGenSize(state.imageGenSize ||
DEFAULT_IMAGE_GEN_SIZE) === '1024x1024' ? 'selected' : ''}>Square</option>
 <option value="1024x1536" ${parseImageGenSize(state.imageGenSize ||
DEFAULT_IMAGE_GEN_SIZE) === '1024x1536' ? 'selected' : ''}>Portrait</option>
 <option value="1536x1024" ${parseImageGenSize(state.imageGenSize ||
DEFAULT_IMAGE_GEN_SIZE) === '1536x1024' ? 'selected' : ''}>Landscape</option>
 </select>
 </div>
 <div class="field">
 <label>Quick targets</label>
 <div class="field-note">The destination dropdown above decides where
uploaded, pasted, AI-generated or imported assets land.</div>
 </div>
 </div>
 </div>
 ${worksheetImages.length ? `<div class="field">
 <label for="editImageNote">Student note above header images</label>
 ${richEditorHtml({
 id: 'editImageNote',

```

```

 value: w.image_note || '',
 placeholder: 'Optional short instruction shown above the header images.',
 dataset: { richTarget: 'worksheetField', field: 'image_note' },
 minHeight: 90
)))
</div>` : ''}
 ${worksheetImages.length ? `<div class="field">
 <label>Header assets</label>
 <div class="field-note" style="margin-bottom:10px">These appear at the top of
the worksheet. 100% width stacks on its own line; smaller widths sit side by
side.</div>
 <div id="worksheetImageEditorList">${worksheetImages.map((img,
idx)=>worksheetImageEditorHtml(img, idx)).join('')}</div>
 </div>` : ''}
 ${taskAssets.length ? `<div class="field">
 <label>Task assets</label>
 <div class="field-note" style="margin-bottom:10px">Manage task visuals and
imported PDF pages here instead of inside every single task card. Same features,
less nonsense.</div>
 <div class="library-list">${taskAssets.map(entry =>
taskAssetManagerCardHtml(entry.task, entry.idx)).join('')}</div>
 </div>` : ''}
 ${!worksheetImages.length && !taskAssets.length ? '<div class="field"><div
class="tiny">No assets on the worksheet yet. Use the controls above to upload,
paste, generate with AI or import PDF pages.</div></div>' : ''}`;

const outcomesBody = `
 <div class="field">
 <label for="editDomain">Domain</label>
 <select id="editDomain" onchange="setWorksheetDomain(this.value);
renderEditorOutcomeBrowser()">${DOMAIN_NAMES.map(name=>`<option
value="${esc(name)}"
${w.domain===name?'selected':''}>${esc(name)}</option>`).join('')}</select>
 </div>
 <div class="field">
 <label>Learning outcomes</label>
 <div class="option-stack" style="margin-bottom:10px">
 <label class="check-option"><input type="checkbox"
${state.displayOptions?.learningObjectives === true ? 'checked' : ''}
onchange="updateDisplayOption('learningObjectives', this.checked)"> Show
assessment table under the learning outcomes</label>
 </div>
 <div class="pill-row" style="margin-bottom:10px">
 <button class="small-btn" onclick="toggleEditorOutcomeBrowser()">Browse
curriculum</button>
 <button class="small-btn primary" id="autoDetectLosBtn"
onclick="autoDetectLearningOutcomes()">Auto-detect LOs</button>
 </div>
 <div id="editorOutcomeBrowser" class="hidden" style="margin-bottom:12px">
 <div class="outcomes-box" id="editorOutcomesBox"

```

```

style="max-height:400px"></div>
</div>
<div id="editorOutcomeList">${w.learning_outcomes.map((outcome, idx)=>`
 <div class="task-card" style="padding:10px 12px">
 <div class="task-head">
 Outcome ${idx + 1}
 <button class="mini danger"
onclick="removeOutcome(${idx})">Remove</button>
 </div>
 <textarea oninput="updateOutcome(${idx}, this.value)"
style="min-height:72px">${esc(outcome)}</textarea>
 </div>
 `).join('')}</div>
 <button class="small-btn" onclick="addOutcome()">Add learning
outcome</button>
</div>`;

const modifyBody = `
 <div class="field">
 <label for="modifyRequest">Modify with AI</label>
 <textarea id="modifyRequest" placeholder="Examples: Make it shorter. Turn
task 3 into multiple choice. Make the vocabulary more demanding. Base it more
heavily on the uploaded images."></textarea>
 <div class="pill-row" style="margin-top:8px">
 <button class="small-btn primary" onclick="modifyWorksheet()">Modify with
AI</button>
 </div>
 </div>`;

const tasksSubtitle = `${w.tasks.length} task${w.tasks.length === 1 ? '' : 's'}`;
ready. Open only the one you need instead of doing a full Everest.`;
const tasksBody = `
 <div class="field">
 <div class="field-note" style="margin-bottom:10px">Rich text shortcuts work
inside passage, prompt and answer fields: Ctrl/Cmd+B, I and U.</div>
 <div class="editor-task-tools">
 <button class="small-btn" type="button"
onclick="setAllTaskAccordions(true)">Expand all tasks</button>
 <button class="small-btn" type="button"
onclick="setAllTaskAccordions(false)">Collapse all tasks</button>
 <button class="small-btn" onclick="addTask()">Add task</button>
 </div>
 <div id="taskEditorList">${w.tasks.map((task, idx)=>taskEditorHtml(task,
idx)).join('')}</div>
 </div>`;

const extensionBody = `
 <div class="field">
 <label>Extension section</label>
 ${sectionEditorHtml('extension')}
 </div>

```

```
</div>`;
```

```
const sectionCount = [w.instructions, w.success_criteria].filter(value =>
accordionPlainText(value)).length;
```

```
wrap.innerHTML = `
 <div class="editor-section-stack">
 ${editorAccordionHtml({ key:'editor_basics', title:'Worksheet basics',
 subtitle:accordionSnippet(w.title, 'Title and preview/export mode.'),
 body:basicsBody, defaultOpen:false })}
 ${editorAccordionHtml({ key:'editor_quality_check', title:'Check worksheet
 before export', subtitle:qualityCheckSubtitle(), body:renderQualityCheckBody(),
 defaultOpen:false })}
 ${editorAccordionHtml({ key:'editor_outcomes', title:'Domain and learning
 outcomes', subtitle:`${esc(w.domain)} · ${w.learning_outcomes.length}
 outcome${w.learning_outcomes.length === 1 ? '' : 's'}`., body:outcomesBody,
 defaultOpen:false })}
 ${editorAccordionHtml({ key:'editor_sections', title:'Worksheet sections',
 subtitle:sectionCount ? `${sectionCount} section${sectionCount === 1 ? '' : 's'}
 currently in use.` : 'Instructions and success criteria are both currently blank.',
 body:sectionsBody, defaultOpen:false })}
 ${editorAccordionHtml({ key:'editor_tasks', title:'Tasks',
 subtitle:tasksSubtitle, body:tasksBody, defaultOpen:false })}
 ${editorAccordionHtml({ key:'editor_assets', title:'Assets',
 subtitle:assetCountLabel, body:assetsBody, defaultOpen:false })}
 ${editorAccordionHtml({ key:'editor_extension', title:'Extension section',
 subtitle:accordionSnippet(w.extension, 'Optional challenge task. Leave it collapsed
 if unused.'), body:extensionBody, defaultOpen:false })}
 ${editorAccordionHtml({ key:'editor_answer_key', title:'Answer key',
 subtitle:hasAnswerKeyData(w) ? 'Teacher copy exists and can be exported.' : 'Still
 student-only. Generate answers later if you need them.', body:answerBody,
 defaultOpen:false })}
 ${editorAccordionHtml({ key:'editor_variations', title:'Generate variations',
 subtitle:'Spin off easier, harder, homework or same-LO versions.', body:`<div
 class="field"><div class="variation-grid"><button class="small-btn"
 onclick="generateVariation('easier')">Easier version</button><button
 class="small-btn" onclick="generateVariation('harder')">Harder
 version</button><button class="small-btn"
 onclick="generateVariation('different_questions')">Same worksheet, different
 questions</button><button class="small-btn"
 onclick="generateVariation('same_lo_new_topic')">Same LO, different
 topic</button><button class="small-btn"
 onclick="generateVariation('homework')">Homework version</button><button
 class="small-btn" onclick="generateVariation('classwork')">Classwork
 version</button></div></div>`, defaultOpen:false })}
 ${editorAccordionHtml({ key:'editor_modify', title:'Modify with AI',
 subtitle:'Use this when you want a broad rewrite instead of hand-editing every
 field.', body:modifyBody, defaultOpen:false })}
 </div>
`;
```



```

 ensureEditorAiOverlay();
 if(state.editorAiBusy) setEditorAiBusy(true, state.editorAiBusyMessage ||
'Generating content...');
};

renderStudentPage = function(includeAnswersSection=false){
 const w = state.worksheet;
 if(!w) return '';
 const hasInstructions = !!String(w.instructions || '').trim();
 const hasSuccessCriteria = !!String(w.success_criteria || '').trim();
 const hasExtension = !!String(w.extension || '').trim();
 const hasLearningOutcomes = Array.isArray(w.learning_outcomes) &&
w.learning_outcomes.length;
 const showLearningObjectiveTicks = state.displayOptions?.learningObjectives ===
true && hasLearningOutcomes;
 const learningObjectivesBlock = hasLearningOutcomes ?
renderLOBlock(w.learning_outcomes, showLearningObjectiveTicks, 'student') : '';
 const worksheetImages = resolveWorksheetImages(w.worksheet_images, []);
 w.worksheet_images = worksheetImages;
 const imgs = worksheetImages.length ? `
 <div>
 ${w.image_note ? `<div class="image-note
rich-render">${renderRichText(w.image_note)}</div>` : ''}
 <div class="worksheet-images">
 ${worksheetImages.map(img=>`<div class="worksheet-image-item" style="flex:0
0 calc(${parseImageWidth(img.width ?? 100)}% -
6px);max-width:calc(${parseImageWidth(img.width ?? 100)}% - 6px)"></div>`).join('')}
 </div>
 </div>` : '';
 return `
 <div class="preview-page">
 <div class="page-header">
 <div>
 <div class="page-kicker">CycleStudio</div>
 <h1 class="worksheet-title">${esc(w.title)}</h1>
 </div>

 </div>

 ${learningObjectivesBlock}

 ${hasInstructions ? `
 <div class="instructions">
 <h3>Instructions</h3>
 <div class="rich-render">${renderRichText(w.instructions)}</div>
 </div>` : ''}

 ${hasSuccessCriteria ? renderCriteriaBlock(w.success_criteria) : ''}

```

```

 ${imgs}
 ${w.tasks.map(renderTaskPrint).join('')}

 ${hasExtension ? `
 <div class="extension-box">
 <h3 style="margin:0 0 8px">Extension</h3>
 <div class="rich-render worksheet-extension-text"
style="margin:0;line-height:1.62">${renderRichText(w.extension)}</div>
 </div>` : ''}

 ${includeAnswersSection ? `
 <div class="answer-key">
 <h3>Answer key</h3>
 ${renderAnswerItems(w.tasks)}
 </div>` : ''}
 </div>
`;
};

renderAnswerPage = function(){
 const w = state.worksheet;
 if(!w) return '';
 const hasInstructions = !!String(w.instructions || '').trim();
 const hasSuccessCriteria = !!String(w.success_criteria || '').trim();
 const hasExtension = !!String(w.extension || '').trim();
 const hasLearningOutcomes = Array.isArray(w.learning_outcomes) &&
w.learning_outcomes.length;
 const showLearningObjectiveTicks = state.displayOptions?.learningObjectives ===
true && hasLearningOutcomes;
 const learningObjectivesBlock = hasLearningOutcomes ?
renderLOBlock(w.learning_outcomes, showLearningObjectiveTicks, 'answer') : '';
 return `
 <div class="preview-page">
 <div class="page-header">
 <div>
 <div class="page-kicker">Teacher copy</div>
 <h1 class="worksheet-title">${esc(w.title)} – Answer Key</h1>
 </div>

 </div>

 ${hasSuccessCriteria ? `<div class="success-box"><h3>Success criteria
reference</h3><div class="rich-render
rich-criteria">${renderRichText(w.success_criteria)}</div></div>` : ''}

 ${learningObjectivesBlock}

 ${hasInstructions ? `
 <div class="section-print">
 <h3 style="margin:0 0 10px;font-size:16px;color:#12325f">Teacher

```

```

notes</h3>
 <div class="rich-render worksheet-teacher-notes"
style="margin:0;line-height:1.65;color:#355579">${renderRichText(w.instructions)}</
div>
 </div>` : ''}

 <div class="answer-key">
 <h3>Answers and marking points</h3>
 ${renderAnswerItems(w.tasks)}
 ${hasExtension ? `<div class="answer-item">Extension.
Teacher judgement / open response unless otherwise specified.</div>` : ''}
 </div>
</div>
`;
};

```

```

function buildStarterDomain(){
 const selectedDomain = canonicaliseDomain(qs('domainSelect')?.value.trim() ||
'');
 if(selectedDomain) return selectedDomain;
 const evidenceText = [qs('worksheetBrief')?.value.trim() || '',
qs('templateText')?.value.trim() || ''].filter(Boolean).join('\n');
 return inferDomainFromText(evidenceText) || defaultStudioDomain();
}

```

```

function buildStarterLearningOutcomes(domain){
 const evidenceText = [qs('worksheetBrief')?.value.trim() || '',
qs('templateText')?.value.trim() || '', domain].filter(Boolean).join('\n');
 let learningOutcomes = [];
 if((state.curriculumMode || 'ai_both') === 'manual_both'){
 learningOutcomes = uniqueList((state.selectedOutcomes || []).map(outcome =>
canonicaliseOutcome(outcome, domain)).filter(Boolean));
 }else{
 learningOutcomes = chooseAllowedOutcomes(domain, evidenceText, 3,
state.selectedOutcomes || []);
 }
 if(!learningOutcomes.length) learningOutcomes =
getFlatDomainOutcomes(domain).slice(0, 3);
 return learningOutcomes;
}

```

```

window.renderTemplateStarter = function(){
 const list = qs('templateStarterList');
 const count = qs('templateStarterCount');
 if(!list || !count) return;
 const templates = getSavedItems('template');
 count.textContent = templates.length
 ? `${templates.length} saved template${templates.length === 1 ? '' : 's'}`
available.`

```

```

 : 'No saved templates yet.';
list.innerHTML = templates.length ? templates.map(item => `
 <div class="saved-item">
 <div>
 ${esc(item.name)}
 ${esc(formatDate(item.createdAt))}
 </div>
 <div class="pill-row">
 <button class="mini" type="button"
onclick="startWorksheetFromTemplate('${jsq(item.id)}')">Start from this</button>
 <button class="mini danger" type="button" onclick="deleteSaved('template',
'${jsq(item.id)}'); renderTemplateStarter()>Delete</button>
 </div>
 </div>
`).join('') : '<div class="tiny">No saved templates yet. Build a worksheet you
like, then click “Save as template” in the editor.</div>';
};

window.toggleTemplateStarter = function(forceOpen){
 const panel = qs('templateStarterPanel');
 if(!panel) return;
 const shouldOpen = typeof forceOpen === 'boolean' ? forceOpen :
panel.classList.contains('hidden');
 panel.classList.toggle('hidden', !shouldOpen);
 if(shouldOpen) renderTemplateStarter();
};

window.toggleAdaptPdfStarter = function(forceOpen){
 const panel = qs('adaptPdfPanel');
 if(!panel) return;
 const shouldOpen = typeof forceOpen === 'boolean' ? forceOpen :
panel.classList.contains('hidden');
 panel.classList.toggle('hidden', !shouldOpen);
};

window.startWorksheetFromTemplate = function(id){
 const item = getSavesItems('template').find(entry => entry.id === id);
 if(!item) return showStatus('Template not found.', 'warn');
 applySnapshot(item.payload);
 toggleTemplateStarter(false);
 goWizardStep(4);
 showStatus(`Template “${item.name}” loaded.`, 'ok');
};

window.startBlankWorksheet = function(){
 const selectedDomain = canonicaliseDomain(qs('domainSelect')?.value.trim() || '')
|| defaultStudioDomain();
 const curriculumMode = state.curriculumMode || 'ai_both';
 const learningOutcomes = (curriculumMode === 'manual_both' && selectedDomain)
? uniqueList((state.selectedOutcomes || []).map(outcome =>

```

```

canonicaliseOutcome(outcome, selectedDomain)).filter(Boolean))
 : [];

state.worksheet = {
 domain: selectedDomain,
 learning_outcomes: learningOutcomes,
 title: 'Untitled worksheet',
 instructions: '',
 success_criteria: '',
 intro_text: '',
 image_note: '',
 worksheet_images: resolveWorksheetImages([], []),
 tasks: [normaliseTask({
 number: 1,
 passage: '',
 prompt: '',
 options: [],
 answer_lines: 0,
 answer: '',
 image_enabled: false,
 image_prompt: '',
 image_size: state.imageGenSize || DEFAULT_IMAGE_GEN_SIZE,
 image_width: 100,
 image: null,
 pdf_images: []
 }, 0)],
 extension: ''
};

if(qs('domainSelect')) qs('domainSelect').value = selectedDomain;
state.selectedOutcomes = [...learningOutcomes];
renderOutcomeOptions();

state.loStatus = {};
learningOutcomes.forEach(lo => { state.loStatus[lo] = ''; });

state.displayOptions = { instructions:false, successCriteria:false,
extension:false, learningObjectives:false };
if(qs('outputMode')) qs('outputMode').value = 'student';
renderEditor();
updatePreview();
savePrefs();
goWizardStep(4);
showStatus('Blank worksheet ready.', 'ok');
};

window.addEventListener('DOMContentLoaded', ()=>{
 function _initStudio(){
 populateDomains();
 }
});

```

```

 ensureImageUsageUi();
 ensureImageGenerationUi();
 loadPrefs();
 bindPrefInputs();
 bindUploads();
 renderImageThumbs();
 renderPromptImageThumbs();
 wireV4Controls();
 wireImageGenerationControls();
 ensureTaskImageUploadInput();
 ensureTaskPdfUploadInput();
 if(qs('imageUsage') && !qs('imageUsage').value) qs('imageUsage').value =
state.imageUsage || DEFAULT_IMAGE_USAGE;
 updatePreview();
 initWizardLayout();
 ensureWorksheetFontSizeUi();
 applyWorksheetFontCssVars(getWorksheetFontSize());
 renderTemplateStarter();
 goWizardStep(state.currentWizardStep || 1);
 document.body.classList.remove('studio-booting');
 }
 if(window.SylCycle && SylCycle.onReady) SylCycle.onReady(_initStudio);
 else _initStudio();
});

```

```

// =====
// — Photocopy print boost & editor AI overlay —
// =====

```

```

const extraPrintCss = `
 /* Photocopy boost: stronger borders + light texture so school copiers do not
wipe out the structure */
 .teacher-box,
 .instructions,
 .task-print,
 .extension-box,
 .success-box,
 .task-passage,
 .section-print{
 border-color:#7387a2 !important;
 border-width:2.1px !important;
 box-shadow:none !important;
 }
 .teacher-box,
 .success-box,
 .task-passage,
 .section-print{
 background:

```

```

 #e4eaf1,
 repeating-linear-gradient(135deg, rgba(72,90,116,.18) 0 3px,
 rgba(255,255,255,0) 3px 8px) !important;
 }
 .instructions{
 background:
 #dde5ee,
 repeating-linear-gradient(135deg, rgba(72,90,116,.20) 0 3px,
 rgba(255,255,255,0) 3px 8px) !important;
 border-left:7px solid #41556e !important;
 }
 .task-print{
 background:
 #ebf0f6,
 repeating-linear-gradient(180deg, rgba(95,113,137,.15) 0 2px,
 rgba(255,255,255,0) 2px 10px) !important;
 }
 .extension-box{
 background:
 #e6ecf3,
 repeating-linear-gradient(135deg, rgba(72,90,116,.17) 0 3px,
 rgba(255,255,255,0) 3px 8px) !important;
 border-style:dashed !important;
 }
 .teacher-tag,
 .task-number,
 .tick-cell.active .tick-box{
 background:#d8e0ea !important;
 color:#102847 !important;
 border-color:#647894 !important;
 }
 .teacher-tag{
 border:1.6px solid #6f839f !important;
 font-weight:800 !important;
 }
 .lo-table th{
 background:#dbe3ec !important;
 color:#29486f !important;
 }
 .lo-table th,
 .lo-table td{
 border-color:#90a2ba !important;
 }
 .answer-line{
 border-bottom:1.8px solid #7f93af !important;
 background:none !important;
 }
 .worksheet-images img,
 .task-image-print{
 border-width:1.8px !important;
 }

```

```

 border-color:#7b8ea8 !important;
}
.line-fill span:last-child{
 border-bottom:2.2px solid #6f839f !important;
}
.answer-key{
 border-top:2.5px solid #7387a2 !important;
}

/* Safe media layout for print/PDF export */
.task-media-frame{
 border:2.1px solid #7387a2;
 border-radius:18px;
 background:#fff;
 padding:6px;
 margin:0 0 8px;
 overflow:hidden;
 box-shadow:none;
 break-inside:avoid;
 break-inside:avoid-page;
 page-break-inside:avoid;
}
.task-media-frame .task-image-wrap{
 margin:0;
 display:flex;
 justify-content:center;
 align-items:flex-start;
}
.task-media-frame .task-image-wrap + .task-image-wrap{margin-top:6px}
.task-media-frame .task-image-print{
 display:block;
 border:0 !important;
 padding:0 !important;
 background:#fff;
 border-radius:12px;
 max-width:100%;
 height:auto;
}
.task-print-after-media{margin-top:0;}

```

```
`;
```

```

function ensureEditorAiOverlay(){
 const host = document.querySelector('.editor-preview');
 if(!host) return null;
 let overlay = document.getElementById('editorAiOverlay');
 if(!overlay){
 overlay = document.createElement('div');
 overlay.id = 'editorAiOverlay';
 overlay.className = 'editor-ai-overlay';
 overlay.innerHTML = `

```



```

 <div class="editor-ai-overlay-box" role="status" aria-live="polite"
aria-atomic="true">
 <div class="editor-ai-overlay-spinner" aria-hidden="true"></div>
 <div>
 <div class="editor-ai-overlay-title">Working on it</div>
 <p class="editor-ai-overlay-text" id="editorAiOverlayText">Generating
content...</p>
 </div>
 </div>`;
 host.appendChild(overlay);
 } else if(overlay.parentElement !== host){
 host.appendChild(overlay);
 }
 return overlay;
}

```

```

function setEditorAiBusy(active, message){
 state.editorAiBusy = !!active;
 state.editorAiBusyMessage = message || 'Generating content...';
 const host = document.querySelector('.editor-preview');
 const overlay = ensureEditorAiOverlay();
 const textEl = document.getElementById('editorAiOverlayText');
 if(textEl) textEl.textContent = state.editorAiBusyMessage;
 if(host) host.classList.toggle('editor-ai-busy', !!active);
 if(overlay) overlay.setAttribute('aria-hidden', active ? 'false' : 'true');
}

```

```

// — Post-boot init (from former IIFE 3) —
window.addEventListener("resize", fitPreviewToPane);

```

```

</script>

```

```

<style id="cyclestudio-multi-task-images-style">
 /* Multi-image task layout: images share a frame and wrap side-by-side when the
display width is small enough. */
 .preview-page .task-media-frame,
 .task-media-frame{
 display:flex !important;
 flex-wrap:wrap !important;
 align-items:flex-start !important;
 justify-content:center !important;
 gap:6px !important;
 border:2.1px solid #7387a2;
 border-radius:18px;
 background:#fff;
 padding:6px;
 margin:0 0 8px;
 overflow:hidden;
 }

```

```

 box-shadow:none;
 break-inside:avoid;
 break-inside:avoid-page;
 page-break-inside:avoid;
}
.preview-page .task-media-frame .task-image-wrap,
.task-media-frame .task-image-wrap{
 margin:0 !important;
 display:flex !important;
 justify-content:center !important;
 align-items:flex-start !important;
 min-width:0 !important;
}
.preview-page .task-media-frame .task-image-print,
.task-media-frame .task-image-print{
 display:block !important;
 width:100% !important;
 max-width:100% !important;
 height:auto !important;
 object-fit:contain !important;
 border:0 !important;
 padding:0 !important;
 background:#fff !important;
 border-radius:12px !important;
}
.task-image-gallery{display:flex;flex-direction:column;gap:10px;margin-top:10px}
.task-image-card{border:1px solid
#dce8f6;border-radius:14px;background:#f8fbff;padding:10px}

.task-image-card-head{display:flex;justify-content:space-between;align-items:center
;gap:10px;margin-bottom:8px}
.task-image-card-title{font-size:12px;font-weight:800;color:#12325f}
.task-image-card
img{display:block;width:100%;max-height:220px;object-fit:contain;border:1px solid
#dce8f6;border-radius:10px;background:#fff}
</style>

```

```

<script id="cyclestudio-multi-task-images-patch">
(function(){
 function safeClean(value){
 return typeof cleanText === 'function' ? cleanText(value || '') : String(value
|| '').trim();
 }
 function imageKey(img){ return String(img?.dataUrl || img?.name || img?.prompt ||
''); }

```

```

window.normaliseTaskImages = function(task={}, previousTask=null){
 const out = [];
 const seen = new Set();
 const add = (candidate, idx=0) => {

```

```

 const normalised = typeof normaliseTaskImage === 'function'
 ? normaliseTaskImage(candidate)
 : (candidate?.dataUrl ? candidate : null);
 if(!normalised?.dataUrl) return;
 const key = imageKey(normalised);
 if(seen.has(key)) return;
 seen.add(key);
 normalised.name = safeClean(normalised.name || `task-image-${out.length +
1}.png`) || `task-image-${out.length + 1}.png`;
 out.push(normalised);
 };

 if(Array.isArray(task?.images)) task.images.forEach(add);
 if(Array.isArray(previousTask?.images)) previousTask.images.forEach(add);
 add(task?.image || null);
 add(previousTask?.image || null);
 return out;
};

window.getTaskImages = function(task){
 if(!task) return [];
 const images = window.normaliseTaskImages(task, null);
 task.images = images;
 task.image = images[0] || null;
 if(images.length) task.image_enabled = true;
 return images;
};

window.syncLegacyTaskImage = function(task){
 const images = window.getTaskImages(task);
 task.images = images;
 task.image = images[0] || null;
 if(images.length) task.image_enabled = true;
 return task;
};

normaliseTask = function(task, idx, previousTask=null){
 const images = window.normaliseTaskImages(task, previousTask);
 const base = {
 number: idx + 1,
 passage: safeClean(task?.passage || ''),
 prompt: safeClean(task?.prompt || ''),
 options: Array.isArray(task?.options) ?
task.options.filter(Boolean).map(v=>String(v).trim()) : [],
 answer_lines: parseAnswerLines(task?.answer_lines),
 answer: safeClean(task?.answer || ''),
 image_enabled: !!(task?.image_enabled ?? previousTask?.image_enabled ??
images.length),
 image_prompt: safeClean(task?.image_prompt || previousTask?.image_prompt ||
''),

```

```

 image_size: parseImageGenSize(task?.image_size ?? previousTask?.image_size ??
state.imageGenSize ?? DEFAULT_IMAGE_GEN_SIZE),
 image_width: parseImageWidth(task?.image_width ?? previousTask?.image_width
?? 100),
 images,
 image: images[0] || null,
 pdf_images: resolveTaskPdfImages(task?.pdf_images, previousTask?.pdf_images
|| []),
 };
 if(base.image_enabled && !base.image_prompt){
 base.image_prompt = buildTaskImagePrompt(base, idx);
 }
 return base;
};

window.taskVisualCount = function(task){
 return window.getTaskImages(task).length + (Array.isArray(task?.pdf_images) ?
task.pdf_images.length : 0);
};

renderTaskPrint = function(task){
 window.syncLegacyTaskImage(task);
 const imageWidth = parseImageWidth(task.image_width ?? 100);
 const flexBasis = imageWidth >= 100 ? '100%' : `calc(${imageWidth}% - 6px)`;
 const wrapStyle = `flex:0 0 ${flexBasis};max-width:${imageWidth}%`;
 const taskImages = (task.image_enabled !== false ? window.getTaskImages(task) :
[]).map((image, imageIdx)=>`
 <div class="task-image-wrap" style="${wrapStyle}"></div>`).join('');
 const pdfImages = Array.isArray(task.pdf_images) ? task.pdf_images.map((image,
pdfIdx)=>`
 <div class="task-image-wrap" style="${wrapStyle}"></div>`).join('') : '';
 const image = taskImages + pdfImages;
 const passage = task.passage ? `<div class="task-passage
rich-render">${renderRichText(task.passage)}</div>` : '';
 const prompt = String(task.prompt || '').trim() ? `<span class="task-prompt
rich-render">${renderRichText(task.prompt)}` : '';
 const options = task.options?.length ? `
 <ol class="mcq-list" type="A">
 ${task.options.map(opt=>`<li>${esc(opt)}</li>`).join('')}
 ` : '';
 const lines = task.answer_lines > 0 ? `
 <div class="answer-lines">
 ${Array.from({length: task.answer_lines}).map(()==>'<div
class="answer-line"></div>').join('')}
 </div>` : '';

```

```

const writtenTask = `
 <div class="task-print ${image ? 'task-print-after-media' : ''}">
 ${passage}
 <div>${task.number}${prompt}</div>
 ${options}
 ${lines}
 </div>`;
if(!image) return writtenTask;
return `<div class="task-media-frame">${image}</div>${writtenTask}`;
};

setTaskImageFromDataUrl = function(idx, dataUrl, options={}){
 const task = state.worksheet?.tasks?.[idx];
 if(!task || !dataUrl) return false;
 const newImage = {
 source: options.source || 'upload',
 provider: options.provider || '',
 prompt: safeClean(options.prompt || task.image_prompt || ''),
 mimeType: options.mimeType || 'image/png',
 dataUrl,
 name: safeClean(options.name || `task-${idx +
1}-image-${window.getTaskImages(task).length + 1}.png`) || `task-${idx +
1}-image.png`,
 size: options.size || task.image_size || ''
 };
 const images = window.getTaskImages(task);
 images.push(newImage);
 task.images = images;
 task.image = images[0] || newImage;
 task.image_enabled = true;
 if(!task.image_prompt) task.image_prompt = buildTaskImagePrompt(task, idx);
 return true;
};

const originalEnsureTaskImageUploadInput = typeof ensureTaskImageUploadInput ===
'function' ? ensureTaskImageUploadInput : null;
ensureTaskImageUploadInput = function(){
 let input = qs('taskImageUploadInput');
 if(input && input.dataset.multiTaskImageReady === '1') return input;
 if(input) input.remove();
 input = document.createElement('input');
 input.type = 'file';
 input.accept = 'image/*';
 input.multiple = true;
 input.id = 'taskImageUploadInput';
 input.className = 'hidden';
 input.dataset.multiTaskImageReady = '1';
 input.addEventListener('change', async (e)=>{
 const idx = parseInt(input.dataset.taskIdx || '-1', 10);
 const files = Array.from(e.target.files || []);

```

```

if(!files.length || !state.worksheet?.tasks[idx]) return;
let added = 0;
try{
 for(const file of files){
 const dataUrl = await fileToDataUrl(file);
 if(setTaskImageFromDataUrl(idx, dataUrl, {
 source: 'upload',
 mimeType: file.type || 'image/png',
 name: file.name || `task-${idx + 1}-image-${added + 1}.png`
 })){ added++;
 }
 renderEditor();
 updatePreview();
 showStatus(`${added} image${added === 1 ? '' : 's'} added to task ${idx +
1}.`, 'ok');
}catch(err){
 showStatus('Could not load one of those images from disk.', 'err');
}finally{
 input.value = '';
}
});
document.body.appendChild(input);
return input;
};

```

```

window.removeTaskImage = function(idx, imageIdx=null){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return;
 const images = window.getTaskImages(task);
 if(Number.isFinite(Number(imageIdx))){
 images.splice(Number(imageIdx), 1);
 task.images = images;
 task.image = images[0] || null;
 showStatus(`Image removed from task ${idx + 1}.`, 'ok');
 }else{
 task.images = [];
 task.image = null;
 showStatus(`Images removed from task ${idx + 1}.`, 'ok');
 }
 if(!task.images.length && !(Array.isArray(task.pdf_images) &&
task.pdf_images.length)) task.image_enabled = false;
 renderEditor();
 updatePreview();
};

```

```

window.generateTaskImage = async function(idx){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return showStatus('No task to illustrate.', 'warn');
 state.imageJobs = state.imageJobs || {};
 if(state.imageJobs[idx]) return;

```

```

setEditorAiBusy(true, `Generating image for task ${Number(idx) + 1}...`);
try{
 task.image_enabled = true;
 const prompt = safeClean(task.image_prompt || buildTaskImagePrompt(task,
idx));
 const size = parseImageGenSize(task.image_size || state.imageGenSize ||
DEFAULT_IMAGE_GEN_SIZE);
 task.image_prompt = prompt;
 task.image_size = size;
 state.imageJobs[idx] = true;
 renderEditor();
 showStatus(`Generating image for task ${idx + 1}...`, 'warn');
 const result = await callOpenAIImage(prompt, size);
 setTaskImageFromDataURL(idx, result.dataUrl, {
 source: 'ai',
 provider: 'openai',
 prompt,
 mimeType: result.mimeType || 'image/png',
 name: `task-${idx + 1}-ai-image-${window.getTaskImages(task).length +
1}.png`,
 size
 });
 updatePreview();
 showStatus(`Image added to task ${idx + 1}.`, 'ok');
}catch(err){
 showStatus(err.message || 'Could not generate that image.', 'err');
}finally{
 delete state.imageJobs[idx];
 setEditorAiBusy(false);
 forceEditorRefresh();
}
};

```

```

function taskImageGalleryHtml(task, idx, imageSize){
 const images = window.getTaskImages(task);
 if(!images.length) return '';
 return `<div class="task-image-gallery">${images.map((image, imageIdx)=>`
 <div class="task-image-card">
 <div class="task-image-card-head">
 <div class="task-image-card-title">Image ${imageIdx + 1}</div>
 <button class="mini danger" type="button"
onclick="removeTaskImage(${idx}, ${imageIdx})">Remove</button>
 </div>

 <div class="task-image-meta">${taskImageSourceLabel(image, imageSize)}
Display width: ${task.image_width || 100}%</div>
 </div>`).join('')}</div>`;
}

```

```

taskAssetSummaryHtml = function(task, idx){

```

```

const imageCount = window.getTaskImages(task).length;
const pdfCount = Array.isArray(task?.pdf_images) ? task.pdf_images.length : 0;
if(!imageCount && !pdfCount){
 return `
```



```

'Crop'</button>
 <button class="mini danger" type="button"
onclick="removeTaskPdfImage(${idx}, ${pdfIdx})">Remove</button>
 </div>
</div>

 <div class="task-pdf-meta">${taskPdfImageSourceLabel(image)} Display
width: ${task.image_width || 100}%.${hasCrop ? ' Cropped version shown below and on
the worksheet.' : ''}</div>
 ${cropOpen ? `
 <div class="task-pdf-crop-panel">
 <div class="task-pdf-crop-grid">
 ${[['left', 'right', 'top', 'bottom']].map(side=>`
 <div class="field">
 <label>${side.charAt(0).toUpperCase() + side.slice(1)} crop
<span class="tiny"
id="taskPdfCropValue-${idx}-${pdfIdx}-${side}">${crop[side]}%</label>
 <input type="range" min="0" max="40" step="1"
value="${crop[side]}" oninput="updateTaskPdfCropValueLabel(${idx}, ${pdfIdx},
'${side}', this.value)" onchange="updateTaskPdfCrop(${idx}, ${pdfIdx}, '${side}',
this.value)">
 </div>`
).join('')}
 </div>
 <div class="task-pdf-crop-note">Crop trims the imported PNG from
each edge after insertion. Release the slider to apply it.</div>
 <div class="task-pdf-crop-actions"><button class="small-btn"
type="button" onclick="resetTaskPdfCrop(${idx}, ${pdfIdx})">Reset
crop</button></div>
 </div>` : ''}
 </div>`;
 }).join('')}
</div>` : '';

return `
 <div class="task-card" style="padding:12px">
 <div class="task-head" style="margin-bottom:8px">
 Task ${idx + 1} assets
 <div class="pill-row">
 <button class="small-btn" type="button"
onclick="setEditorAssetDestination('task_${idx}')">Target in add controls</button>
 ${images.length ? `<button class="small-btn ghost-danger" type="button"
onclick="removeTaskImage(${idx})" ${imageBusy ? 'disabled' : ''}>Remove all
images</button>` : ''}
 ${pdfImages.length ? `<button class="small-btn ghost-danger"
type="button" onclick="clearTaskPdfImages(${idx})">Clear PDF pages</button>` : ''}
 </div>
 </div>
 <div class="field-note" style="margin-bottom:10px">Add as many task images

```

as you want. If the display size is 50%, two can sit side by side; 30-35% can make a comparison row.</div>

```
<div class="field" style="margin-bottom:10px">
 <label>Display size in worksheet <span id="taskAssetWidthValue-${idx}"
class="tiny">${task.image_width || 100}%</label>
 <input type="range" min="25" max="100" step="5" value="${task.image_width
|| 100}" oninput="updateTaskImageWidth(${idx}, this.value);
syncTaskAssetWidthLabels(${idx}, this.value)">
</div>
 ${images.length ? `<div class="field"
style="margin-bottom:12px"><label>Task images</label>${imageGallery}</div>` : ''}
 ${pdfImages.length ? `<div class="field"><label>Imported PDF
pages</label>${pdfGallery}</div>` : ''}
</div>`;
};
```

```
window.updateTaskImageWidth = function(idx, value){
 const task = state.worksheet?.tasks?.[idx];
 if(!task) return;
 task.image_width = parseImageWidth(value);
```

```
['taskImageWidthValue', 'taskPdfWidthValue', 'taskAssetWidthValue', 'taskAiImageWidthV
alue'].forEach(prefix => {
 const el = document.getElementById(`${prefix}-${idx}`);
 if(el) el.textContent = `${task.image_width}%`;
});
updatePreview();
};
```

```
window.uploadEditorImage = function(){
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 const dest = qs('imageDestination')?.value || 'header';
 const input = document.createElement('input');
 input.type = 'file';
 input.accept = 'image/*';
 input.multiple = true;
 input.style.display = 'none';
 input.addEventListener('change', async () => {
 const files = Array.from(input.files || []);
 if(!files.length) return;
 let added = 0;
 for(const file of files){
 try{
 const dataUrl = await fileToDataURL(file);
 addImageToDestination(dest, dataUrl, file.name || 'Uploaded image',
file.type || 'image/png');
 added++;
 }catch(err){ console.warn('Could not read image', file.name, err); }
 }
 input.remove();
```

```

 renderEditor();
 updatePreview();
 const label = dest === 'header' ? 'worksheet header' : `task
${parseInt(dest.split('_')[1],10) + 1}`;
 showStatus(` ${added} image${added === 1 ? '' : 's'} added to ${label}.`,
'ok');
 });
 document.body.appendChild(input);
 input.click();
};

addImageToDestination = function(dest, dataUrl, name, mimeType){
 if(dest === 'header'){
 if(!Array.isArray(state.worksheet.worksheet_images))
state.worksheet.worksheet_images = [];
 state.worksheet.worksheet_images.push({ dataUrl, name, mimeType,
source:'upload', width:100 });
 } else {
 const taskIdx = parseInt(dest.split('_')[1], 10);
 setTaskImageFromDataUrl(taskIdx, dataUrl, { name, mimeType, source:'upload'
});
 }
};

window.generateEditorAssetImage = async function(){
 if(!state.worksheet) return showStatus('Generate a worksheet first.', 'warn');
 const dest = qs('imageDestination')?.value || 'header';
 const promptEl = qs('assetAiPrompt');
 const size = parseImageGenSize(qs('assetAiSize')?.value || state.imageGenSize
|| DEFAULT_IMAGE_GEN_SIZE);
 let prompt = safeClean(promptEl?.value || '');
 if(!prompt){
 if(dest === 'header') prompt = buildWorksheetHeaderImagePrompt();
 else {
 const idx = parseInt(dest.split('_')[1], 10);
 const task = state.worksheet?.tasks?.[idx];
 prompt = safeClean(task?.image_prompt || buildTaskImagePrompt(task, idx));
 }
 if(promptEl) promptEl.value = prompt;
 }
 setEditorAiBusy(true, dest === 'header' ? 'Generating worksheet asset...' :
`Generating asset for task ${parseInt(dest.split('_')[1], 10) + 1}...`);
 try{
 showStatus('Generating image asset...', 'warn');
 const result = await callOpenAIImage(prompt, size);
 if(dest === 'header'){
 if(!Array.isArray(state.worksheet.worksheet_images))
state.worksheet.worksheet_images = [];
 state.worksheet.worksheet_images.push({
 dataUrl: result.dataUrl,

```

```

 name: `worksheet-header-image-${state.worksheet.worksheet_images.length +
1}.png`,
 mimeType: result.mimeType || 'image/png',
 source: 'ai', provider: 'openai', prompt, size, width: 100
 });
 }else{
 const idx = parseInt(dest.split('_')[1], 10);
 const task = state.worksheet?.tasks?.[idx];
 if(!task) throw new Error('Choose a valid task destination first.');
```

task.image\_enabled = true;

task.image\_prompt = prompt;

task.image\_size = size;

setTaskImageFromDataURL(idx, result.dataUrl, {

name: `task-\${idx + 1}-ai-image-\${window.getTaskImages(task).length +

1}.png`,

mimeType: result.mimeType || 'image/png',

source:'ai', provider:'openai', prompt, size

});

}

renderEditor();

updatePreview();

showStatus(dest === 'header' ? 'Worksheet asset generated.' : `Image added to

task \${parseInt(dest.split('\_')[1], 10) + 1}.`, 'ok');

}catch(err){

showStatus(err.message || 'Could not generate that asset image.', 'err');

}finally{

setEditorAiBusy(false);

forceEditorRefresh();

}

};

```

const originalConfirmTaskPdfImport = window.confirmTaskPdfImport;
window.confirmTaskPdfImport = async function(){
 const session = state.taskPdfImport;
 if(!session || session.inserting) return;
 const task = state.worksheet?.tasks?.[session.taskIdx];
 if(!task) return closeTaskPdfImport();
 const selectedItems = session.pages.filter(page => page.selected);
 if(!selectedItems.length) return showStatus('Choose at least one page to
import.', 'warn');
```

try{

session.inserting = true;

renderTaskPdfImportModal();

const imported = selectedItems.map(page => {

const sourceDataUrl = page.insertDataUrl || page.thumbDataUrl;

return {

source: 'pdf', mimeType: 'image/png', dataUrl: sourceDataUrl,

originalDataUrl: sourceDataUrl,

crop: { left:0, right:0, top:0, bottom:0 },

name: `task-\${session.taskIdx + 1}-pdf-page-\${page.pageNumber}.png`,

```

 file_name: session.fileName, page_number: page.pageNumber
 });
});
task.pdf_images = [...(Array.isArray(task.pdf_images) ? task.pdf_images :
[]), ...imported];
closeTaskPdfImport();
renderEditor();
updatePreview();
showStatus(`${imported.length} PDF page${imported.length === 1 ? '' : 's'}
added to task ${session.taskIdx + 1}.`, 'ok');
} catch (err) {
 console.error(err);
 showStatus(err.message || 'Could not import those PDF pages.', 'err');
 if (state.taskPdfImport) { state.taskPdfImport.inserting = false;
renderTaskPdfImportModal(); }
}
};

const originalRenderEditor = renderEditor;
renderEditor = function() {
 if (state.worksheet?.tasks?.length) {
 state.worksheet.tasks.forEach(task => window.syncLegacyTaskImage(task));
 }
 return originalRenderEditor();
};

const originalApplySnapshot = applySnapshot;
applySnapshot = function(saved) {
 originalApplySnapshot(saved);
 if (state.worksheet?.tasks?.length) {
 state.worksheet.tasks.forEach(task => window.syncLegacyTaskImage(task));
 renderEditor();
 updatePreview();
 }
};
})();
</script>

</body>
</html>

```